22AIBI Computer Vision for Manufacturing

– Team 9 –

Nathan DiGilio: Major in CS
Jacob Dzikowski: Major in CS
Collin Miller: Major in SWE
Daniel Pora: Major in SWE

Course Instructor: Naseem Ibrahim
Faculty Advisor: Jonathan Liaw

Industry Sponsor: AIBI Solutions
Industry Mentor: Kevin DiGilio

A capstone project report submitted to the faculty of
Computer Science and Software Engineering
Pennsylvania State University, Erie

December 2025
(Version 3.00)

Table of Contents

1. Abstract 5
2. Report Revision History 6
2.1. Changes in Version 1.5 6
2.2. Changes in Version 2.0 6
2.3 Changes in Version 2.5 6
2.4 Changes in Version 3.0 6
3. Problem Statement 8
3.1. Business Background 8
3.2. Needs 8
3.3. Objectives 8
4. Initial Requirements 10
4.1. User Requirements 10
4.1.1. Glossary of Relevant Domain Terminology 10
4.1.2. User Groups 10
4.1.3. Functional Requirements 12
4.1.4. Non-functional Requirements 14
4.2. System Requirements 15
4.2.1. Functional Requirements 15
4.2.2. Non-functional Requirements 17
4.3. Requirements Trace Table 19
5. Exploratory Studies 20
5.1. Relevant Techniques 20
5.2. Relevant Packages/Products 20
5.3. Broader Impacts 20
6. System Design 21
6.1. Part and Material Tracking System 21
6.1.1. Architectural Design 21
6.1.2. Structural Design 22
6.1.3. User Interface Design 23
6.1.4. Behavioral Design 23
6.1.5. Design Alternatives & Design Rationale 23
6.2. Worker-step Verification System 24
6.2.1. Architectural Design 24
6.2.2. Structural Design 25
6.2.3. User Interface Design 27

6.2.4. Behavioral Design 28
6.2.5. Design Alternatives & Design Rationale 33
7. System Implementation 34
7.1. Programming Languages & Tools 34
7.2. Coding Conventions 35
7.3. Code Version Control 35
7.4. Implementation Alternatives & Decision Rationale 35
8. System Testing 37
8.1. Test Automation Framework 37
8.1.1. Steps for Installing Test Framework 37
8.1.2. Steps for Running Test Cases 39
8.2. Test Case Design 40
8.2.1. Test Suites 40
8.2.2. Unit Test Cases 42
8.2.3. Integration Test Cases 43
8.3. Test Case Execution Report 43
8.3.1. Unit Testing Report 43
8.3.2. Integration Testing Report 44
9. Challenges & Open Issues 45
9.1. Challenges Faced in Requirements Engineering 45
9.2. Challenges Faced in System Development 45
10. System Manuals 47
10.1. Instructions for System Development 47
10.1.1 How to Set Up a Development Environment 48
12. References 49
13. Team Members' Log Sheets 50
13.1 Nathan DiGilio 50
13.2 Jacob Dzikowski 52
13.3 Daniel Pora 54
13.4 Collin Miller 56
Appendix U 58
1. Use Case Tables 58
Appendix R 63
1. User Functional Requirements: 63
2. User Non-functional Requirements: 74
3. System Functional Requirements 81

4. System Non-functional Requirements.....95
5. Trace Table.....111
Appendix T.....118
Appendix TE.....136

1. Abstract

Manufacturing companies operating under strict compliance and government contract requirements face major challenges in maintaining traceability, efficiency, and data security. Many rely on outdated barcode systems or manual tracking, leading to inefficiencies and human error. To address these issues, AIBI Solutions, in collaboration with KMD Technology Solutions, is developing an on-premises computer vision system that automates QR code scanning and verifies manufacturing processes in real time. The system uses fixed cameras at each workstation to automatically detect and decode multiple QR codes, monitor process steps, and provide immediate feedback through an intuitive interface. Data is formatted into structured JSON outputs and integrated securely with KMD's existing workflow management system through a local API connector, ensuring full compatibility without cloud dependencies.

The project aims to increase production efficiency, reduce human error, and enhance operational transparency while maintaining data privacy and regulatory compliance. Beyond the immediate business value, the system demonstrates how artificial intelligence and computer vision can bring smart automation to secure, high stakes manufacturing environments, paving the way for broader industrial modernization.

This report documents the results of the requirements in engineering and design process. It

2. Report Revision History

2.1. Changes in Version 1.5

- Fixed the Use Case Diagram
- Removed “Senior Design Student Developer” from Key Users
- Removed excessive references to “uv”
- Re-engineered system requirements so they can be testable
- Various grammatical improvements

2.2. Changes in Version 2.0

- Removed most references to barcodes, as our project has shifted to focus solely on QR codes
- Removed SFR3, which pertained to using our system to recognize etchings in a manufactured part
- Edited SFR5 to allow keyboard emulation to be used in different applications
- Split SFR13 into 3 requirements (SFR13, SFR17, SFR18)
- Updated all requirement cards and trace table related to SFR1, SFR2, SFR13, SFR17, and SFR18
- Redesigned the architectural design diagram to accommodate new information in section 6.1
- Created an activity behavioral diagram and description to go with it in section 6.4
- Updated timesheets
- Updated requirement cards/trace tables to correspond with all changes
- Added sections 6.2, 6.3, 6.5, 7.4, 8, 9.2, and 10.1.1
- Added Appendices T and TE

2.3 Changes in Version 2.5

- Various grammatical fixes
- Rewrote the Class Diagram to improve readability
- Rewrote the Activity Diagram to fix merge point error
- Rewrote the Use Case Diagram to fix errors

2.4 Changes in Version 3.0

- Updated use cases
    - Deleted UC-004 and all mentions
    - Clarified training process
    - Other various edits
- Requirement Updates
    - Added UFR13 – 23
    - Added SFR19- 32
    - Split SFR 7 into SFR 7, 26, 27
    - Updated Trace table
    - Updated and added requirement cards

Team 9

- Split section 6 into two sections to represent both the systems: 6.1 and 6.2
    - Added all of the system designs to section 6.2
- Overhauled test cases
    - Removed redundant cases
    - Simplified overcomplicated cases
    - Added the cases for the step verification
    - Changed the test

3. Problem Statement

3.1. Business Background

AIBI Solutions was founded as an AI-focused sister company of KMD Technology Solutions, a company known for its project management software and workflow optimization tools. While KMD focused on improving digital coordination and project efficiency, AIBI was created to extend that mission into artificial intelligence, with a focus on secure, on-premises systems built for regulated industries.

The goal behind AIBI Solutions is to deliver advanced AI technology that can operate entirely within a company's local infrastructure, allowing manufacturers that work under government contracts or strict compliance regulations to take advantage of automation without compromising security. The company's first major product was a large language model–based quoting system designed to speed up and standardize the part quoting process. It reduced manual effort, increased consistency across quotes, and helped teams respond faster to customer requests while keeping all sensitive data stored on-site.

AIBI's next step is expanding into computer vision. This project focuses on building an intelligent tracking system that continuously monitors parts as they move through the facility, verifying that workers follow each step in the correct order. The goal is to bring greater transparency, efficiency, and accountability to manufacturing workflows, while maintaining strict compliance and ensuring that all processing happens locally, within the client's own secure environment.

3.2. Needs

Manufacturing companies that work under government contracts or strict compliance standards are under constant pressure to keep their production accurate, traceable, and fully accountable. Many still rely on manual tracking or outdated barcode systems that slow down operations and leave room for mistakes. At the same time, stricter data protection rules make it difficult to adopt cloud-based AI tools. These companies need solutions that can run completely on-premises, systems that deliver the speed and intelligence of modern AI without sending any data off-site.

AIBI Solutions was built to meet that need. The goal is to bring automation and insight into manufacturing environments that can't rely on traditional cloud infrastructure, combining large language models and computer vision to make production faster, safer, and more reliable.

3.3. Objectives

The objective of this phase is to develop a computer vision system that keeps track of parts and verifies that workers are completing each step in the correct order. The system will automatically monitor parts as they move through the facility, ensure each process follows the proper sequence, and alert supervisors when something is out of order. Everything will run locally to stay compliant with strict security requirements.

By doing this, the project aims to make manufacturing more transparent and efficient while reducing human error and maintaining full control of the data. The end goal is a reliable, on-premises AI solution that supports smarter decision-making and smoother production without adding more complexity to the workflow.

4. Initial Requirements

4.1. User Requirements

4.1.1. Glossary of Relevant Domain Terminology

- **AI:** Artificial Intelligence
- **API:** Application Programming Interfaces
- **GPU:** Graphical Processing Unit
- **JSON:** JavaScript Object Notation
- **LLM:** Large Language Model
- **QR:** Quick Response
- **REST:** Representational State Transfer
- **SQL:** Structured Query Language
- **UI:** User Interface

4.1.2. User Groups

1. Factory Operators (Key Users)

- **Role Responsibilities:** Operate machinery and handle parts during production. Use the vision system to scan codes and verify the correct completion of each manufacturing step. Will respond to alerts when a deviation is detected by the system.
- **User Priority:** Key User - the system's success depends on their adoption and satisfaction.
- **Subject Matter Experience:** Master - extensive experience with manufacturing procedures and workflows.
- **Technological Experience:** Novice to Journeyman - varying comfort levels with technology; most are non-technical.
- **Other Characteristics:** Adult factory workers; may have limited time for training. Interfaces must be simple, visual, and accessible, supporting varying abilities and potential physical constraints.

2. Production Supervisors (Key Users)

- **Role Responsibilities:** Monitor workstation performance, review system logs, validate that steps are followed in sequence, and address reported process violations.
- **User Priority:** Key User - their input is essential for defining operational requirements and ensuring the system fits daily workflow oversight.
- **Subject Matter Experience:** Master - deep understanding of production flow and quality control.
- **Technological Experience:** Journeyman - familiar with digital dashboards and manufacturing data systems.
- **Other Characteristics:** Prefer concise, visual feedback and reliable alerts. May need multi-station access for monitoring.

3. AIBI Integration Engineers (Secondary Users)

- **Role Responsibilities:** Install, configure, and maintain the vision system; ensure integration with KMD Technology Solutions workflow management software.
- **User Priority:** Secondary User - critical during deployment and maintenance but less involved in daily use.
- **Subject Matter Experience:** Journeyman to Master - skilled in AI and computer vision systems.
- **Technological Experience:** Master - advanced technical skills
- **Other Characteristics:** Engineering background; focused on performance and reliability metrics rather than user experience.

4. KMD Technology Solutions Software Team (Secondary Users)

- **Role Responsibilities:** Oversee integration between the AIBI vision module and KMD's existing workflow management platform; validate data exchange and API compatibility.
- **User Priority:** Secondary User.
- **Subject Matter Experience:** Master - expert knowledge of KMD systems and factory workflow data structures.
- **Technological Experience:** Master - proficient in API development and system architecture.
- **Other Characteristics:** Focused on data consistency and long-term interoperability requirements.

5. AIBI Solutions Management and Client Representatives (Secondary Users)

- **Role Responsibilities:** Review reports, set project goals, and evaluate system performance for business impact and compliance with security requirements.
- **User Priority:** Secondary User - stakeholders for project success and regulatory alignment.
- **Subject Matter Experience:** Journeyman - familiar with manufacturing operations and AI deployment constraints.
- **Technological Experience:** Journeyman - comfortable with analytics dashboards and reporting tools.
- **Other Characteristics:** Interested in ROI, reliability, and compliance.

4.1.3. Functional Requirements

4.1.3.1. Project Scope (Use Case Diagram)

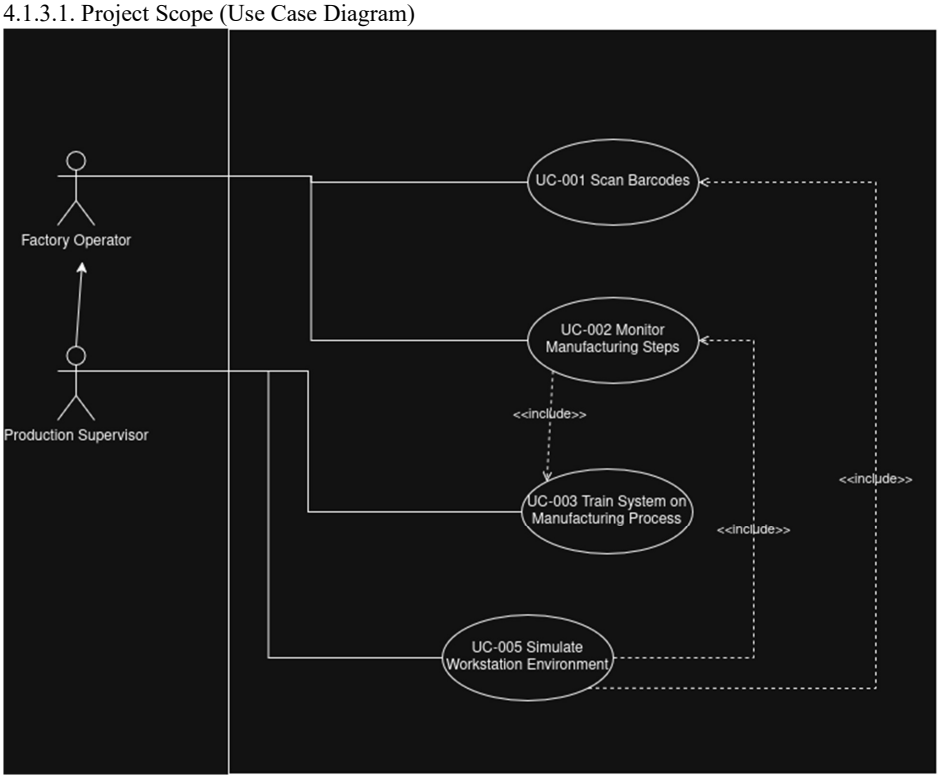

Figure 4.1 Use Case Diagram

The Use Case diagram in Figure 4.1 consists of 2 actors: “Factory Operator” and “Production Supervisor”. “Factory Operator” is the primary actor for UC-001 and UC-002. UC-002 includes the functionality of UC-003, which trains the system. This is required for the model to work. UC-003 will have the “Production Supervisor” as the main actor. “Production Supervisor” is responsible for training the system so it can monitor the steps properly, and also has access to the Violation Log which takes the functionality of UC-002 to get the errors.

4.1.3.2. User Scenarios

UC-001: Scan Part QR codes

- A factory operator places paper QR codes within the camera's field of view. The system automatically detects and decodes all visible QR codes, maps them to configured data fields, and outputs structured JSON data to the downstream system. If QR codes fail to decode, the system alerts the operator.

UC-002: Monitor Manufacturing Step

- A factory operator performs manufacturing steps while the system continuously monitors the video feed. The system detects when each step begins and ends, validates the correct sequence, and logs completion with timestamps. If a step is performed out of sequence or skipped, the system displays alerts and prevents progression until corrected.

UC-003: Train System on Manufacturing Process

- A production supervisor records a reference video of an experienced operator performing a new manufacturing process. The supervisor marks the start and end of each step, labels them, and adds notes. The system saves the training session with metadata for use in future process verification.

UC-005: Simulate Workstation Environment

- A developer loads recorded camera footage into the simulation environment and injects synthetic QR code data to test system functionality. The system processes the simulated inputs, generates outputs, and displays verification results. The developer validates system behavior before production deployment.

4.1.3.3. User Functional Requirements

UFR1. Fixed cameras at each workstation must detect, decode, and process multiple QR codes from objects presented within the camera's field of view

UFR2. Provide a configurable mapping system to assign QR code positions to specific data fields

UFR3. Identify and track QR codes on large parts rather than sheets of paper

Commented [MC1]: We can probably remove this one

UFR4. Aggregate all decoded values into a structured JSON payload for downstream system integration

UFR5. Monitor and validate that manufacturing steps are performed in the correct sequence at each workstation

UFR6. Provide immediate visual feedback and alerts for process status and violations on terminals at workstations

UFR7. Log all errors caught by the system for the users to review

UFR8. Provide an intuitive UI for non-technical users to train the system on correct manufacturing processes

UFR9. Seamless integration with the existing KMD Technology Solutions workflow management system

Commented [MC2]: Review this Requirement

UFR10. Implement intermediate data storage to decouple computer vision processing from KMD data consumption

UFR11. Provide a simulated workstation environment for development and validation before production deployment

UFR12. Provide tools for capturing, organizing, and managing training datasets for model development

UFR13. Show all errors in the process on screen

UFR14. Show all steps for the process on screen

UFR15. Have a visual marker for completed or missed steps

UFR16. Click on check box to show the step

UFR17. Admin function for the in-site capture tool

UFR18. Pass in what workstation when running the tool

UFR19. Command line control

UFR20. Provide a reset button for transition to the next part

UFR21. Have configuration for the number of QR codes expected

UFR22. Have configuration for the direction of which the user wants them to be scanned.

UFR23. Have configuration for if they want to add a character at the end of the QR data.

4.1.4. Non-functional Requirements

4.1.4.1. Product: Usability Requirements

PUUR1. The System must be extremely easy to use for manufacturing workers with minimal training, and it must be easier than what is currently being done

PUUR2. Minimize physical interaction requirements during normal operation

PUUR3. Training and configuration interfaces must be accessible to non-technical staff

PUUR4. Design a system to accommodate workers with varying technical skills and possible disabilities

4.1.4.2. Product: Performance Requirements

PPUR1. Process video feeds and provide feedback with minimal latency to prevent manufacturing errors

PPUR2. Maintain high accuracy in both QR code detection and process monitoring for aerospace quality standards

PPUR3. Function effectively with lower-quality, cost-effective camera equipment

4.1.4.3. Product: Dependability/Reliability/Security

PDUR1. All processing and data storage must remain on-premises with no cloud dependencies or external services

PDUR2. Reliable operation in industrial factory conditions with minimal downtime

4.1.4.4. Organizational: Development Requirements

ODUR1. Utilize open-source libraries and frameworks to minimize licensing costs

4.1.4.5. Organizational: Operational Requirements

OOUR1. Minimize hardware and deployment costs while meeting performance requirements

OOUR2. Support flexible deployment configurations for different workstation setups

OOUR3. Support gradual implementation with minimal disruption to existing operations

4.1.4.6. External: Interoperability Requirements

EIUR1. Maintain compatibility with existing KMD Technology Solutions software interfaces

EIUR2. Publish a comprehensive JSON schema for scan results and system outputs

EIUR3. Support integration with existing manufacturing systems and processes

4.2. System Requirements

4.2.1. Functional Requirements

4.2.1.1. System Functional Requirements

SFR1. Fixed-station cameras shall detect all QR codes visible in a single frame

SFR2. Provide per-workstation field mapping to assign which fields the camera must detect from the QR code.

SFR4. Aggregate decoded and recognized values into a structured JSON document per scan event

SFR5. Support real-time delivery via keyboard-emulation mode

SFR6. Detect and report start/completion of defined manufacturing steps and validate configured sequence; flag skipped or out of sequence steps

SFR7. Provide per workstation UI overlay with live video

SFR8. Provide an in-site capture tool for non-technical users to record reference video, mark step start/end, label steps, and save training sessions with metadata

SFR9. Models must tolerate natural worker variations and provide configurable sensitivity per step to reduce nuisance alerts

SFR10. The system shall log every detected incorrect, skipped, or out of sequence manufacturing step and save it for review

SFR11. Persist scan and event data locally with audit metadata and support replay/synchronization when downstream systems are unavailable

SFR12. Publish a stable JSON schema and human readable documentation for scan and event payloads including examples and required fields

SFR13. Provide a simulated workstation environment to replay camera feeds

SFR14. Provide tools and storage to capture, tag, version, and export labeled datasets for model training, including metadata

SFR15. Fixed-station cameras shall decode all QR codes visible in a single frame

SFR16. Fixed-station cameras shall output all QR codes visible in a single frame

SFR17. Provide a simulated workstation environment to inject synthetic QR codes

SFR18. Provide a simulated workstation environment to verify keyboard-emulation outputs before production deployment

SFR19. Provide a message to the user for all errors in the step processing on screen

SFR20. Provide a list of all steps in the process on the UI

SFR21. Provide a visual marker in the form of a green checkmark to indicate if a step was completed.

SFR22. Provide a visual marker in the form of a red "X" to indicate if a step was missed.

SFR23. Click on the step in the list to view recorded footage of the step

SFR24. Implement Administrative functionality for the in-site capture tool

SFR25. Integrate command line control for the users.

SFR26. Provide per workstation UI overlay with step progress indicators

SFR27. Provide per workstation UI overlay with optional audio alerts for violations

SFR28. Provide a reset key to allow the program to get ready for the next part

SFR29. Provide configuration for the user to specify how many QR codes are to be expected in the scanning process.

SFR30. Provide configuration for the user to specify what direction they want the QR codes to be scanned.

SFR31. Provide configuration for the user to specify if they would like to add a character at the end of the QR data.

SFR32. Provide the ability for the user to enter the workstation that the program starts at

4.2.1.2. Data Requirements

The system depends on a combination of real-time camera data, QR code information, and process-tracking records to support its monitoring and verification functions. Each data source must be handled securely and consistently to ensure accuracy and reliability in a manufacturing environment.

The main input data comes from live video streams captured by cameras at each workstation. These frames are processed locally on edge devices to identify QR codes, track parts, and verify worker actions. Each frame includes a timestamp and workstation ID to maintain synchronization and traceability across multiple cameras.

All data is stored securely on the local network and managed according to the manufacturer's privacy and security policies. Sensitive information, such as worker identifiers or connection credentials, is protected using encrypted .env files and role-based access control. These data requirements ensure that the system maintains accurate and traceable records of all part and process events, supporting efficient monitoring, auditing, and future scalability.

4.2.2. Non-functional Requirements

4.2.2.1. Product: Usability Requirements

PUSR1. The UI shall be intuitive and require no more than 30 minutes of training for first time users

PUSR2. The system shall provide clear on-screen guidance and tooltips for all major functions

PUSR3. The UI shall minimize the amount of effort required to complete required tasks compared to the existing process

PUSR4. The system shall support touch-free operation through automatic detection and feedback mechanisms where possible

PUSR5. The system shall automate common workflow steps to reduce manual input from operators

PUSR6. The configuration interface shall use plain-language prompts and visual menus, avoiding technical jargon

PUSR7. The system shall include a guided setup and tutorial modes for initial configuration

PUSR8. The system shall allow non-administrative users to modify basic settings without needing programming knowledge.

PUSR9. The interface shall include support for large text and high-contrast display options

PUSR10. The UI shall use simple iconography, color coding, and minimal text to accommodate diverse language proficiencies and provide clear workflows for non-technical staff

PUSR11. The system shall comply with accessibility standards

PUSR12. The system shall support both mouse and keyboard input/navigation for inclusivity

4.2.2.2. Product: Performance Requirements

PPSR1. The system shall process video input and deliver analysis feedback within 200 milliseconds

PPSR2. The system shall prioritize real-time operations over background tasks to minimize latency

PPSR3. The system shall achieve a QR code detection accuracy of at most one error per 100 scans.

PPSR4. The system shall detect and report process deviations with an error margin of less than 1%

PPSR5. The system shall maintain required accuracy levels using cameras with resolutions as low as 720p

PPSR6. The system shall automatically adjust detection parameters for varying camera quality and lighting conditions

4.2.2.3. Product: Dependability/Reliability/Security

PDSR1. The system shall store all data on local servers within the facility

PDSR2. The system shall perform all processing operations using on-premises hardware without cloud reliance

PDSR3. The system shall maintain a minimum uptime of 99% during operational hours

PDSR4. The system shall be tested for stable operation under factory environmental conditions

4.2.2.4. Organizational: Development Requirements

ODSR1. The system shall use open-source libraries or frameworks unless a commercial dependency is explicitly approved

ODSR2. All open-source components shall be documented, including their licenses and version numbers

4.2.2.5. Organizational: Operational Requirements

OOSR1. The system shall maintain performance benchmarks on standard hardware

OOSR2. The installation process shall include customizable deployment options

OOSR3. The system shall support phased deployment alongside existing systems

OOSR4. The system shall maintain backward compatibility with legacy data formats

4.2.2.6. External: Interoperability Requirements

EISR1. The system shall integrate with all existing KMD Technology Solutions APIs and communication protocols

EISR2. The system shall maintain consistent data exchange formats used by KMD

EISR3. The system shall define and publish a complete JSON schema for all output data

EISR4. The JSON schema shall be version-controlled and included in system documentation

EISR5. The system shall support data exchange with existing Manufacturing Execution Systems and ERP tools

EISR6. The system shall be configurable to align with existing factory workflows and data pipelines

4.3. Requirements Trace Table

The trace table, at the bottom of Appendix R, has all the mappings from user to system requirements. The table is used as a reference to see what system requirements were engineered to meet system requirements.

5. Exploratory Studies

5.1. Relevant Techniques

The exploratory phase focused on identifying computer-vision techniques that could operate reliably in real manufacturing environments without constant retraining or cloud support. Because each facility may use different layouts, materials, and workflows, the system needed to be flexible, lightweight, and fully on-premises.

For part and material tracking, traditional vision methods proved the most effective. QR decoding libraries such as pyzbar [1] and ZXing [2] were evaluated as dependable solutions for linking visual data to unique part identifiers. These techniques provide precise and verifiable tracking of materials as they move through each stage of production, forming the backbone of the system's traceability layer.

For worker-step verification, a more adaptive and semantic approach was needed. The project explored OpenCLIP [3] and MobileCLIP [4], which generate multimodal embeddings capable of connecting visual input to textual task descriptions. By comparing live camera frames against step definitions or reference images encoded in the same embedding space, the system can determine whether the correct procedure is being followed without requiring model retraining for each new process.

This hybrid strategy, symbolic tracking for materials, semantic understanding for actions, allows the system to remain both accurate and flexible. Edge optimization and model quantization were also studied to ensure that OpenCLIP [3] can operate efficiently on local GPU hardware, maintaining real-time performance while keeping all data processing securely on-site.

5.2. Relevant Packages/Products

Several open-source frameworks and libraries were examined during the exploratory phase. OpenCV [5] remained the foundation for video capture, preprocessing, and visualization. OpenCLIP [3] was integrated through the PyTorch [6] ecosystem for embedding generation and similarity matching. Though we cannot use the MobileCLIP [4] model itself as it falls under a limited research license. FastAPI [7] was reviewed for real-time data exchange between the vision modules, the LLM-based reasoning system, and the operator dashboard. Data management and analysis relied on NumPy [8], Pandas [9], and SQL databases to record event logs and validation results. Package management tools such as uv [10] and conda [11] were evaluated for reproducible, isolated deployments suitable for regulated installations.

5.3. Broader Impacts

Using OpenCLIP [3] expands what's possible for on-premises AI in manufacturing. These models allow semantic understanding of human actions and part interactions without requiring cloud-scale datasets or manual labeling, opening the door to more adaptive and intelligent factory systems.

In the long term, this technology can improve safety by recognizing incorrect procedures in real time, support training by providing immediate visual feedback, and streamline audits by automatically logging validated process steps. More broadly, it demonstrates how advanced AI can be localized, offering powerful visual reasoning while preserving data ownership, privacy, and compliance.

6. System Design

6.1. Part and Material Tracking System

6.1.1. Architectural Design

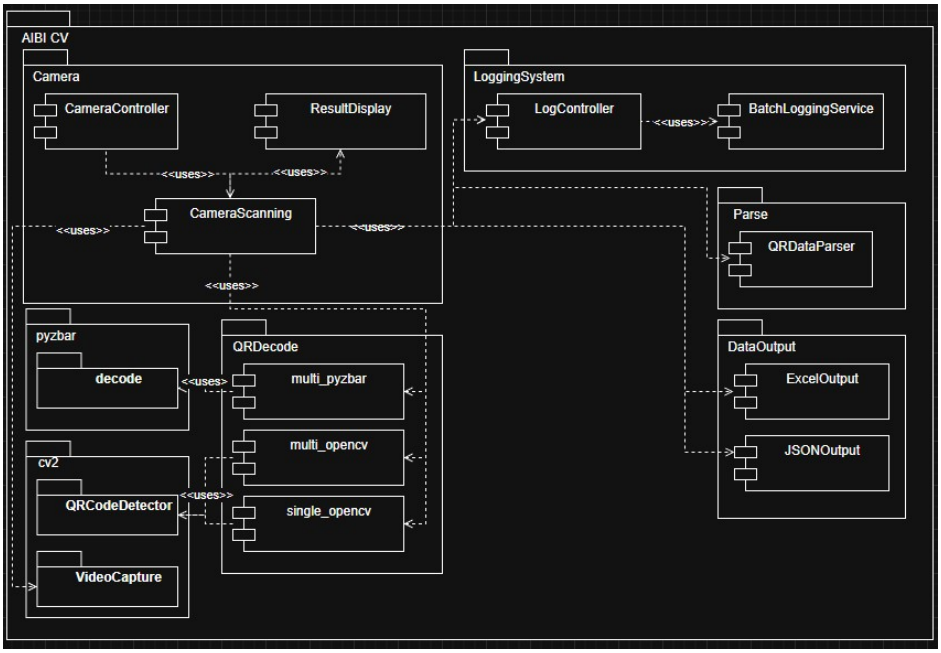

Figure 6.1.1 UML Package Diagram

The UML package diagram presented in Figure 6.1.1 illustrates a high-level view of the system architecture. The camera package consists of three primary modules: CameraController, CameraScanning, and ResultsDisplay. The CameraController initializes and utilizes the CameraScanning module, which is responsible for orchestrating the complete process of frame detection, decoding, and parsing.

The QRDecode package uses three strategies for detecting and decoding QR codes. The system first attempts to read multiple codes using the pyzbar decode library. If this method fails, it

proceeds to the cv2.QRCodeDetector package. This detector is applied using two techniques: it initially attempts to scan multiple QR codes simultaneously; if unsuccessful, it falls back to detecting only the first code found.

Decoded results are passed to the Parse package, where they are converted into usable objects. Once parsing is complete and all required data has been captured, the output is generated either as an Excel file or as a JSON file, depending on the workstation configuration. During camera operation, live scan results are displayed via the ResultsDisplay module. Throughout the scanning process, the CameraScanner continuously logs system data, including detected results and captured exceptions, to a log file.

6.1.2. Structural Design

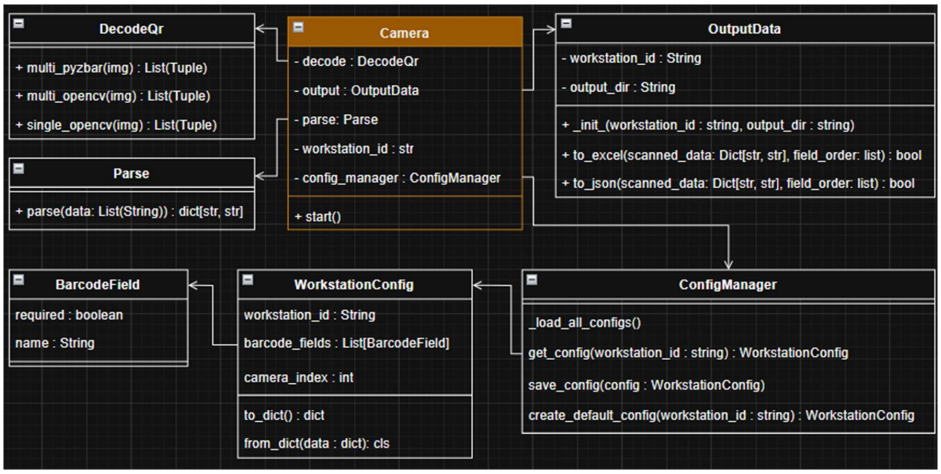

Figure 6.1.2 UML Class Diagram

The UML class diagram in Figure 6.1.2 highlights the QR code scanning part of our project. The designed diagram makes it easier to debug, test, and maintain. This design uses dependency injection to support loose coupling.

The general flow of the diagram is the following. The camera starts up with a config based on the workstation_id. Then it will continually attempt to scan and decode the QR codes when they appear on the screen. It will then take the required data and any optional data if it has it and then parse that data into a JSON format. Lastly, it will try to output to Excel or create a JSON file with the data if Excel output fails.

The config is loaded from the ConfigManager that gets it from the WorkstationConfig. Having it this way, if there is a need for any other types of configs, then they can simply be added into a new class and then added to ConfigManager.

6.1.3. User Interface Design

6.1.4. Behavioral Design

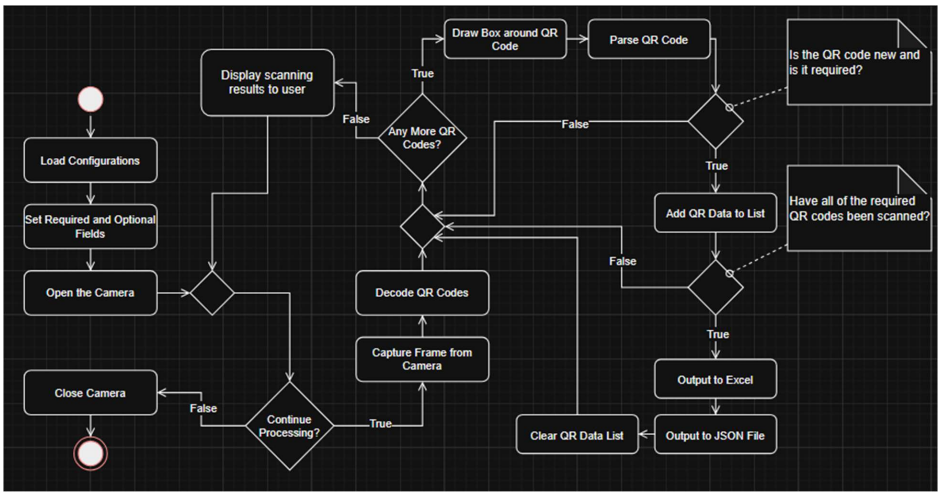

Figure 6.1.4 UML Activity Diagram

The UML activity diagram in Figure 6.1.4 illustrates a high-level overview of the system's behavior. When the system starts, it first loads all configuration and environment variables and sets both the required and optional fields. The camera is then initialized, and frame processing begins. At this point, the system enters its main loop, continuously scanning for QR codes until the user stops the program.

During each iteration of the main loop, a frame is captured from the camera and processed to detect and decode any QR codes present. If one or more QR codes are found, the system enters a secondary loop to iterate through each detected code. For every code, the system displays its contents to the user and then parses it. After parsing, the system checks whether the code has already been scanned and whether it is required. If both conditions are met, the code is added to the tracked list.

When all required codes have been successfully scanned, the system outputs the results to both an Excel file and a JSON file. It then clears the list of scanned codes so the process can begin again. Once all codes are captured, the final results are displayed to the user.

6.1.5. Design Alternatives & Design Rationale

For the architectural view, we considered several diagram options, including deployment diagrams, module diagrams, and component diagrams. We ultimately chose to use a module diagram because it aligns most closely with how our system is structured. Our codebase is

organized into larger functional modules rather than the smaller, fine-grained pieces typically represented in a component diagram. Additionally, a deployment diagram was not necessary, as our system does not involve a complex distributed environment or multiple hardware nodes. The module diagram effectively captures the toolbox-style organization of our system, where each module groups together related functionality in a clear and manageable way.

For our behavioral diagram, we chose to use an activity diagram because it best represents the overall flow of our system. The alternatives we considered were state diagrams and sequence diagrams. We decided against using a state diagram because our system does not have clearly defined or meaningful states, making it a poor fit for illustrating the system's behavior. We also chose not to use a sequence diagram because our system does not rely on complex interactions between multiple objects or components; the process is largely linear and centered around a single continuous workflow rather than detailed message exchanges. An activity diagram provides a simpler and more intuitive representation of this workflow, making it the most suitable choice for our system.

6.2. Worker-step Verification System

6.2.1. Architectural Design

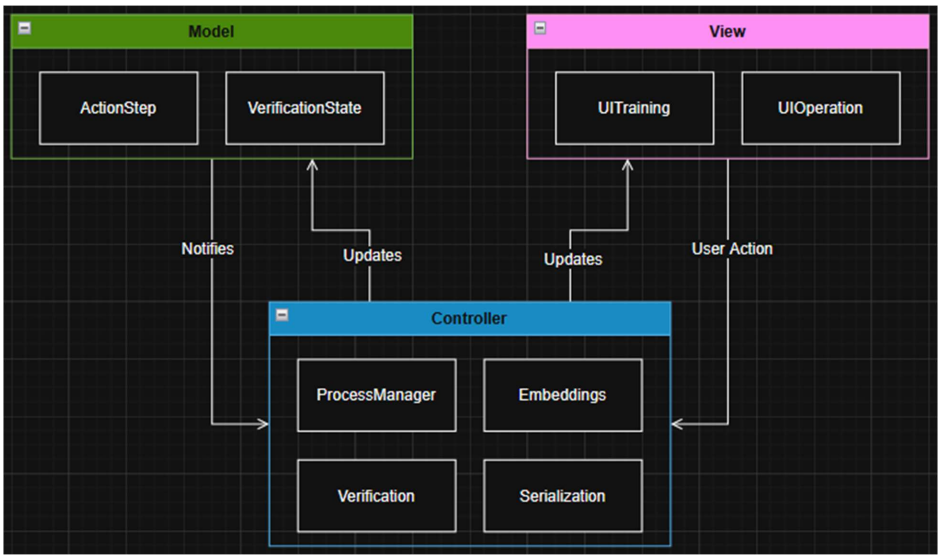

Figure 6.2.1 Architectural Design Diagram

architectural design presented in Figure 6.2.1 illustrates the high-level structure of the system, which utilizes the **Model-View-Controller (MVC)** pattern to ensure a clean separation of concerns. This design choice decouples the internal representation of information from the ways

it is presented to and accepted from the user, allowing for a more modular and maintainable codebase.

In this implementation, the **Model** layer manages the core data and logic. It is comprised of the ActionStep class, which encapsulates data for individual process steps such as sequence order and centroids, and the VerificationState, which tracks the operational status of the system as detailed further in Section 6.2.4.

The **View** layer handles the graphical user interface, encompassing both the **UITraining** and **UIOperation** phases to provide the necessary visual context for user interaction. Acting as the intermediary, the **Controller** processes incoming user actions from the View and interacts with the Model to update data or trigger system logic. It houses critical services including serialization for persistent storage, embedding and verification processes, and the **ProcessManager**, which orchestrates these components to ensure cohesive system behavior.

6.2.2. Structural Design

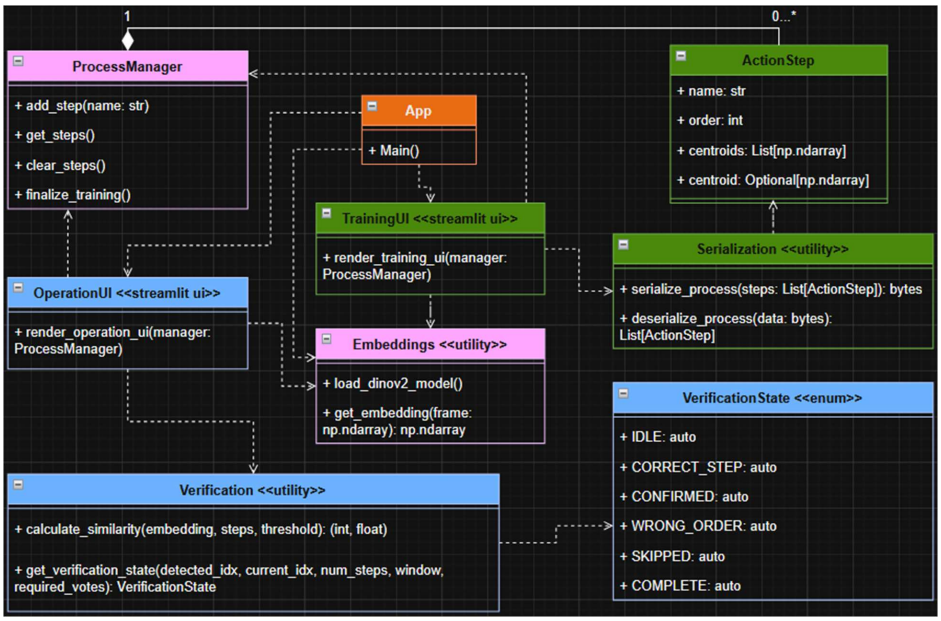

Figure 6.2.2 UML Class Diagram

The UML class diagram presented in Figure 6.2.2 provides a detailed structural view of the system's components, outlining the specific classes, their attributes, methods, and

interdependencies. At the structural core of the application is the App class, which serves as the main execution entry point and initializes the foundational utilities and user interfaces.

Acting as the primary controller, the ProcessManager orchestrates the application's core workflow. It manages a one-to-many relationship with the ActionStep class, providing methods to add, retrieve, and clear steps, as well as finalize the training sequence. The ActionStep class functions as the primary data model, encapsulating necessary metadata for each step, including its string identifier, integer sequence order, and numerical centroid representations stored as NumPy arrays.

The system's presentation layer is implemented using Streamlit and is distinctly divided into the TrainingUI and OperationUI classes. Both interface components take the ProcessManager as a dependency to render their specific views. The TrainingUI coordinates the initial setup phase, interfacing with the Serialization utility to convert lists of ActionStep objects to and from byte streams for persistent storage. Meanwhile, OperationUI drives the live execution phase, relying heavily on real-time data processing.

To handle complex backend operations, the system employs several specialized utility classes. The Embeddings utility is universally accessed across the application to load the DINOv2 machine learning model and generate vector embeddings from image frames. During the operational phase, the Verification utility calculates similarities between these real-time embeddings and the stored step data based on defined thresholds. This verification process ultimately resolves to a VerificationState enumeration, which defines discrete operational statuses—such as IDLE, CORRECT_STEP, WRONG_ORDER, SKIPPED, and COMPLETE—allowing the system to accurately track and respond to the user's progress.6.2.3. User Interface Design

6.2.3. User Interface Design

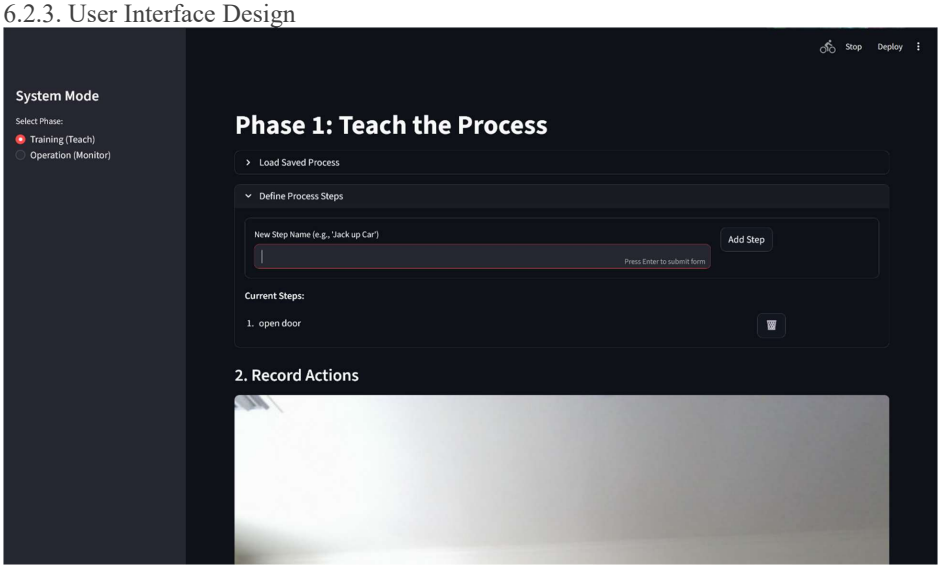

Figure 6.2.3.1 UI-Design

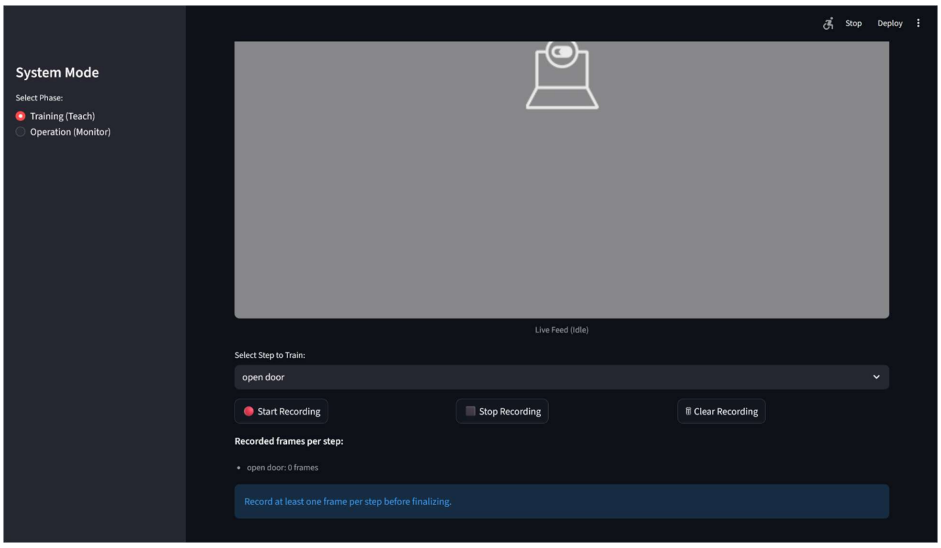

Figure 6.2.3.2 UI-Design

The pictures above (figures 6.2.3.1 and 6.2.3.2) show the process definition and training process for our system. This will allow the user to input as many manufacturing steps as they need and then

move on to train the system on what the steps should look like. The user will click “Start Recording” and then perform the task. Once they recorded as many frames as they wanted, they could then click stop recording. They can repeat that for as many steps as they have.

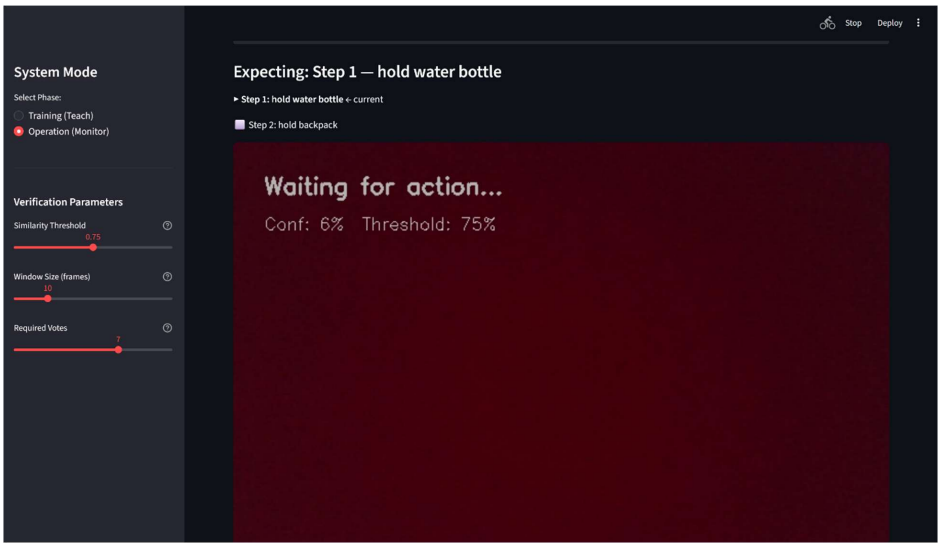

Figure 6.2.3.3 UI-Desing

This is the second part of the system user interface. Here, the user will be able to start step detection. The UI will show the camera feed as well as the step you are working on, if you missed one, or if you are incorrect order. The user can also modify some verification parameters.

6.2.4. Behavioral Design

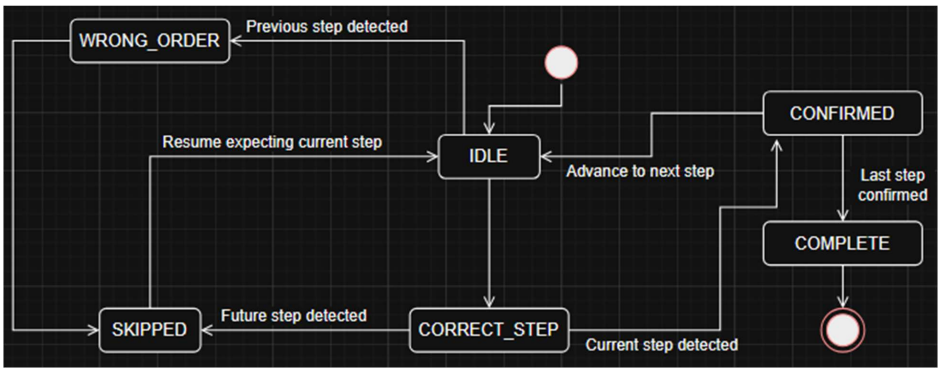

Figure 6.2.4.1 UML State Diagram

The UML state machine diagram presented in Figure 6.2.4.1 illustrates the dynamic behavior of the system's operational phase, specifically mapping the lifecycle and transitions of the VerificationState enumeration. Execution begins at an initial entry node and immediately transitions into the IDLE state. This state serves as the central resting phase and primary hub while the system actively monitors incoming data to verify user actions against the expected process sequence.

From the IDLE state, the system evaluates real-time detections and can branch into several distinct paths based on user behavior. If the system detects an action associated with a previously completed step, it shifts into the WRONG_ORDER state to flag the regression. Conversely, if it identifies an action belonging to a future sequence, it transitions to the SKIPPED state. The system is designed to recover from these sequence deviations, automatically looping back to the IDLE state to "resume expecting the current step" once the user corrects their workflow.

The primary success path is triggered when the expected action is recognized, moving the system from IDLE into the CORRECT_STEP state. Upon further validation that the action was held or completed successfully, it progresses to the CONFIRMED state. At this critical juncture, the system evaluates the overall process progress. If there are remaining steps in the workflow, it triggers an "advance to next step" transition, returning to the IDLE state to monitor for the subsequent action. However, if the confirmed action was the final requirement, the system transitions to the COMPLETE state ("last step confirmed"), ultimately reaching the final termination node and successfully concluding the operational cycle.

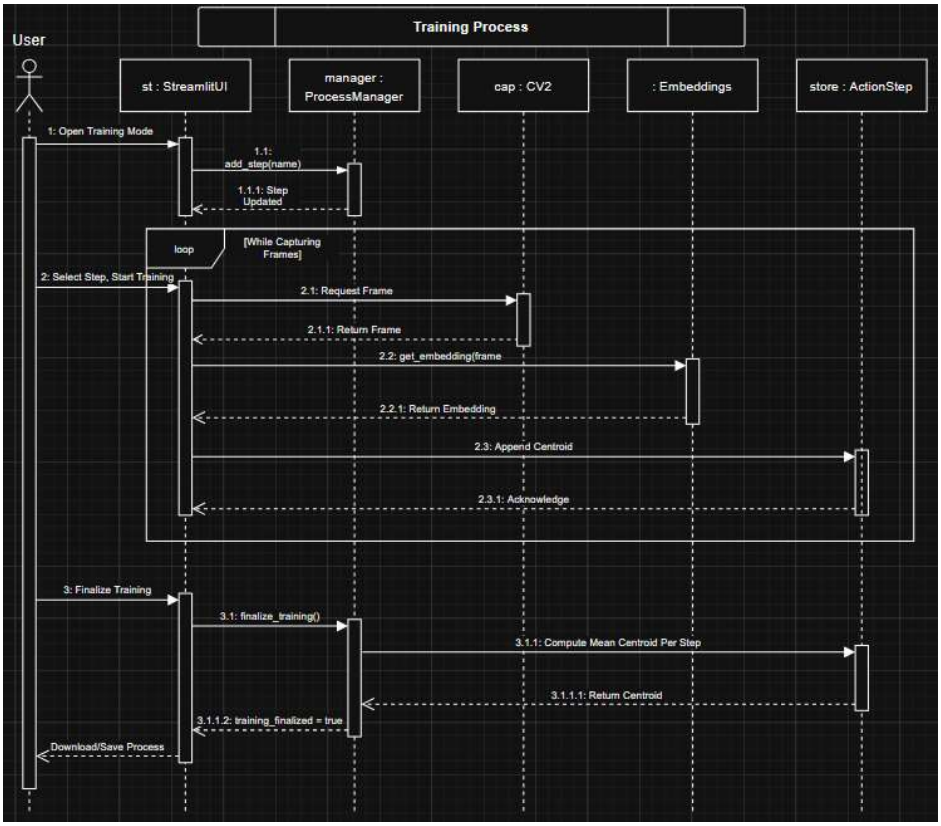

Figure 6.2.4.2 UML Sequence Diagram for Training

The UML sequence diagram presented in Figure 6.2.4.2 details the chronological interactions between the user and the system's components during the training phase. The workflow initiates when the user opens the training mode through the Streamlit graphical interface (StreamlitUI). To begin defining a new procedure, the interface prompts the ProcessManager to register a new step via the add_step(name) method, which subsequently acknowledges that the step has been successfully updated in the system's tracking logic.

Once the initial setup is complete, the system enters the core data capture loop. After the user selects a specific step and starts the training execution, a continuous cycle begins while the system captures video frames. For each iteration, the user interface requests a frame from the computer vision utility (CV2). Upon receiving the raw frame data, the interface immediately passes it to the Embeddings utility using the get_embedding(frame) method. The utility processes the image and returns a numerical vector representation, which is then sent to the

ActionStep store to be appended as a centroid. The data store acknowledges the successful addition of the centroid, and this cycle repeats continuously until the frame capture phase is halted by the user.

The process concludes when the user explicitly chooses to finalize the training. The interface triggers the finalize_training() method within the ProcessManager. The manager then directs the ActionStep store to compute the mean centroid across all the individual embeddings collected during the capture loop for that specific step. Once the averaged centroid is calculated and returned, the manager signals to the user interface that the training process is officially finalized. As a final action, the system prompts the user to download or save the newly trained process, securing the data for future operational use.

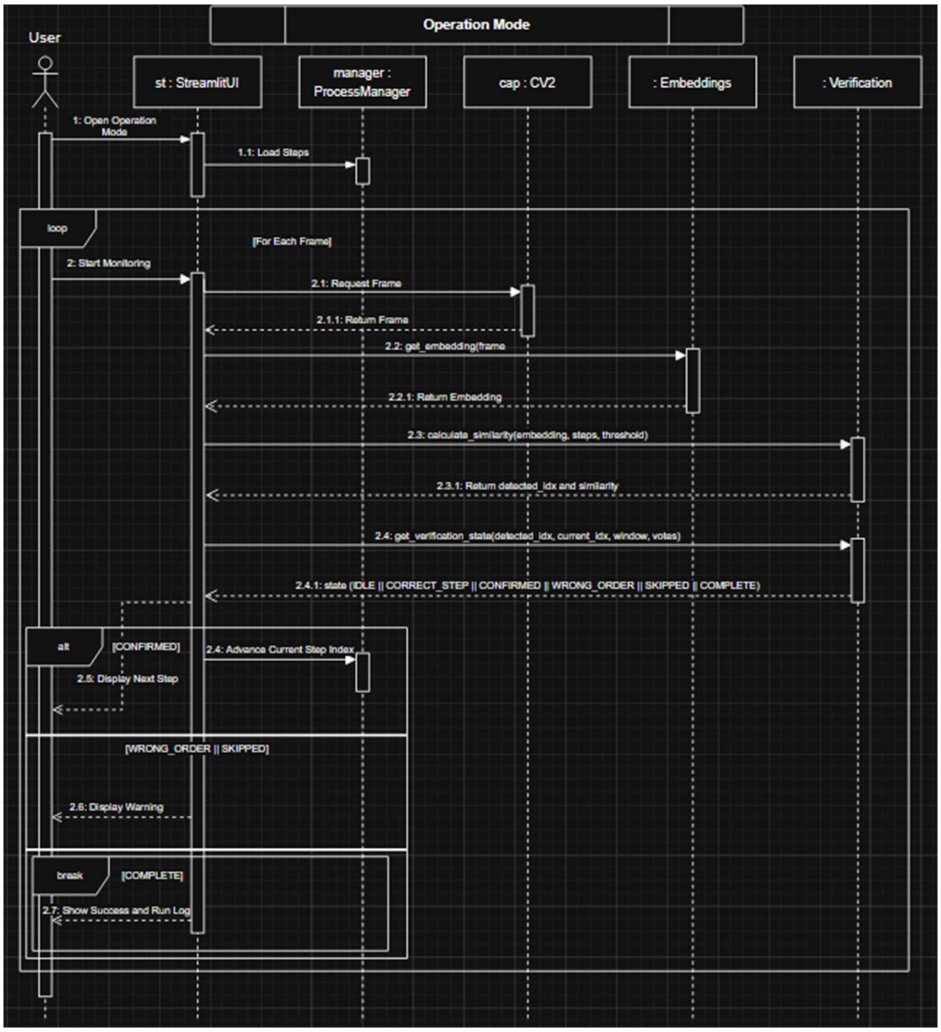

Figure 6.2.4.3 UML Sequence Diagram for Operation

The UML sequence diagram presented in the final figure details the chronological interactions between the user and the system's components during the live operation phase. The sequence initiates when the user accesses the Operation Mode through the Streamlit graphical interface. Before active monitoring begins, the interface prompts the ProcessManager to load the previously trained procedure steps into active memory, ensuring the system has the necessary baseline data for comparison.

Once the user triggers the monitoring process, the system enters a continuous, frame-by-frame execution loop. For each iteration, the user interface requests a live video frame from the computer vision utility (CV2). This raw visual data is immediately routed to the Embeddings utility, which processes the image and returns its corresponding numerical vector embedding. The interface then hands this real-time embedding, along with the loaded reference steps and a predefined similarity threshold, over to the Verification utility. The verification component performs a mathematical comparison, returning the index of the most probable detected step along with its similarity score.

Following this similarity calculation, the interface queries the Verification utility a second time to determine the current operational status using the get_verification_state() method. This method evaluates the detected index against the current expected index over a defined voting window, ultimately returning a specific state enumeration. The system's subsequent behavior dynamically branches based on this returned state. If the state evaluates to CONFIRMED, the interface instructs the ProcessManager to advance the internal step index and updates the user display to prompt the next expected step. If the system detects a sequence deviation, resulting in a WRONG_ORDER or SKIPPED state, it immediately surfaces a visual warning to the user. Finally, if the evaluation returns a COMPLETE state, the system breaks the continuous capture loop, presents a final success message and run log to the user, and successfully terminates the monitoring sequence.

6.2.5. Design Alternatives & Design Rationale

For the architectural view, we decided to represent our system through an MVC model. Since our system has well defined models, views, and controllers this seemed like the best choice. Other options included MVP (Model-View-Presenter) and MVVM (Model-View-ViewModel). However, we found that MVP would have introduced unnecessary complexity by requiring a one-to-one mapping between the View and the Presenter, which felt too rigid for our UI requirements. Similarly, while MVVM is excellent for two-way data binding, its reliance on a 'ViewModel' felt redundant given the direct controller-to-view flow we established for handling user requests and data updates. Ultimately, MVC offered the best balance of separation of concerns, allowing us to maintain a lightweight frontend while keeping the business logic and data validation centralized within the controllers.

For the behavioral diagrams, we chose to use both state and sequence diagrams because it best represents the overall flow of our system. The alternatives we considered were use case diagrams and activity diagrams. We decided against using an activity diagram since it’s representation is too simple and high level to accurately represent the complex portions of our system. The sequence diagram was the better choice for this since our has system complex interactions between multiple objects and components. A use case diagram focuses more on the users interaction with the system rather than the result of the user’s actions within the system. We decided to use a state diagram because our system does have clearly defined and meaningful states, making it a great fit for illustrating the system’s behavior at a mid to high level.

7. System Implementation

7.1. Programming Languages & Tools

The system is developed using Python and modern open-source tools optimized for on-premise deployment, reproducibility, and performance. Python was chosen for its rich ecosystem of computer vision, machine learning, and systems integration libraries, as well as its readability and flexibility for both research and production environments.

Primary Language

- **Python 3.10+**: Core language for all image processing, barcode decoding, and logic components. Python's mature libraries support rapid development and integration with existing AI toolkits.

Supporting Tools and Libraries

- **OpenCV [5]**: Provides video-frame capture, image preprocessing (contrast enhancement, thresholding, edge detection), and visualization.
- **pyzbar [1] / ZXing [2]**: Enables decoding of standard barcodes and QR codes from image frames.
- **open_clip [3]:** Constructs embeddings of the current image, which can be used in an algorithm for our worker-step verification system
- **NumPy [8] / Pandas [9]**: Used for numerical operations, data analysis, and structured logging of scan events.
- **FastAPI [7]**: Lightweight web framework for creating administrative dashboards, REST endpoints, and configuration interfaces.
- **SQL Server**: Database back end for storing scan results, process validation data, and system configuration. Existing systems already use SQL Server, and are currently running on an on-premise server.
- **Uv [10]**: Used for environment and dependency management. Chosen for its speed, reproducibility, and ability to handle isolated Python environments more efficiently than pip or Poetry.
- **Git + GitHub**: Source control and collaboration platform for version tracking, documentation, and code review.
- **Ubuntu Server 22.04 LTS**: Target deployment environment, ensuring compatibility, long-term support, and security for on-premise installation.
- **VS Code**: Primary IDE for development and debugging.

Optional / Future Tools

- **Dynamsoft Barcode SDK**: Considered for enhanced barcode decoding robustness in poor lighting or complex conditions.

- **Docker (TBD)**: May be adopted for containerized deployment and easier replication across multiple workstations.

This toolchain emphasizes efficiency, security, and flexibility. Using Python and OpenCV [5] enables modular, camera-driven computer vision workflows that align with industry standards.

7.2. Coding Conventions

To maintain a consistent, readable, and maintainable codebase across the team, the project follows standardized Python development practices. These conventions support collaboration, ease of debugging, and future extensibility.

The development environment consists of Python 3.10+ on Ubuntu 22.04 LTS. We manage the dependencies using the uv [10] library, which uses pyproject.toml and uv.lock to ensure reproducible builds.

The use of inline comments to explain what the code does and why the decisions were made will keep the code easy to read and continue further development on. Proper naming conventions will be utilized in files, classes, methods, and variables to promote readability. Each subsystem will include a short README.md to explain the use of said subsystem. Using Python's built-in logging, we will be able to produce a verbose output that will let developers know exactly where the code failed.

7.3. Code Version Control

All project code is managed using Git and hosted on GitHub, providing a centralized platform for version tracking, collaboration, and peer review. The team follows a structured branching workflow, with the main branch reserved for stable, production-ready releases and a dev branch used for active development and integration testing. New features or fixes are developed in dedicated feature branches (e.g., feature/<name> or fix/<name>), ensuring that changes remain isolated until verified. Each commit will include a detailed message of what was changed. All pull requests must be reviewed by at least one team member and successfully pass automated testing before merging. Sensitive information, including API keys and credentials, is secured through .env files and excluded from the repository via .gitignore. Access to the repository is role-based, with maintainers responsible for approving merges into the main branch to preserve stability and code integrity.

7.4. Implementation Alternatives & Decision Rationale

One of the main areas of alternative consideration for our project was during the process of selecting an open-source package for QR code scanning. Initially, we were faced with selecting between three options: OpenCV, pyzbar, and zxing. The zxing scanning library has been placed into maintenance mode and no longer functions as intended, and experimentation with the library led us to acknowledge that it would not be suitable for the project.

Next, we began experimenting with OpenCV as it had been used before by one of our group members for a previous project. At first glance, the library appeared to contain everything we needed in regards to QR scanning. However, once the group began experimenting with the library, we found that it failed multiple of our tests. Most importantly, the library was unable to process a single frame containing multiple QR codes. When this scenario occurred, the library would continuously attempt to extract information from a single QR code in the frame, causing the program to enter an infinite loop.

This lack of support for one of our project's main requirements was largely why the group decided to use pyzbar for future QR code scanning functionality. Our testing has found that the library is capable of scanning dozens of barcodes from a single frame from multiple different angles in differing lighting scenarios.

8. System Testing

This section describes the testing approach used to validate the AIBI Computer Vision for Manufacturing system. We combined automated unit and integration tests with manual system, usability, and interoperability tests.

All test cases are defined in the Test Case Design Matrix (see Appendix T – Test Design Cards), and execution records for a selected subset of tests are captured in Appendix TE – Test Execution Cards.

8.1. Test Automation Framework

The automated tests are implemented in Python using a lightweight test automation framework built around:

- pytest for unit and integration test structure and assertions
- OpenCV and stored test images for QR detection/decoding tests
- JSON schema validation tools for SFR4/SFR12/EISR3 interoperability tests
- Simple Python benchmark harnesses for performance-related tests (PPSR2, PPSR5)

Tests are organized to match the requirements structure in the System and Non-Functional Requirements:

- Functional automated tests: SFR1, SFR2, SFR4, SFR5, SFR11, SFR12, SFR15, SFR16, SFR17, SFR18
- Non-functional automated tests:
  - Performance: PPSR2, PPSR5
  - Reliability / deployment: PDSR1, PDSR2, PDSR4
  - Development / quality constraints: ODSR1, OOSR4
  - Interoperability: EISR3, EISR5, EISR6

8.1.1. Steps for Installing Test Framework

The automated test framework can be installed on any development workstation with Python available. The following steps outline the complete setup procedure.

1. Clone the Project Repository

Obtain the latest source code by cloning the AIBI Computer Vision for Manufacturing repository from version control.

2. Install the uv Package Manager

The test framework uses the uv environment and dependency manager. Install it using the appropriate command for your operating system:

Windows (PowerShell):

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Linux/macOS:

curl -LsSf https://astral.sh/uv/install.sh | sh

3. Install Project and Test Dependencies

Install all required dependencies, including development and test packages, by running:

uv sync --extra dev

This command installs the following toolchain components:

- pytest – automated test runner
- opencv-python – computer vision processing
- numpy – numerical computation
- keyboard – keyboard event handling
- pygetwindow – window management utilities
- pyzbar (optional) – barcode/QR decoding
- pytest-cov – coverage reporting

4. Prepare Test Assets

Ensure all required test artifacts are present before running the test suites:

QR code test images located in tests/assets/

Workstation configuration files located in data/config/

Sample JSON test outputs stored in the outputs/ directory

These files are used by both unit and integration tests.

5. Verify Installation

Run a basic smoke test to confirm the testing environment is correctly configured:

Validate SFR test compliance

uv run python run_sfr_tests.py --sfr-only

Test individual components if desired:

uv run pytest tests/test_unit_requirements.py::TestTCSFR1AUTO01::test_multi_qr_detection_returns_n_codes -v

6. Run the Complete Test Suite

Execute all automated tests using:

uv run python run_sfr_tests.py

To include a coverage report:

uv run python run_sfr_tests.py --coverage

8.1.2. Steps for Running Test Cases

Once the automated test framework is installed and configured, test cases can be executed using the procedures outlined below.

1. Launch the Test Environment

Verify that all project dependencies and test assets are available, and ensure the active terminal session is using the configured uv environment.

2. Run Unit Test Cases

Unit tests validate individual functions and modules.

Run all unit tests:

uv run pytest tests/ -v

Run a specific unit test or test method:

uv run pytest tests/<test_file>.py::<TestClassName>::<test_method> -v

Example:

uv run pytest tests/test_unit_requirements.py::TestTCSFR1AUTO01::test_multi_qr_detection_returns_n_codes -v

3. Run Basic Functionality Tests

Basic functionality tests validate core components without full requirement coverage.

uv run python run_sfr_tests.py --basic-only

4. Run SFR (Software Functional Requirement) Compliance Tests

The automated framework verifies Software Functional Requirements, including SFR, PPSR, PDSR, ODSR, and EISR test categories.

uv run python run_sfr_tests.py --sfr-only

This ensures implemented features meet all defined system and software requirements.

5. Run the Complete Test Suite

Execute all unit tests, basic functionality tests, and SFR compliance tests in a single unified workflow:

uv run python run_sfr_tests.py

6. Generate a Coverage Report

Coverage analysis identifies the portions of the codebase exercised by the automated tests.

Run the full suite with coverage enabled:

uv run python run_sfr_tests.py --coverage

This produces:

- A coverage summary in the terminal
- A detailed HTML report was generated in the htmlcov/ directory

7. Review and Log Test Results

After execution, perform the following steps:

- Review the console output for failures or errors
- Inspect logs and coverage reports located at:
- htmlcov/index.html
- Document results in the Test Case Execution Report (Section 8.3)
- Verify all SFR compliance test cases pass before deployment

8.2. Test Case Design

All test cases are defined in Appendix T, which is derived directly from the functional and non-functional requirements.

Each test case includes:

- Test Case ID (e.g., TC-SFR1-AUTO-01)
- What to Test (short description)
- Test Data Input
- Expected Result
- Traceability to:
    - User Requirements (UFR, PUSR, etc.)
    - System Requirements (SFR, PPSR, PDSR, ODSR, EISR)
    - Use Cases (UC-001 … UC-005)

We group tests into suites by test type (unit, integration, system/manual, usability, performance).

8.2.1. Test Suites

Based on Appendix T, the following main test suites are defined:

1. Automated Functional Suite (Unit / Integration)
    - Covers core behavior of the QR detection, mapping, JSON aggregation, persistence, simulation, and keyboard-emulation logic.
    - Representative tests:
        - TC-SFR1-AUTO-01 – QR detection on multi-code image
        - TC-SFR2-AUTO-01 – field mapping
        - TC-SFR4-AUTO-01 – JSON builder
        - TC-SFR5-AUTO-01 – keyboard-emulation sequence
        - TC-SFR11-AUTO-01 – persistence
        - TC-SFR12-AUTO-01 – schema validation

        - TC-SFR15-AUTO-01, TC-SFR16-AUTO-01 – multi-code decoding/output
        - TC-SFR17-AUTO-01, TC-SFR18-AUTO-01 – simulation flows
2. Automated Non-Functional Suite (Performance, Reliability, Quality, Interoperability)
    - Focuses on performance, stability, local-only processing, and data exchange:
        - Performance: TC-PPSR2-AUTO-01, TC-PPSR5-AUTO-01
        - Reliability / on-prem constraints: TC-PDSR1-AUTO-01, TC-PDSR2-AUTO-01, TC-PDSR4-AUTO-01
        - Quality / development constraints: TC-ODSR1-AUTO-01, TC-OOSR4-AUTO-01
        - Interoperability and configuration: TC-EISR3-AUTO-01, TC-EISR5-AUTO-01, TC-EISR6-AUTO-01
3. System / Manual Functional Suite
    - Validates the integrated system from an operator's perspective:
        - TC-SFR1-MAN-01 – live camera multi-QR detection
        - TC-SFR2-MAN-01 – workstation mapping configuration
        - TC-SFR4-MAN-01 – real JSON output
        - TC-SFR5-MAN-01 – keyboard-emulation into a target app
        - TC-SFR11-MAN-01 – persistence and replay across outages
        - TC-SFR12-MAN-01 – documentation consistency
        - TC-SFR15-MAN-01, TC-SFR16-MAN-01 – multi-code decoding/output in real conditions
        - TC-SFR17-MAN-01, TC-SFR18-MAN-01 – simulation scenarios
4. Usability and Operator Workflow Suite
    - Focuses on user experience and interaction requirements:
        - TC-PUSR1-MAN-01 – training time ≤ 30 minutes
        - TC-PUSR2-MAN-01 – clarity of UI guidance
        - TC-PUSR3-MAN-01 – reduced effort vs legacy workflow
        - TC-PUSR4-MAN-01 – touch-free operation
        - TC-PUSR5-MAN-01 – automation of repetitive steps
        - TC-PUSR10-MAN-01 – meaning of icons/colors without text
5. System-Level Non-Functional Suite (Manual)
    - Focuses on runtime characteristics and deployment constraints verified in the deployed environment:
        - TC-PPSR2-MAN-01, TC-PPSR5-MAN-01 – perceived performance with real hardware
        - TC-PDSR1-MAN-01, TC-PDSR2-MAN-01, TC-PDSR4-MAN-01 – on-prem storage and stable operation
        - TC-ODSR2-MAN-01, TC-OOSR4-MAN-01 – OSS documentation and real legacy imports
        - TC-EISR4-MAN-01, TC-EISR5-MAN-01, TC-EISR6-MAN-01 – schema versioning, MES/ERP integration, configuration-driven workflows

The detailed content of each test case is provided in Appendix T.

8.2.2. Unit Test Cases

Unit tests focus on individual functions or modules and use controlled test data. The key unit-level test cases include:

- QR detection and decoding
  - TC-SFR1-AUTO-01 – Validate that detect_qr_codes() returns all codes and correct payloads on a multi-code image.
  - TC-SFR15-AUTO-01 – Verify decoding accuracy and error rate for complex multi-code images.
- Field mapping and JSON generation
  - TC-SFR2-AUTO-01 – Verify map_fields() correctly assigns decoded values to configured logical fields.
  - TC-SFR4-AUTO-01 – Verify build_scan_json() aggregates all decoded values and metadata into a single schema-compliant JSON document.
- Keyboard-emulation and simulation logic
  - TC-SFR5-AUTO-01 – Validate that build_keystroke_sequence() produces the expected keystroke string.
  - TC-SFR17-AUTO-01 – Validate that simulate_qr_injection() injects synthetic codes into frames.
  - TC-SFR18-AUTO-01 – Validate simulated events flow through the keyboard-emulation pipeline.
- Persistence and schema validation
  - TC-SFR11-AUTO-01 – Test saving and reloading scan events with audit metadata.
  - TC-SFR12-AUTO-01 – Test that all JSON outputs validate against the published schema.
  - TC-EISR3-AUTO-01 – Test that outputs also meet interoperability schema requirements.
- Non-functional unit tests
  - TC-PPSR2-AUTO-01, TC-PPSR5-AUTO-01 – Latency and accuracy benchmarks.
  - TC-PDSR1-AUTO-01, TC-PDSR2-AUTO-01, TC-PDSR4-AUTO-01 – Configuration and long-duration stability checks.
  - TC-ODSR1-AUTO-01, TC-OOSR4-AUTO-01 – Dependency and legacy import checks.
  - TC-EISR5-AUTO-01, TC-EISR6-AUTO-01 – Automated checks for integration payload structure and config-driven workflows.

The detailed unit test case definitions, including inputs and expected results, are provided in Appendix T.

8.2.3. Integration Test Cases

Integration tests combine multiple components or external systems:

- Persistence and replay path
  - TC-SFR11-AUTO-01 (unit/integration) and TC-SFR11-MAN-01 (system) together validate that events are persisted locally and replayed correctly after downstream outages.
- End-to-end scan to JSON output
  - TC-SFR4-AUTO-01 + TC-SFR4-MAN-01 validate that the entire path from detection → mapping → JSON aggregation produces correct, schema-compliant scan events.
- Simulation environment
  - TC-SFR17-AUTO-01, TC-SFR18-AUTO-01 and their manual counterparts (TC-SFR17-MAN-01, TC-SFR18-MAN-01) validate integration between simulated frames, detection, and keyboard-emulation.
- Interoperability with MES/ERP
  - TC-EISR5-AUTO-01 – Automated tests against a mock MES/ERP endpoint.
  - TC-EISR5-MAN-01 – System-level test with an actual MES/ERP test instance.
- Configuration-driven workflow
  - TC-EISR6-AUTO-01 – Automated check that different config files alter behavior without code changes.
  - TC-EISR6-MAN-01 – System-level test where a field engineer changes the workflow via configuration only.

These integration tests ensure that no major regressions occur when components are combined and that the system interacts correctly with external factory systems.

8.3. Test Case Execution Report

This section summarizes how the defined test cases are (or will be) executed and how results are recorded.

- All test case designs are documented in Appendix T – Test Design Cards (generated from the Appendix T).
- Execution details (tester, date, actual result, pass/fail, defects) are recorded in Appendix TE – Test Execution Cards for each executed test.

In practice, only a prioritized subset of tests (covering critical paths and high-risk requirements) must be executed for each iteration, while the full matrix provides a roadmap for regression testing.

8.3.1. Unit Testing Report

Unit and automated integration tests are executed using the pytest-based framework described in Section 8.1.

For each unit test case (e.g., TC-SFR1-AUTO-01, TC-SFR4-AUTO-01, TC-SFR11-AUTO-01, TC-PPSR2-AUTO-01):

- The Test Design Card (Appendix T) specifies:
  - What is being tested

    - Inputs and expected results
    - Traceability to requirements and use cases
- The Test Execution Card (Appendix TE) records:
    - Tools used (Python, pytest, OpenCV, etc.)
    - Execution steps followed
    - Execution records (tester, date, actual result, pass/fail, defect IDs)
    - A brief execution summary

As tests are run, their status is updated in Appendix TE. This provides clear traceability from requirements → test design → executed test → result.

8.3.2. Integration Testing Report

Integration and system-level tests are executed using both the automated framework and manual procedures:

- Automated integration tests (e.g., TC-SFR11-AUTO-01, TC-EISR5-AUTO-01, TC-EISR6-AUTO-01) are run via pytest and recorded in the same way as unit tests.
- System and interoperability tests (e.g., TC-SFR11-MAN-01, TC-EISR5-MAN-01, TC-EISR6-MAN-01) are executed on a deployed system in a lab or pilot factory environment. Execution records are captured using the Test Execution Card template.

For each executed integration test, Appendix TE contains:

- The exact configuration used (e.g., mock vs real MES/ERP instance, test workstation)
- The observed behavior across components
- Whether the overall scenario passed or failed against the expected result

Together, the unit and integration execution reports show that the system not only behaves correctly at the component level but also functions as intended when deployed in realistic manufacturing scenarios.

9. Challenges & Open Issues

9.1. Challenges Faced in Requirements Engineering

Defining clear requirements for this project has been an ongoing challenge due to its evolving scope and layered communication structure. Unlike a traditional software project with fixed specifications, this initiative is still in the developmental phase within the manufacturer's broader modernization effort. As a result, many system expectations continue to shift as the manufacturer refines its own internal processes and priorities.

Another difficulty stems from the project's two-in-one nature. AIBI Solutions is developing the AI component while KMD Technology Solutions oversees the larger software integration. This means that decisions are often split between technical design for AIBI's on-premise AI system and operational requirements defined by KMD's project management tools. Balancing both sides has required ongoing clarification to ensure that features remain aligned with the manufacturer's real-world workflow.

Communication flow also adds complexity. Because the development is being conducted for a manufacturer through a software company, information moves through several layers before reaching the engineering team. Questions and feedback can take time to travel back and forth, which slows iteration and can cause temporary gaps in understanding.

Additionally, from the manufacturer's perspective, the AI system is currently viewed as a lower priority compared to its core production and infrastructure initiatives. This limits the amount of dedicated feedback and testing time available, making it harder to gather detailed operational requirements early on.

Many questions at this stage remain speculative, there are still unknowns about how the system will integrate with existing machinery, what data streams will be available, and what the long-term performance expectations will look like. Because of that uncertainty, much of the early requirements work has focused on flexibility and modularity rather than strict definitions.

Finally, testing in a production-like environment presents logistical challenges. Access to the actual manufacturing floor is limited, and replicating realistic lighting, noise, and workflow conditions off-site is difficult. This means that validating the system's accuracy and reliability under true operating conditions will likely occur later in the development cycle, increasing the importance of careful simulation and staged deployment.

Overall, these challenges have required an adaptive, iterative approach to requirements engineering, focusing on communication, flexibility, and traceability while allowing the technical design to evolve alongside the manufacturer's own readiness.

9.2. Challenges Faced in System Development

One of the major challenges we faced during system development was remote collaboration. Not all members of our team were located in the same area, so we were required to work entirely

remotely. This created difficulties with both code reviews and peer programming. Because we weren't physically together, we couldn't engage in traditional peer-programming sessions where two developers work at the same workstation and communicate naturally in real time.

To overcome this, we relied heavily on Visual Studio Code's Live Share feature, which allowed multiple team members to interact with the same codebase simultaneously. Combined with Discord voice calls and virtual meetings, we were able to recreate much of the experience of in-person peer programming. These tools allowed us to talk through problems, guide each other through code sections, and collaboratively resolve issues as if we were in the same room.

Code reviews also posed challenges, as it was more difficult to clearly communicate the rationale behind changes or walk through updates together. We addressed this by using GitHub pull requests to structure our reviews and by holding virtual meetings whenever more detailed explanations were needed. This setup allowed us to maintain clear communication and effective collaboration despite the limitations of remote development.

10. System Manuals

10.1. Instructions for System Development

System development for this project follows a structured process designed to maintain consistency, reliability, and reproducibility across all team members and deployment environments. All development is conducted using Python 3.10+ on Ubuntu Server 22.04 LTS, ensuring compatibility with the on-premise infrastructure already in place at the manufacturer. Dependency management is handled through uv [10], which provides deterministic builds that enable developers to quickly synchronize environments across different workstations. Development and debugging are performed using VS Code, chosen for its integration with Git, Python, and FastAPI-based tooling.

All source code is tracked using Git and hosted on GitHub, following the branching workflow established in Section 7.3. The main branch is reserved for stable, production-ready code, while all active work occurs in the dev branch. All merges require successful test execution and peer review. Commit messages follow a consistent format describing the scope of each change. Sensitive credentials, including database connection strings and API keys, are never committed to version control and are securely managed through .env files that are ignored by Git.

The system is organized into modular subsystems to ensure scalability and maintainability. The QR code recognition subsystem uses OpenCV [5] for image preprocessing and pyzbar [1] / ZXing [2] for decoding QR codes, while the worker-step verification subsystem utilizes OpenCV and open_clip [3] to extract image embeddings and verify human task completion through step-specific inference. Both subsystems rely on NumPy [8] and Pandas [9] for numerical processing and structured data logging. Logging throughout the system is handled using Python's built-in logging library, providing detailed runtime information to assist in debugging and error tracing. Each subsystem includes a README.md file describing its purpose, structure, and key functions, ensuring that future developers can quickly understand and extend the implementation.

All scan results, worker verification data, and configuration settings are stored in an on-premises SQL Server database. Connections are established through environment-defined credentials and utilize parameterized queries to maintain data integrity and security. Testing is an integral part of the development workflow: unit tests are written with pytest while integration testing is conducted using sample camera feeds and controlled production simulations. Benchmarking is used to evaluate performance metrics such as frame rate, recognition accuracy, and latency under realistic load conditions.

Deployment targets include both edge workstations and a central application server, each running isolated uv environments for reproducibility. Logs and configuration files are maintained in standardized directories on the host system. All changes to the system are documented through inline code comments, updated README.md files, and version entries in CHANGELOG.md.

By adhering to these development instructions, the team ensures that the AIBI computer-vision system remains modular, secure, and reproducible. The consistent use of Python, uv, and GitHub

provides a reliable foundation for collaboration, testing, and long-term maintainability, while the project's standardized structure supports ongoing refinement and expansion as new features and subsystems are introduced.

10.1.1 How to Set Up a Development Environment

1. First, clone the project's GitHub repository using your preferred IDE, then open the project inside the IDE.
2. Install "uv", which is the tool used to manage the project's Python environment and dependencies.
    - For Windows, open PowerShell and run the following command: powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
    - For Linux/macOS, run 'curl -LsSf https://astral.sh/uv/install.sh | sh'
3. After installing uv, install all project dependencies.
    - Navigate to the project directory and run: uv sync
    - This will install all required packages and set up the development environment.

12. References

[1] L. Hudson, “Naturalhistorymuseum/pyzbar: Read One-dimensional barcodes and QR codes from Python 2 and 3.,” GitHub, https://github.com/NaturalHistoryMuseum/pyzbar (accessed Oct. 13, 2025).

[2] “Zxing/Zxing: Zxing (‘Zebra crossing’) Barcode Scanning Library for Java, Android,” GitHub, https://github.com/zxing/zxing (accessed Oct. 13, 2025).

[3] G. Ilharco et al., OpenCLIP. Zenodo, 2021. doi: 10.5281/zenodo.5143773 (accessed Oct. 13, 2025).

[4] F. Faghri et al., “MobileCLIP2: Improving Multi-Modal Reinforced Training,” Transactions on Machine Learning Research, 2025, [Online]. Available: https://openreview.net/forum?id=WeF9zolng8 (accessed Oct. 13, 2025).

[5] G. Bradski, “The OpenCV Library,” Dr. Dobb’s Journal of Software Tools, 2000 (accessed Oct. 13, 2025).

[6] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library.” 2019. [Online]. Available: https://arxiv.org/abs/1912.01703 (accessed Oct. 13, 2025).

[7] S. Ramírez, FastAPI. [Online]. Available: https://github.com/FastAPI/FastAPI (accessed Oct. 13, 2025).

[8] C. R. Harris et al., “Array programming with NumPy,” Nature, vol. 585, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2 (accessed Oct. 13, 2025).

[9] The pandas development team, pandas-dev/pandas: pandas. doi: 10.5281/zenodo.3509134 (accessed Oct. 13, 2025).

[10] C. Marsh et al., “uv,” UV, https://docs.astral.sh/uv/#the-pip-interface (accessed Oct. 13, 2025).

[11] conda contributors, conda: A system-level, binary package and environment manager running on all major operating systems and platforms. [Online]. Available: https://github.com/conda/conda (accessed Oct. 13, 2025).

13. Team Members' Log Sheets

13.1 Nathan DiGilio

Report 1 Date	Task	Duration
9/25/2025	Mentor/advisor meetings, group discussion	3 hours
10/02/2025	Mentor/advisor meetings, group discussion	3 hours
10/8/2025	Problem Statement section	4 hours
10/09/2025	Mentor/advisor meetings, group discussion	3 hours
10/9/2025	Exploratory Studies	5 hours
10/10/2025	System Implementation section	4 hours
10/11/2025	Challenges & Open Issues	4 hours
10/13/2025	Report Formatting	3 hours
Report 2		Total: 29 hours
10/23/2025	Mentor/advisor meetings, group discussion	3 hours
10/30/2025	UML Class Diagram Tour of production facility, group discussion	2 hours
11/6/2025	Mentor/advisor meetings, group discussion	3 hours
11/12/2025	Fixing old requirements	5 hours
11/13/2025	Mentor/advisor meetings, group discussion	3 hours
11/14/2025	Test Cases	4 hour
11/28/2025	Test Cases Implementation	4 hours
12/1/2025	Tests Cases Cont.	6 hours
		Total: 30 hours
Report 3		Total: 29 hours
1/23/2026	Mentor/advisor meetings, group discussion	3 hours
1/30/2026	Part 2 implementation	2 hours
2/6/2026	Mentor/advisor meetings, group discussion	3 hours
2/12/2026	Part 2 implementation	5 hours

2/13/2026	Mentor/advisor meetings, group discussion	3 hours
2/14/2026	Part 2 implementation	4 hour
2/15/2026	Test Cases Overhaul	4 hours
2/16/2026	Tests Cases Cont.	6 hours
		Total: 30 hours

13.2 Jacob Dzikowski

Report 1 Date	Task	Duration
9/25/2025	Mentor/advisor meetings, group discussion	3 hours
10/02/2025	Mentor/advisor meetings, group discussion. Initial document creation (outline, title page, etc.)	5 hours
10/07/2025	Initial abstract draft, added some requirements	2 hours
10/09/2025	Mentor/advisor meetings, group discussion	3 hours
10/11/2025	Use cases, table generation, general improvements	5 hours
10/12/2025	Requirements table generation, general improvements	4 hours
10/13/2025	General improvements, formatting, etc.	10 hours
Report 2		Total: 32 hours
10/23/2025	Mentor/advisor meetings, group discussion	3 hours
10/30/2025	UML Class Diagram Tour of production facility, group discussion	5 hours
11/6/2025	Mentor/advisor meetings, group discussion	3 hours
11/12/2025	Initial QR scanner implementation Pair programming with Collin	7 hours
11/13/2025	Mentor/advisor meetings, group discussion	3 hours
11/18/2025	Added keyboard/excel functionality Revised scanner display information	4 hours
12/1/2025	Revised requirements, worked on various sections with the group	6 hours
Report 3		Total: 31 hours

1/21/2026	Mentor meeting, group work/discussion	1.5 hours
1/26/2026	Advisor meeting, group work/discussion	1.5 hours
1/28/2026	Mentor meeting, group discussion	1.5 hours
2/4/2026	Mentor meeting, group work/discussion	3 hours
2/8/2026	QR code directional scanning testing	2 hours
2/9/2026	Advisor meeting, group work/discussion	1.5 hours
2/11/2026	Group working session, tested training process and updates to QR scanning	4 hours
2/12/2026	Group meeting after Senior Design class, assigned tasks and discussed report	2 hours
2/16/2026	Advisor meeting, group discussion	1.5 hours
2/18/2026	Mentor meeting and step-training demo	1 hour
2/20/2026	Work on Report 3, updates to everything involving use cases and other various grammatical errors	7 hours
		Total: 26.5 hours

13.3 Daniel Pora

Report 1 Date	Task	Duration
9/25/2025	Mentor/advisor meetings, group discussion	3 hours
9/30/2025	Created user non-functional and functional requirements from 9/19 meeting	2.5 hours
10/02/2025	Wrote questions for the Sponsor, from the advisor meeting and group discussion	2.5 hours
10/02/2025	Mentor/advisor meetings, group discussion	3 hours
10/09/2025	Mentor/advisor meetings, group discussion	3 hours
10/09/2025	Created user non-functional and functional requirements from 10/02 meeting	2.5 hours
10/09/2025	Combined user non-functional and functional requirements into a single document for system conversion	1.5 hours
10/12/2025	Worked on report	4 hours
10/13/2025	Worked on report	8 hours
Report 2		Total: 30 hours
10/23/2025	Mentor/advisor meetings, group discussion + documentation	4 hours
10/30/2025	Tour of production facility, group discussion + documentation	6 hours
11/6/2025	Mentor/advisor meetings, group discussion + documentation	4 hours
11/7/2025	Researched different behavioral diagrams, Created Activity Diagram	2.5 hours
11/13/2025	Mentor/advisor meetings, group discussion + documentation	4 hours
11/10/2025	Worked on report	3 hours
11/28/2025	Helped Collin Revise the Class Diagram	1.5 hours
11/28/2025	Rewrote the Package Diagram	1.5 hours
12/1/2025	Reviewed and Revised Report	6 hours

		32.5 hours
Report 3		Total: 31 hours
1/21/2026	Mentor meeting, group work/discussion	1.5 hours
1/26/2026	Advisor meeting, group work/discussion	1.5 hours
1/28/2026	Mentor meeting, group discussion	1.5 hours
2/4/2026	Mentor meeting, group work/discussion	3 hours
2/8/2026	QR code directional scanning testing	2 hours
2/9/2026	Advisor meeting, group work/discussion	1.5 hours
2/11/2026	Group working session, tested training process and updates to QR scanning	4 hours
2/12/2026	Group meeting after Senior Design class, assigned tasks and discussed report	2 hours
2/14/2026	Major code refactoring to both systems	3 hours
2/16/2026	Advisor meeting, group discussion	1.5 hours
2/18/2026	Mentor meeting and step-training demo	1 hour
2/20/2026	Work on Report 3, Created new structural, behavioral, and architectural diagrams for our second system.	4 hours
2/21/2026	Work on Report 3, Described the new diagrams for our second system, as well as design alternatives and rational.	3 hours
		Total: 29.5 hours

13.4 Collin Miller

Report 1 Date	Task	Duration
9/25/2025	Mentor/advisor meetings, group discussion	3 hours
10/02/2025	Mentor/advisor meetings, group discussion	3 hours
10/09/2025	Mentor/advisor meetings, group discussion	3 hours
10/10/2025	Converted User to System Requirements; Made some User Requirement Cards	4 hours
10/11/2025	Added System Requirements to the document and reviewed them	4 hour
10/11/2025-10/13/2025	Made all System Requirement cards; Made the trace table; Worked in general on report; Checked over all requirement cards to make sure the traces were correct; Worked on use case diagram	16 hours
Report 2		Total: 33 hours
10/23/2025	Mentor/advisor meetings, group discussion	3 hours
10/30/2025	UML Class Diagram Tour of production facility, group discussion	5 hours
11/6/2025	Mentor/advisor meetings, group discussion	3 hours
11/12/2025	Pair programming with Jacob	5 hours
11/13/2025	Mentor/advisor meetings, group discussion	3 hours
11/14/2025	UML Class Diagram	1 hour
11/28/2025	Revised Class Diagram with Dan, refactored code to be more modular	3 hours
12/1/2025	Revised requirements, added class diagram	10 hours
		Total: 33 hours
Report 3		
1/30/2026	Requirement Gathering and converting to system requirements	4 hours

2/4/2026	Altered program to fit new requirements	3 hours
2/9/2026 – 2/19/2026	Created requirement cards; Updated trace table	4 hours
1/21/2026 - 2/20/2026	Meetings/general discussions	10 hours
2/18/2026	Mentor step demo	1 hour
2/19/2026	General work: helping with report/code	3 hours
		Total: 25 hours

Appendix U

1. Use Case Tables

Project Name:	**AIBI Computer Vision for Manufacturing**
Use Case ID:	UC-001
Use Case Name:	Scan Part QR codes
User Goal:	Automatically detect and decode multiple QR codes at a workstation
Scope:	QR code Recognition Module
Level:	Primary Task
Primary Actor:	Factory Operator
Precondition:	Camera system is operational, and workstation is configured
Minimal Guarantee:	System logs scan attempt
Success Guarantee:	All QR codes decoded, and data formatted into JSON output
Trigger:	Paper QR codes are placed within the camera's field of view
Success Scenario:	**Step Action**
	1. Factory Operator places part with visible QR codes within camera field of view
	2. System detects object presence in frame
	3. System captures video frame
	4. System detects all visible QR codes in frame
	5. System decodes each QR code and extracts data
	6. System maps decoded values to configured data fields based on spatial position
	7. System aggregates all decoded values into structured JSON payload
	8. System outputs JSON to configured destination (API, database, or keyboard emulation)
	9. System displays success feedback on workstation terminal
Extensions:	**Branching Scenarios**
4A	**Condition:** No QR codes detected
	Step Actions
	1. System logs detection failure
	2. System displays warning messages to operator
	3. System waits for operator to reposition paper with QR codes

	4. Continue from main step 2
5A	**Condition:** QR code decode failure
	Step Actions
	1. System logs which QR code(s) failed to decode
	2. System displays error message indicating failure
	3. System prompts operator to clean or reposition paper QR codes, or enter information manually
	4. Continue from main step 2
8A	**Condition:** Downstream system unavailable
	Step Actions
	1. System stores JSON payload locally with timestamp
	2. System displays warning to operator
	3. System queues data for later synchronization
	4. Continue from main step 9

Project Name:	**AIBI Computer Vision for Manufacturing**
Use Case ID:	UC-002
Use Case Name:	Monitor Manufacturing Steps
User Goal:	Verify that manufacturing steps are performed in correct sequence
Scope:	Step Verification Module
Level:	Primary Task
Primary Actor:	Factory Operator
Precondition:	Process steps are configured and camera system is operational
Minimal Guarantee:	System logs monitoring attempt
Success Guarantee:	Step verified as complete and logged correctly
Trigger:	Operator begins manufacturing step within view of camera
Success Scenario:	**Step Action**
	1. Factory Operator begins manufacturing step
	2. System continuously analyzes video feed

	3. System detects step start based on configured visual indicators
	4. System updates UI overlay to show step in progress
	5. System monitors for step completion indicators
	6. System detects step completion
	7. System validates step occurred in correct sequence
	8. System logs step completion with timestamp and metadata
	9. System updates UI to show step complete
	10. System enables next step in sequence
Extensions:	**Branching Scenarios**
7A	**Condition:** Step performed out of sequence
	Step Actions
	1. System detects sequence violation 2. System displays visual alert on workstation terminal 3. System plays optional audio alert 4. System logs violation with details 5. System prevents progression to next step 6. Production Supervisor reviews violation log 7. Use case ends
7B	**Condition:** Step skipped
	Step Actions
	1. System detects operator attempting to skip to later step 2. System displays warning message 3. System prevents progression until skipped step completed 4. Continue from main step 1

Project Name:	**AIBI Computer Vision for Manufacturing**
Use Case ID:	UC-003
Use Case Name:	Train System on Manufacturing Process
User Goal:	Record and label reference video of correct manufacturing process
Scope:	Training Data Capture Module
Level:	Primary Task
Primary Actor:	Production Supervisor
Precondition:	User has access to training interface and camera system is operational

Minimal Guarantee:	The user is informed that there was an error during training, and no data is saved.
Success Guarantee:	Complete training dataset captured with all step labels and metadata
Trigger:	Supervisor selects "New Training Session" from system interface
Success Scenario:	**Step Action**
	1. Production Supervisor launches training capture tool
	2. System displays live video feed and training controls
	3. System prompts for step label/description
	4. Supervisor enters step label
	5. System begins recording video feed
	6. Operator performs first manufacturing step
	7. Supervisor clicks "Mark Step Start"
	8. System records timestamp and frame number
	9. Operator completes first step
	10. Supervisor clicks "Mark Step End"
	11. Steps 6-10 repeat for each manufacturing step
	12. Supervisor clicks "Complete Training"
	13. System saves training session with all metadata
	14. System displays confirmation message
Extensions:	**Branching Scenarios**
7A	**Condition:** Supervisor marks incorrect start point
	Step Actions
	1. Supervisor clicks "Delete Step" 2. System removes the incorrect step 3. Continue from main step 7
12A	**Condition:** Storage unavailable
	Step Actions
	1. System displays error message, notifying supervisor 2. Supervisor allocates device storage for saved training data 3. Supervisor clicks “Complete Training” 4. Continue from main step 13

Project Name:	**AIBI Computer Vision for Manufacturing**
Use Case ID:	UC-005
Use Case Name:	Simulate Workstation Environment
User Goal:	Test system functionality before production deployment
Scope:	Simulation and Testing Module
Level:	Primary Task
Primary Actor:	KMD Technology Solutions Software Team
Precondition:	Simulation environment is installed and configured
Minimal Guarantee:	Simulation data logged
Success Guarantee:	System tested and validated in simulated environment
Trigger:	Developer selects "Start Simulation" from testing interface
Success Scenario:	**Step Action**
	1. Developer launches simulation environment
	2. System displays simulation controls
	3. Developer loads recorded camera feed
	4. System displays video playback controls
	5. Developer injects synthetic QR code data
	6. System processes simulated input
	7. System generates JSON output
	8. Developer verifies JSON format and content
	9. Developer tests API/keyboard-emulation outputs
	10. System displays output verification results
	11. Developer reviews results and logs
Extensions:	**Branching Scenarios**
6A	**Condition:** Processing error in simulation
	Step Actions

	1. System logs error details 2. System displays error message to developer 3. Developer reviews error logs 4. Developer adjusts simulation parameters 5. Continue from main step 5
8A	**Condition:** JSON validation fails
	Step Actions
	1. System highlights validation errors 2. Developer reviews schema mismatches 3. Developer corrects configuration 4. Continue from main step 6
10A	**Condition:** Output verification fails
	Step Actions
	1. System displays specific failure points 2. Developer reviews system configuration 3. Developer makes necessary adjustments 4. Continue from main step 9

Appendix R

1. User Functional Requirements:

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	UFR1			Type	Functional	Non-Functional
Creation	October 9, 2025, 3:51pm			User	✓	
Modification	12/1/2025, 2:45pm			System		
Description	Fixed camera at each workstation must detect, decode, and process multiple QR codes from objects presented within the camera's field of view					
Priority	**Highest** ✓	High	Medium	Low	Lowest
This Req. is Refined Into	SFR1, SFR4, PPSR2, PPSR3, SFR15, SFR16				
Justify why UFR1 can be completely **covered by SFR1, SFR4, PPSR1, PPSR2, PPSR3**	The camera will scan a QR code and then decode those values into a structured JSON format prioritizing it happening in real time with an accuracy of at most 1 error per 100 scans				

Traceability	Use cases	UC-001, UC-005
	Test cases	Not yet covered

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	UFR2		Type	Functional	Non-Functional
Creation	October 9, 2025, 3:54pm		User	✓	
Modification	12/1/2025, 2:45pm		System		
Description	Provide configurable field mapping system to define required and optional QR code data fields per workstation				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Refined Into	SFR2				
Justify why UFR2 can be completely covered by SFR2	This will allow configurable field definitions for QR code data validation at each workstation				
Traceability	Use cases	UC-001			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	UFR3		Type	Functional	Non-Functional
Creation	October 12, 2025, 8:38pm		User	✓	
Modification			System		
Description	Identify and track QR codes on large parts rather than sheets of paper				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Refined Into	PPSR5, PPSR6				
Justify why UFR3 can be completely covered by PPSR5, PPSR6	The camera will recognize any QR code on any part with great accuracy levels and will adjust with varying camera qualities and lighting conditions				
Traceability	Use cases	UC-001			

	Test cases	Not yet covered

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	UFR4		Type	Functional	Non-Functional
Creation	October 12, 2025, 9:10pm		User	✓	
Modification			System		
Description	Aggregate all decoded values into structured JSON payload for downstream system integration				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Refined Into	SFR4, SFR12, EISR3, EISR4				
Justify why UFR4 can be completely covered by SFR4, SFR12, EISR3, EISR4	The camera will take the scanned data and put it into a structed JSON and be published. It will be version-controlled and included in system documentation				
Traceability	Use cases	UC-001			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	UFR5		Type	Functional	Non-Functional
Creation	10/12/2025, 9:11pm		User	✓	
Modification			System		
Description	Monitor and validate that manufacturing steps are performed in correct sequence at each workstation				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Refined Into	SFR6, SFR10, PPSR4,				
Justify why UFR5 can be completely covered by SFR6, SFR10, PPSR4	SFR6 ensures the system detects and validates each manufacturing step in sequence, while SFR10 logs any deviations. PPSR4 guarantees that the system flags errors with less than 1% margin, fulfilling the need for accurate step monitoring and traceability.				
Traceability	Use cases	UC-002			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	UFR6		Type	Functional	Non-Functional
Creation	10/12/2025, 9:12pm		User	✓	
Modification	2/17/2026		System		
Description	Provide immediate visual feedback and alerts for process status and violations on terminals at workstations				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Refined Into	SFR7, SFR26, SFR27, PPSR1				
Justify why UFR6 can be completely covered by SFR7, PPSR1	SFR7, SFR26, and SFR27 provides live UI overlays and alerts, giving operators immediate feedback. PPSR1 ensures this feedback is delivered within 200ms, meeting the need for real-time responsiveness and visual clarity at workstations				
Traceability	Use cases	UC-002			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	UFR7		Type	Functional	Non-Functional
Creation	10/12/2025, 9:13pm		User	✓	
Modification			System		
Description	Log all errors caught by the system for the users to review				
Priority	Highest	High	**Medium ✓**	Low	Lowest
This Req. is Refined Into	SFR10, SFR11, PDSR1, PDSR2, PDSR3				
Justify why 7 can be completely covered by SFR10, SFR11, PDSR1, PDSR2, PDSR3	SFR10 logs all process violations, while SFR11 ensures local storage and replay capabilities. PDSR1 and PDSR2 enforce on-premises data handling, and PDSR3 guarantees system uptime, collectively supporting reliable error tracking and review.				
Traceability	Use cases	UC-002			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**			
Requirement #	UFR8	Type	Functional	Non-Functional
Creation	10/12/2025, 9:14pm	User	✓	

Modification				System		
Description	Provide intuitive UI for non-technical users to train the system on correct manufacturing processes				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Refined Into		SFR8, SFR9, PUSR1, PUSR2, PUSR3, PUSR4, PUSR5, PUSR6, PUSR7, PUSR8, PUSR9, PUSR10			
Justify why UFR8 can be completely covered by SFR8, SFR9, PUSR1-PUSR10		SFR8 enables non-technical users to record and label training videos. SFR9 ensures the system adapts to natural worker variations. The PUSR requirements collectively ensure the UI is intuitive, accessible, and easy to configure, fulfilling the need for a user-friendly training interface.			
Traceability	Use cases	UC-003			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	UFR9			Type	Functional	Non-Functional
Creation	10/12/2025, 9:14pm			User	✓	
Modification	12/1/2025, 2:40pm			System		
Description	Seamless integration with existing KMD Technology Solutions workflow management system				
Priority	Highest	High	**Medium ✓**	Low	Lowest
This Req. is Refined Into		SFR5, SFR12			
Justify why UFR9 can be completely covered by SFR5, SFR12		SFR5 supports integration via keyboard emulation, while SFR12 ensures consistent JSON schema documentation. Together, they enable seamless interoperability with KMD's workflow system.			
Traceability	Use cases	N/A			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**			
Requirement #	UFR10	Type	Functional	Non-Functional
Creation	10/12/2025, 9:16pm	User	✓	
Modification		System		

Description	Implement intermediate data storage to decouple computer vision processing from KMD data consumption				
Priority	Highest	High	**Medium ✓**	Low	Lowest
This Req. is Refined Into		SFR11, PDSR1, PDSR2			
Justify why 10 can be completely covered by SFR11, PDSR1, PDSR2		SFR11 provides intermediate local storage and replay capabilities. PDSR1 and PDSR2 ensure all data remains on-premises, fulfilling the need to decouple vision processing from KMD's data consumption securely.			
Traceability	Use cases	N/A			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	UFR11		Type	Functional	Non-Functional
Creation	10/12/2025, 9:16pm		User	✓	
Modification	12/1/25, 2:40pm		System		
Description	Provide simulated workstation environment for development and validation before production deployment				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Refined Into		SFR13, PDSR4			
Justify why UFR11 can be completely covered by SFR13, PDSR4, SFR17, SFR18		SFR13, SFR17, SFR18 enables simulation of workstation environments for testing. PDSR4 ensures the system is validated under real factory conditions, supporting safe and reliable pre-deployment testing.			
Traceability	Use cases	UC-005			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**			
Requirement #	UFR12	Type	Functional	Non-Functional
Creation	10/12/2025, 9:17pm	User	✓	
Modification		System		
Description	Provide tools for capturing, organizing, and managing training datasets for model development			

Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Refined Into		SFR14			
Justify why UFR12 can be completely covered by SFR14		SFR14 provides tools for dataset management, including tagging and versioning, fulfilling the need to support model development and training workflows.			
Traceability	Use cases	UC-003			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	UFR13		Type	Functional	Non-Functional
Creation	2/8/2026 11:35am		User	✓	
Modification			System		
Description	Show all errors in the process on screen				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Refined Into		SFR19			
Justify why UFR13 can be completely covered by SFR19		This can be covered by SFR19 because SFR19 states that it will provide a message to the user for all errors in the step processing			
Traceability	Use cases	UC-003			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	UFR14		Type	Functional	Non-Functional
Creation	2/8/2026		User	✓	
Modification			System		
Description	Show all steps for the process on screen				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Refined Into		SFR20			
Justify why UFR14 can be completely covered by SFR20		SFR20 provides the user with a list of all steps in the process on the UI			
Traceability	Use cases	Not available			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	UFR15		Type	Functional	Non-Functional
Creation	2/8/2026		User	✓	
Modification			System		
Description	Have a visual marker for completed or missed steps.				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Refined Into	SFR 21, SFR 22				
Justify why UFR 15 can be completely covered by SFR21, 22	This requirement can be covered by SFR21 and 22 because 21 is for providing a green check on the UI for a correct step and 22 if for providing a red "X" on the UI for an incorrect step.				
Traceability	Use cases	Not yet covered			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	UFR16			Type	Functional	Non-Functional
Creation	2/8/2026			User	✓	
Modification				System		
Description	Click on check box to show the step				
Priority	**Highest**	High	Medium ✓	Low	Lowest
This Req. is Refined Into	SFR 23				
Justify why UFR 16 can be completely covered by SFR23	This can be covered by SFR 23 because it states that you can click on the step in the list to view what the step should look like.				
Traceability	Use cases	Not yet covered			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	UFR17			Type	Functional	Non-Functional
Creation	2/8/2026			User	✓	
Modification				System		
Description	Admin function for the in-site capture tool				
Priority	**Highest**	High	Medium ✓	Low	Lowest
This Req. is Refined Into	SFR24				
Justify why UFR 17 can be completely covered by SFR 24	This can be covered by SFR 24 because it provides administrative functionality for the in-site capture tool.				
Traceability	Use cases	Not yet covered			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**		
Requirement #	UFR18	Type	Functional	Non-Functional
Creation	2/8/2026	User	✓	
Modification		System		
Description	Pass in what workstation when running the tool		

Priority	**Highest**	High ✓	Medium	Low	Lowest
This Req. is Refined Into		SFR 32			
Justify why UFR 18 can be completely covered by SFR 32		It can be covered by SFR 32 because it provides the ability for the user to enter the workstation that the program start at			
Traceability	Use cases	Not yet available			
	Test cases	Not yet available			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	UFR19		Type	Functional	Non-Functional
Creation	2/8/2026		User	✓	
Modification			System		
Description	Command line control				
Priority	**Highest**	High	Medium ✓	Low	Lowest
This Req. is Refined Into		SFR25			
Justify why UFR 19 can be completely covered by SFR 25		It can be completely covered because it integrates the command line for the users to control the program			
Traceability	Use cases	Not yet available			
	Test cases	Not yet available			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	UFR20		Type	Functional	Non-Functional
Creation	2/8/2026		User	✓	
Modification			System		
Description	Provide a reset button for transition to the next part				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Refined Into		SFR 28			
Justify why UFR 20 can be completely covered by SFR 28		It can be completely covered because it provides a reset key to allow the program to get ready for the next part.			
Traceability	Use cases	Not yet covered			

	Test cases	Not yet covered

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	UFR21			Type	Functional	Non-Functional
Creation	2/8/2026			User	✓	
Modification				System		
Description	Have configuration for the number of QR codes expected				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Refined Into	SFR 29				
Justify why UFR 21 can be completely covered by SFR 29	It can be completely covered because it provides a configuration for the user to specify the amount of QR codes that are expected.				
Traceability	Use cases	Not yet covered			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	UFR22			Type	Functional	Non-Functional
Creation	2/8/2026			User	✓	
Modification				System		
Description	Have configuration for the direction of which the user wants them to be scanned				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Refined Into	SFR 30				
Justify why UFR 22 can be completely covered by SFR 30	It can be covered because it provides a configuration to specify the direction the QR codes should be scanned				
Traceability	Use cases	Not yet covered			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**			
Requirement #	UFR23	Type	Functional	Non-Functional
Creation	2/8/2026	User	✓	

Modification				System		
Description	Have configuration for if they want to add a character at the end of the QR data				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Refined Into		SFR 31			
Justify why UFR 23 can be completely covered by SFR 31		It can be completely covered because it allows the user to specify if they want a character added to the end of the QR data.			
Traceability	Use cases	Not yet covered			
	Test cases	Not yet covered			

2. User Non-functional Requirements:

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PUUR1			Type	Functional	Non-Functional
Creation	10/12/2025, 9:20pm			User		✓
Modification				System		
Description	System must be extremely easy to use for manufacturing workers with minimal training, needs to be easier than what is currently being done				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Refined Into		PUSR1, PUSR2, PUSR6, PUSR7, PUSR10			
Justify why PUUR1 can be completely covered by PUSR1, PUSR2, PUSR6, PUSR7, PUSR10		These requirements ensure the UI is intuitive, uses plain language, includes tutorials, and supports diverse user needs, making the system easier to use than current manual processes.			
Traceability	Use cases	UC-001, UC-002, UC-003, UC-005			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**			
Requirement #	PUUR2	Type	Functional	Non-Functional

Creation	10/12/2025, 9:21pm		User		✓
Modification			System		
Description	Minimize physical interaction requirements during normal operation				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Refined Into	PUSR4, PUSR5				
Justify why PUUR2 can be completely covered by PUSR4, PUSR5	PUSR4 enables touch-free operation, and PUSR5 automates common tasks, minimizing physical interaction during normal use.				
Traceability	Use cases	UC-001, UC-002			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PUUR3		Type	Functional	Non-Functional
Creation	10/12/2025, 9:22pm		User		✓
Modification			System		
Description	Training and configuration interfaces must be accessible to non-technical staff				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Refined Into	PUSR6, PUSR7, PUSR8				
Justify why PUUR3 can be completely covered by PUSR6, PUSR7, PUSR8	These requirements ensure that configuration interfaces are simple, guided, and modifiable without programming, making them accessible to non-technical staff.				
Traceability	Use cases	UC-003			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**			
Requirement #	PUUR4	Type	Functional	Non-Functional
Creation	10/12/2025, 9:22pm	User		✓

Modification				System		
Description	Design system to accommodate workers with varying technical skills and possible disabilities				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Refined Into	PUSR9, PUSR10, PUSR11, PUSR12				
Justify why PUUR4 can be completely **covered by PUSR9, PUSR10, PUSR11, PUSR12**	These requirements ensure accessibility through visual aids, iconography, compliance standards, and input flexibility, supporting users with varying skills and disabilities.				
Traceability	Use cases	UC-001, UC-002, UC-003, UC-005			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PPUR1			Type	Functional	Non-Functional
Creation	10/12/2025, 9:23pm			User		✓
Modification				System		
Description	Process video feeds and provide feedback with minimal latency to prevent manufacturing errors				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Refined Into	PPSR1, PPSR2				
Justify why PPUR1 can be completely **covered by PPSR1, PPSR2**	PPSR1 ensures fast feedback (<200ms), and PPSR2 prioritizes real-time tasks, preventing manufacturing errors due to latency.				
Traceability	Use cases	UC-001, UC-002			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**			
Requirement #	PPUR2	Type	Functional	Non-Functional
Creation	10/12/2025, 9:24pm	User		✓
Modification		System		

Description	Maintain high accuracy in both QR code detection and process monitoring for aerospace quality standards				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Refined Into		PPSR3			
Justify why PPUR2 can be completely covered by PPSR3		PPSR3 enforces high QR code detection accuracy, meeting aerospace-level quality standards.			
Traceability	Use cases	UC-001, UC-002			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PPUR3		Type	Functional	Non-Functional
Creation	10/12/2025, 9:25pm		User		✓
Modification			System		
Description	Function effectively with lower-quality, cost-effective camera equipment				
Priority	Highest	High	**Medium ✓**	Low	Lowest
This Req. is Refined Into		PPSR5, PPSR6			
Justify why PPUR3 can be completely covered by PPSR5, PPSR6		These requirements ensure the system works with low-cost cameras and adapts to lighting, fulfilling the need for cost-effective deployment.			
Traceability	Use cases	UC-001, UC-002, UC-003			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**			
Requirement #	PDUR1	Type	Functional	Non-Functional
Creation	10/12/2025, 9:25pm	User		✓
Modification		System		
Description	All processing and data storage must remain on-premises with no cloud dependencies			

Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Refined Into		PDSR1, PDSR2			
Justify why PDUR1 can be completely covered by PDSR1, PDSR2		These requirements ensure all data and processing remain on-premises, meeting strict security and compliance needs.			
Traceability	Use cases	UC-001, UC-002, UC-003			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PDUR2		Type	Functional	Non-Functional
Creation	10/12/2025, 9:25pm		User		✓
Modification			System		
Description	Reliable operation in industrial factory conditions with minimal downtime				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Refined Into		PDSR3, PDSR4			
Justify why PDUR2 can be completely covered by PDSR3, PDSR4		PDSR3 ensures high uptime, and PDSR4 validates operation under factory conditions, supporting reliability.			
Traceability	Use cases	UC-001, UC-002			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	ODUR1		Type	Functional	Non-Functional
Creation	10/12/2025, 9:28pm		User		✓
Modification			System		
Description	Utilize open-source libraries and frameworks to minimize licensing costs				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Refined Into		ODSR1, ODSR2			
Justify why ODUR1 can be completely		ODSR1 mandates open-source use, and ODSR2 documents licenses, minimizing costs and ensuring transparency.			

covered by ODSR1, ODSR2		
Traceability	Use cases	N/A
	Test cases	Not yet covered

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	OOUR1		Type	Functional	Non-Functional
Creation	10/12/2025, 9:33pm		User		✓
Modification			System		
Description	Minimize hardware and deployment costs while meeting performance requirements				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Refined Into	OOSR1, OOSR2				
Justify why OOUR1 can be completely **covered by OOSR1, OOSR2**	OOSR1 ensures performance on standard hardware, and OOSR2 allows customizable deployment, reducing hardware costs.				
Traceability	Use cases	N/A			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	OOUR2		Type	Functional	Non-Functional
Creation	10/12/2025, 9:34pm		User		✓
Modification			System		
Description	Support flexible deployment configurations for different workstation setups				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Refined Into	OOSR2, OOSR3				
Justify why OOUR2 can be completely **covered by OOSR2, OOSR3**	These requirements support flexible deployment and phased rollout, accommodating different workstation setups.				
Traceability	Use cases	UC-001, UC-002			

	Test cases	Not yet covered

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	OOUR3		Type	Functional	Non-Functional
Creation	10/12/2025, 9:34pm		User		✓
Modification			System		
Description	Support gradual implementation with minimal disruption to existing operations				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Refined Into	OOSR3, OOSR4				
Justify why OOUR3 can be completely covered by OOSR3, OOSR4	OOSR3 supports gradual implementation, and OOSR4 ensures compatibility with legacy formats, minimizing disruption.				
Traceability	Use cases	N/A			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	EIUR1		Type	Functional	Non-Functional
Creation	10/12/2025, 9:35pm		User		✓
Modification			System		
Description	Maintain compatibility with existing KMD Technology Solutions software interfaces				
Priority	Highest	High	**Medium ✓**	Low	Lowest
This Req. is Refined Into	EISR1, EISR2				
Justify why EIUR1 can be completely covered by EISR1, EISR2	EISR1 ensures API compatibility, and EISR2 maintains consistent data formats, fulfilling integration needs with KMD systems.				
Traceability	Use cases	N/A			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	EIUR2		Type	Functional	Non-Functional
Creation	10/12/2025, 9:36pm		User		✓
Modification			System		
Description	Publish comprehensive JSON schema for scan results and system outputs				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Refined Into		EISR3, EISR4			
Justify why EIUR2 can be completely covered by EISR3, EISR4		EISR3 defines the JSON schema, and EISR4 version-controls it, ensuring reliable and documented data exchange.			
Traceability	Use cases	UC-001, UC-002, UC-003			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	EIUR3		Type	Functional	Non-Functional
Creation	10/12/2025, 9:37pm		User		✓
Modification			System		
Description	Support integration with existing manufacturing systems and processes				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Refined Into		EISR5, EISR6			
Justify why EIUR3 can be completely covered by EISR5, EISR6		EISR5 supports integration with MES and ERP tools, and EISR6 allows configuration to match factory workflows, fulfilling broader interoperability needs.			
Traceability	Use cases	UC-001, UC-002			
	Test cases	Not yet covered			

3. System Functional Requirements

Project Name	**AIBI Computer Vision for Manufacturing**			
Requirement #	SFR1	Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm	User		

Modification	12/1/2025, 1:12pm			System	✓
Description	Fixed-station cameras shall detect all QR codes visible in a single frame				
Priority	Highest ✓	**High**	Medium	Low	Lowest
This Req. is Engineered From	UFR1				
Justify why meeting SFR1 Can contribute to the fulfillment of UFR1	SFR1 enables the system to detect and decode QR codes in real time, directly fulfilling UFR1's need for automated QR code processing at each workstation.				
Traceability	Use cases	UC-001			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR2			Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm			User		
Modification	12/1/2025, 2:40pm			System	✓	
Description	Provide per-workstation field mapping to assign what fields the camera has to detect from the QR code				
Priority	Highest ✓	**High**	Medium	Low	Lowest
This Req. is Engineered From	UFR2				
Justify why meeting SFR2 Can contribute to the fulfillment of UFR2	SFR2 provides a configurable field mapping to assign what data the camera is supposed to find, fulfilling UFR2's requirement for flexible data association.				
Traceability	Use cases	UC-001			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**			
Requirement #	SFR4	Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm	User		
Modification		System	✓	
Description	Aggregate decoded and recognized values into a structured JSON document per scan event			

Priority	Highest ✓	**High**	Medium	Low	Lowest
This Req. is Engineered From		UFR1, UFR4			
Justify why meeting SFR4 Can contribute to the fulfillment of UFR1 and UFR4		SFR4 aggregates decoded QR code data into structured JSON, fulfilling UFR1's need for data formatting and UFR4's requirement for downstream system integration.			
Traceability	Use cases	UC-001			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR5		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification	12/1/2025, 2:25pm		System	✓	
Description	Support real-time delivery via keyboard-emulations mode				
Priority	Highest ✓	**High**	Medium	Low	Lowest
This Req. is Engineered From		UFR9			
Justify why meeting SFR5 Can contribute to the fulfillment of UFR9		SFR5 supports integration via keyboard emulation, fulfilling UFR9's need for seamless interoperability with KMD's workflow system.			
Traceability	Use cases				
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR6		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System	✓	
Description	Detect and report start/completion of defined manufacturing steps and validate configured sequence				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From		UFR5			
Justify why meeting SFR6 Can contribute to the fulfillment of UFR5		SFR6 detects and validates manufacturing steps in sequence, directly fulfilling UFR5's need for process monitoring and error prevention.			

Traceability	Use cases	UC-002, UC-003
	Test cases	

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR7		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification	2/17/2026		System	✓	
Description	Provide per workstation UI overlay with live video				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From		UFR6			
Justify why meeting SFR7 Can contribute to the fulfillment of UFR6		SFR7 provides visual overlay fulfilling UFR6's requirement to have live video			
Traceability	Use cases	UC-001, UC-002, UC-003			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR8		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System	✓	
Description	Provide an in-site capture tool for non-technical users to record reference video, mark step start/end, label steps, and save training sessions with metadata				
Priority	Highest	**High**	Medium ✓	Low	Lowest
This Req. is Engineered From		UFR8			
Justify why meeting SFR8 Can contribute to the fulfillment of UFR8		SFR8 enables non-technical users to record and label training videos, fulfilling UFR8's need for an intuitive training interface.			
Traceability	Use cases	UC-003			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**

Requirement #	SFR9			Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm			User		
Modification				System	✓	
Description	Models must tolerate natural worker variations and provide configurable sensitivity per step to reduce nuisance alerts					
Priority	Highest	**High ✓**	Medium	Low	Lowest	
This Req. is Engineered From		UFR8				
Justify why meeting SFR9 Can contribute to the fulfillment of UFR8		SFR9 ensures the system tolerates natural worker variations and allows sensitivity adjustments, fulfilling UFR8's need for flexible and user-friendly training.				
Traceability	Use cases	UC-002, UC-003				
	Test cases					

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR10			Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm			User		
Modification				System	✓	
Description	The system shall log every detected deviation from the step process including errors.					
Priority	Highest	**High ✓**	Medium	Low	Lowest	
This Req. is Engineered From		UFR5, UFR7				
Justify why meeting SFR10 Can contribute to the fulfillment of UFR5 and UFR7		SFR10 logs all process violations, fulfilling UFR5's need for step validation and UFR7's requirement for error tracking and review.				
Traceability	Use cases	UC-002				
	Test cases					

Project Name	**AIBI Computer Vision for Manufacturing**			
Requirement #	SFR11	Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm	User		
Modification		System	✓	

Description	Persist scan and event data locally with audit metadata and support replay/synchronization when downstream systems are unavailable				
Priority	Highest	High	**Medium ✓**	Low	Lowest
This Req. is Engineered From	UFR10				
Justify why meeting SFR11 Can contribute to the fulfillment of UFR7	SFR11 provides local data storage and replay capabilities, fulfilling UFR10's requirement for decoupled data handling.				
Traceability	Use cases				
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR12		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System	✓	
Description	Publish stable JSON schema and human readable documentation for scan and event payloads including examples and required fields				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From	UFR4, UFR9				
Justify why meeting SFR12 Can contribute to the fulfillment of UFR4 and UFR 9	SFR12 publishes a stable JSON schema and documentation, fulfilling UFR4's need for structured output and UFR9's requirement for integration support.				
Traceability	Use cases				
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR13		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification	12/1/2025, 2:25pm		System	✓	
Description	Provide a simulated workstation environment to replay camera feeds				
Priority	Highest	High	**Medium ✓**	Low	Lowest

This Req. is Engineered From		UFR11
Justify why meeting SFR13 Can contribute to the fulfillment of UFR11		SFR13 provides a simulated environment for testing, fulfilling UFR11's need for validation before production deployment.
Traceability	Use cases	UC-005
	Test cases	Not yet covered

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR14		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System	✓	
Description	Provide tools and storage to capture, tag, version, and export labeled datasets for model training, including metadata				
Priority	Highest	High	**Medium ✓**	Low	Lowest
This Req. is Engineered From		UFR12			
Justify why meeting SFR14 Can contribute to the fulfillment of UFR12		SFR14 offers tools for dataset management, fulfilling UFR12's need to support model training and development.			
Traceability	Use cases	UC-003			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR15		Type	Functional	Non-Functional
Creation	10/27/2025, 9:35am		User		
Modification			System	✓	
Description	Fixed-station cameras shall decode all QR codes visible in a single frame				
Priority	Highest ✓	**High**	Medium	Low	Lowest
This Req. is Engineered From		UFR1			
Justify why meeting SFR1 Can contribute to the fulfillment of UFR1		SFR15 enables the system to decode all of the QR codes fulfilling part of UFR1			
Traceability	Use cases	UC-001			

	Test cases	Not yet covered

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR16		Type	Functional	Non-Functional
Creation	10/27/2025, 9:35am		User		
Modification			System	✓	
Description	Fixed-station cameras shall output all QR codes visible in a single frame				
Priority	Highest ✓	**High**	Medium	Low	Lowest
This Req. is Engineered From	UFR1				
Justify why meeting SFR1 Can contribute to the fulfillment of UFR1	SFR16 allows the system to output all of the QR codes to go where they need to go, partially fulfilling UFR1.				
Traceability	Use cases	UC-001			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR17		Type	Functional	Non-Functional
Creation	12/1/25, 2:25pm		User		
Modification			System	✓	
Description	Provide a simulated workstation environment to inject synthetic QR codes				
Priority	Highest ✓	**High**	Medium	Low	Lowest
This Req. is Engineered From	UFR11				
Justify why meeting SFR17 Can contribute to the fulfillment of UFR11	SFR17 provides a simulated workstation environment to inject synthetic QR codes into thus fulfilling UFR11's needs to validate before it hits production line.				
Traceability	Use cases	UC-005			
	Test cases	TC-SFR17-MAN-01, TC-SFR17-AUTO-01			

Project Name	**AIBI Computer Vision for Manufacturing**			
Requirement #	SFR18	Type	Functional	Non-Functional
Creation	12/1/2025, 2:25pm	User		

Modification			System	✓	
Description	Provide a simulated workstation environment to verify keyboard-emulation outputs before production deployment				
Priority	Highest ✓	**High**	Medium	Low	Lowest
This Req. is Engineered From	UFR11				
Justify why meeting SFR18 Can contribute to the fulfillment of UFR11	SFR18 provides a simulated workstation to verify keyboard-emulation outputs thus fulfilling UFR11's requirement of validation before it hits production				
Traceability	Use cases	UC-005			
	Test cases	TC-SFR18-MAN-01, TC-SFR18-AUTO-01			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR19		Type	Functional	Non-Functional
Creation	2/17/2026		User		
Modification			System	✓	
Description	Provide a message to the user for all errors in the step processing on screen				
Priority	Highest ✓	**High**	Medium	Low	Lowest
This Req. is Engineered From	UFR13				
Justify why meeting SFR19 Can contribute to the fulfillment of UFR13	SFR19 provides a message for all errors on a screen, thus fulfilling UFR13				
Traceability	Use cases	Not yet covered			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR20		Type	Functional	Non-Functional
Creation	2/17/2026		User		
Modification			System	✓	
Description	Provide a list of all steps in the process on the UI				
Priority	Highest ✓	**High**	Medium	Low	Lowest

This Req. is Engineered From		UFR14
Justify why meeting SFR20 Can contribute to the fulfillment of UFR14		UFR14 requires that all steps are listed in the UI, which SFR20 fulfills
Traceability	Use cases	Not yet covered
	Test cases	Not yet covered

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR21		Type	Functional	Non-Functional
Creation	2/17/2026		User		
Modification			System	✓	
Description	Provide a visual marker in the form of a green checkmark to indicate if a step was completed				
Priority	Highest ✓	**High**	Medium	Low	Lowest
This Req. is Engineered From		UFR15			
Justify why meeting SFR21 Can contribute to the fulfillment of UFR15		UFR 15 requires a visual marker for completed steps, which SFR21 completes			
Traceability	Use cases	Not yet covered			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR22		Type	Functional	Non-Functional
Creation	2/17/2026		User		
Modification			System	✓	
Description	Provide a visual marker in the form of a red “X” to indicate if a step was missed				
Priority	Highest ✓	**High**	Medium	Low	Lowest
This Req. is Engineered From		UFR15			
Justify why meeting SFR22 Can contribute to the fulfillment of UFR15		UFR 15 requires a visual marker for missed steps, which SFR22 completes			
Traceability	Use cases	Not yet covered			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR23			Type	Functional	Non-Functional
Creation	2/17/2026			User		
Modification				System	✓	
Description	Click on the step in the list to view recorded footage of the step					
Priority	Highest	**High**	Medium ✓	Low	Lowest	
This Req. is Engineered From	UFR16					
Justify why meeting SFR23 Can contribute to the fulfillment of UFR16	SFR23 provides the ability to view recorded footage by clicking on the step, which UFR16 requires					
Traceability	Use cases	Not yet covered				
	Test cases	Not yet covered				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR24			Type	Functional	Non-Functional
Creation	2/17/2026			User		
Modification				System	✓	
Description	Implement Administrative functionality for the in-site capture tool					
Priority	Highest	**High**	Medium ✓	Low	Lowest	
This Req. is Engineered From	UFR17					
Justify why meeting SFR24 Can contribute to the fulfillment of UFR17	UFR17 requires the need for administrative functionality, which SFR24 provides					
Traceability	Use cases	Not yet covered				
	Test cases	Not yet covered				

Project Name	**AIBI Computer Vision for Manufacturing**			
Requirement #	SFR25	Type	Functional	Non-Functional
Creation	2/17/2026	User		
Modification		System	✓	

Description	Integrate command line control for the users				
Priority	Highest	**High**	Medium ✓	Low	Lowest
This Req. is Engineered From		UFR19			
Justify why meeting SFR25 Can contribute to the fulfillment of UFR19		SFR25 provides command line controls, which UFR19 needs			
Traceability	Use cases	Not yet Covered			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR26		Type	Functional	Non-Functional
Creation	2/17/2026		User		
Modification			System	✓	
Description	Provide per workstation UI overlay with step progress indicators				
Priority	Highest	**High** ✓	Medium	Low	Lowest
This Req. is Engineered From		UFR6			
Justify why meeting SFR26 Can contribute to the fulfillment of UFR6		SFR26 provides step progress indicators, which help cover UFR6			
Traceability	Use cases	UC-001, UC-002			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR27		Type	Functional	Non-Functional
Creation	2/17/2026		User		
Modification			System	✓	
Description	Provide per workstation UI overlay with optional audio alerts for violations				
Priority	Highest	**High** ✓	Medium	Low	Lowest
This Req. is Engineered From		UFR6			

Justify why meeting SFR27 Can contribute to the fulfillment of UFR6		SFR27 provides optional audio alerts for violations, which UFR6 needs
Traceability	Use cases	UC-001, UC-002
	Test cases	Not yet covered

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR28		Type	Functional	Non-Functional
Creation	2/17/2026		User		
Modification			System	✓	
Description	Provide a reset key to allow the program to get ready for the next part				
Priority	Highest ✓	**High**	Medium	Low	Lowest
This Req. is Engineered From		UFR20			
Justify why meeting SFR28 Can contribute to the fulfillment of UFR20		SFR28 provides a reset key for the program to restart, which UFR20 requires.			
Traceability	Use cases	Not yet covered			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR29		Type	Functional	Non-Functional
Creation	2/17/2026		User		
Modification			System	✓	
Description	Provide configuration for the user to specify how many QR codes are to be expected in the scanning process				
Priority	Highest ✓	**High**	Medium	Low	Lowest
This Req. is Engineered From		UFR21			
Justify why meeting SFR29 Can contribute to the fulfillment of UFR21		SFR29 provides a configuration for expected QR codes, which is needed by UFR21			
Traceability	Use cases	Not yet covered			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR30			Type	Functional	Non-Functional
Creation	2/17/2026			User		
Modification				System	✓	
Description	Provide configuration for the user to specify what direction they want the QR codes to be scanned					
Priority	Highest ✓	**High**	Medium	Low	Lowest
This Req. is Engineered From	UFR22				
Justify why meeting SFR30 Can contribute to the fulfillment of UFR22	UFR22 requires a configuration for scan direction, which SFR30 provides				
Traceability	Use cases	Not yet covered			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	SFR31			Type	Functional	Non-Functional
Creation	2/17/2026			User		
Modification				System	✓	
Description	Provide configuration for the user to specify if they would like to add a character at the end of the QR data					
Priority	Highest ✓	**High**	Medium	Low	Lowest
This Req. is Engineered From	UFR23				
Justify why meeting SFR31 Can contribute to the fulfillment of UFR23	UFR23 requires configuration to append a character at the end of the data, which SFR31 provides				
Traceability	Use cases	Not yet covered			
	Test cases	Not yet covered			

Project Name	**AIBI Computer Vision for Manufacturing**			
Requirement #	SFR32	Type	Functional	Non-Functional
Creation	2/17/2026	User		
Modification		System	✓	

Description	Provide the ability for the user to enter the workstation that the program starts at				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From		UFR18			
Justify why meeting SFR32 Can contribute to the fulfillment of UFR18		SFR32 provides the ability to enter a workstation at program start which UFR18 needs			
Traceability	Use cases	Not yet covered			
	Test cases	Not yet covered			

4. System Non-functional Requirements

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PUSR1		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The UI shall be intuitive and require no more than 30 minutes of training for first time users				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Engineered From		UFR8, PUUR1			
Justify why meeting PUSR1 Can contribute to the fulfillment of UFR8, PUUR1		PUSR1 ensures the UI is intuitive and quick to learn, fulfilling UFR8's need for usability and PUUR1's goal of minimizing training time.			
Traceability	Use cases	UC-001, UC-002, UC-003			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PUSR2		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The system shall provide clear on-screen guidance and tooltips for all major functions				
Priority	**Highest ✓**	High	Medium	Low	Lowest

This Req. is Engineered From		PUUR1
Justify why meeting PUSR2 Can contribute to the fulfillment of PUUR1		PUSR2 provides tooltips and guidance, fulfilling PUUR1's requirement for ease of use for manufacturing workers.
Traceability	Use cases	UC-001, UC-002, UC-003
	Test cases	

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PUSR3		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The UI shall minimize the amount of effort required to complete required tasks compared to the existing process				
Priority	Highest	**High** ✓	Medium	Low	Lowest
This Req. is Engineered From		UFR8			
Justify why meeting PUSR3 Can contribute to the fulfillment of UFR8		PUSR3 minimizes effort compared to current processes, fulfilling UFR8's goal of improving workflow efficiency.			
Traceability	Use cases	UC-001, UC-002			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PUSR4		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The system shall support touch-free operation through automatic detection and feedback mechanisms where possible				
Priority	Highest	**High** ✓	Medium	Low	Lowest
This Req. is Engineered From		PUUR2, UFR8			
Justify why meeting PUSR4 Can contribute to the fulfillment of PUUR2, UFR8		PUSR4 enables touch-free operation, fulfilling PUUR2's goal of minimizing physical interaction and UFR8's usability needs.			
Traceability	Use cases	UC-001, UC-002			

	Test cases	

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PUSR5			Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm			User		
Modification				System		✓
Description	The system shall automate common workflow steps to reduce manual input from operators				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From	PUUR2, UFR8				
Justify why meeting PUSR5 Can contribute to the fulfillment of PUUR2, UFR8	PUSR5 automates common tasks, fulfilling PUUR2's goal of reducing manual input and UFR8's need for intuitive operation.				
Traceability	Use cases	UC-001, UC-002			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PUSR6			Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm			User		
Modification				System		✓
Description	The configuration interface shall use plain-language prompts and visual menus, avoiding technical jargon				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From	PUUR3, PUUR1, UFR8				
Justify why meeting PUSR6 Can contribute to the fulfillment of PUUR3, PUUR1, UFR8	PUSR6 uses plain language and visual menus, fulfilling PUUR3's accessibility goal and UFR8's requirement for non-technical usability.				
Traceability	Use cases	UC-001, UC-002, UC-003			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**			
Requirement #	PUSR7	Type	Functional	Non-Functional

Creation	10/13/2025, 3:39pm			User		
Modification				System		✓
Description	The system shall include a guided setup and tutorial modes for initial configuration				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From	PUUR3, PUUR1, UFR8				
Justify why meeting PUSR7 Can contribute to the fulfillment of PUUR3, PUUR1, UFR8	PUSR7 provides guided setup and tutorials, fulfilling PUUR3's need for accessible configuration and UFR8's training interface requirement.				
Traceability	Use cases	UC-001, UC-002, UC-003			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PUSR8			Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm			User		
Modification				System		✓
Description	The system shall allow non-administrative users to modify basic settings without needing programming knowledge.				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From	PUUR3, UFR8				
Justify why meeting PUSR8 Can contribute to the fulfillment of PUUR3, UFR8	PUSR8 allows basic settings to be modified without programming, fulfilling PUUR3's accessibility goal and UFR8's usability requirement.				
Traceability	Use cases	UC-003			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**			
Requirement #	PUSR9	Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm	User		
Modification		System		✓
Description	The interface shall include support for large text and high-contrast display options			

Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From		PUUR4, UFR8			
Justify why meeting PUSR9 Can contribute to the fulfillment of PUUR4, UFR8		PUSR9 supports large text and high contrast, fulfilling PUUR4's goal of accommodating users with visual impairments and UFR8's inclusivity requirement.			
Traceability	Use cases	UC-001, UC-002, UC-003, UC-005			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PUSR10		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The UI shall use simple iconography, color coding, and minimal text to accommodate diverse language proficiencies and provide clear workflows for non-technical staff				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From		PUUR4, PUUR1, UFR8			
Justify why meeting PUSR10 Can contribute to the fulfillment of PUUR4, PUUR1, UFR8		PUSR10 uses iconography and minimal text, fulfilling PUUR4's accessibility goal and UFR8's need for a clear, user-friendly interface.			
Traceability	Use cases	UC-001, UC-002, UC-003, UC-005			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PUSR11		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The system shall comply with accessibility standards				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Engineered From		PUUR4			
Justify why meeting PUSR11 Can contribute to the fulfillment of PUUR4		PUSR11 ensures compliance with accessibility standards, fulfilling PUUR4's requirement for inclusive design.			

Traceability	Use cases	UC-001, UC-002, UC-003, UC-005
	Test cases	

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PUSR12		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The system shall support both mouse and keyboard input/navigation for inclusivity				
Priority	**Highest** ✓	High	Medium	Low	Lowest
This Req. is Engineered From	PUUR4				
Justify why meeting PUSR12 Can contribute to the fulfillment of PUUR4	PUSR12 supports both mouse and keyboard input, fulfilling PUUR4's goal of accommodating diverse user needs.				
Traceability	Use cases	UC-001, UC-002, UC-003, UC-005			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PPSR1		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The system shall process video input and deliver analysis feedback within 200 milliseconds				
Priority	**Highest** ✓	High	Medium	Low	Lowest
This Req. is Engineered From	UFR6				
Justify why meeting PPSR1 Can contribute to the fulfillment of UFR6	PPSR1 ensures feedback is delivered within 200ms, fulfilling UFR6's need for real-time alerts at workstations.				
Traceability	Use cases	UC-001, UC-002			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**			
Requirement #	PPSR2	Type	Functional	Non-Functional

Creation	10/13/2025, 3:39pm			User		
Modification				System		✓
Description	The system shall prioritize real-time operations over background tasks to minimize latency				
Priority	**Highest** ✓	High	Medium	Low	Lowest
This Req. is Engineered From		UFR1			
Justify why meeting PPSR2 Can contribute to the fulfillment of UFR1		PPSR2 prioritizes real-time operations, fulfilling UFR1's need for fast QR code processing during manufacturing.			
Traceability	Use cases	UC-001, UC-002			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PPSR3			Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm			User		
Modification				System		✓
Description	The system shall achieve a QR code detection accuracy of at most one error per 100 scans				
Priority	**Highest** ✓	High	Medium	Low	Lowest
This Req. is Engineered From		UFR1			
Justify why meeting PPSR3 Can contribute to the fulfillment of UFR1		PPSR3 enforces high QR code detection accuracy, fulfilling UFR1's requirement for reliable scanning.			
Traceability	Use cases	UC-001			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**			
Requirement #	PPSR4	Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm	User		
Modification		System		✓
Description	The system shall detect and report process deviations with an error margin of less than 1%			

Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Engineered From		UFR5			
Justify why meeting PPSR4 Can contribute to the fulfillment of UFR5		PPSR4 ensures process deviations are detected with less than 1% error, fulfilling UFR5's need for accurate step monitoring.			
Traceability	Use cases	UC-002			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PPSR5		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The system shall maintain required accuracy levels using cameras with resolutions as low as 720p				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Engineered From		UFR3			
Justify why meeting PPSR5 Can contribute to the fulfillment of UFR3		PPSR5 supports QR code detection using 720p cameras, fulfilling UFR3's need for cost-effective hardware compatibility.			
Traceability	Use cases	UC-001, UC-002			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PPSR6		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The system shall automatically adjust detection parameters for varying camera quality and lighting conditions				
Priority	**Highest ✓**	High	Medium	Low	Lowest
This Req. is Engineered From		UFR3			
Justify why meeting PPSR6 Can contribute to the fulfillment of UFR3		PPSR6 adjusts detection parameters for varying conditions, fulfilling UFR3's need for robust performance across environments.			

Traceability	Use cases	UC-001, UC-002
	Test cases	

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PDSR1		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The system shall store all data on local servers within the facility				
Priority	Highest	High	**Medium ✓**	Low	Lowest
This Req. is Engineered From	UFR10, UFR7				
Justify why meeting PDSR1 Can contribute to the fulfillment of UFR10, UFR7	PDSR1 ensures data is stored locally, fulfilling UFR10's need for decoupled processing and UFR7's requirement for secure error logging.				
Traceability	Use cases	UC-001, UC-002			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PDSR2		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The system shall perform all processing operations using on-premises hardware without cloud reliance				
Priority	Highest	High	**Medium ✓**	Low	Lowest
This Req. is Engineered From	UFR10, UFR7				
Justify why meeting PDSR2 Can contribute to the fulfillment of UFR10, UFR7	PDSR2 enforces on-premises processing, fulfilling UFR10's need for secure data handling and UFR7's requirement for reliability.				
Traceability	Use cases	UC-001, UC-002			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PDSR3			Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm			User		
Modification				System		✓
Description	The system shall maintain a minimum uptime of 99% during operational hours					
Priority	Highest	High	**Medium ✓**	Low	Lowest
This Req. is Engineered From	UFR7				
Justify why meeting PDSR3 Can contribute to the fulfillment of UFR7	PDSR3 ensures 99% uptime, fulfilling UFR7's need for consistent error tracking and system availability.				
Traceability	Use cases				
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	PDSR4			Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm			User		
Modification				System		✓
Description	The system shall be tested for stable operation under factory environmental conditions					
Priority	Highest	High	**Medium ✓**	Low	Lowest
This Req. is Engineered From	UFR11				
Justify why meeting PDSR4 Can contribute to the fulfillment of UFR11	PDSR4 validates system operation under factory conditions, fulfilling UFR11's need for reliable pre-deployment testing.				
Traceability	Use cases	UC-005			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**			
Requirement #	ODSR1	Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm	User		
Modification		System		✓

Description	The system shall use open-source libraries or frameworks unless a commercial dependency is explicitly approved				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From		ODUR1			
Justify why meeting ODSR1 Can contribute to the fulfillment of ODUR1		ODSR1 mandates open-source use, fulfilling ODUR1's goal of minimizing licensing costs.			
Traceability	Use cases				
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	ODSR2		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	All open-source components shall be documented, including their licenses and version numbers				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From		ODUR1			
Justify why meeting ODSR2 Can contribute to the fulfillment of ODUR1		ODSR2 documents licenses and versions, fulfilling ODUR1's requirement for transparency and cost control.			
Traceability	Use cases				
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	OOSR1		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The system shall maintain performance benchmarks on standard hardware				
Priority	Highest	**High ✓**	Medium	Low	Lowest

This Req. is Engineered From		OOUR1
Justify why meeting OOSR1 Can contribute to the fulfillment of OOUR1		OOSR1 ensures performance on standard hardware, fulfilling OOUR1's goal of minimizing deployment costs.
Traceability	Use cases	UC-001, UC-002, UC-003, UC-005
	Test cases	

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	OOSR2		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The installation process shall include customizable deployment options				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From		OOUR1, OOUR2			
Justify why meeting OOSR2 Can contribute to the fulfillment of OOUR1, OOUR2		OOSR2 provides customizable deployment options, fulfilling OOUR1's cost-efficiency goal and OOUR2's need for flexible setups.			
Traceability	Use cases	UC-001, UC-002, UC-003			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	OOSR3		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The system shall support phased deployment alongside existing systems				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From		OOUR2, OOUR3			
Justify why meeting OOSR3 Can contribute to the fulfillment of OOUR2, OOUR3		OOSR3 supports phased deployment, fulfilling OOUR2's flexibility goal and OOUR3's need for gradual implementation.			
Traceability	Use cases				
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	OOSR4		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The system shall maintain backward compatibility with legacy data formats				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From	OOUR3				
Justify why meeting OOSR4 Can contribute to the fulfillment of OOUR3	OOSR4 ensures backward compatibility, fulfilling OOUR3's goal of minimizing disruption to existing operations.				
Traceability	Use cases	UC-001, UC-002			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	EISR1		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The system shall integrate with all existing KMD Technology Solutions APIs and communication protocols				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From	EIUR1				
Justify why meeting EISR1 Can contribute to the fulfillment of EIUR1	EISR1 ensures integration with KMD APIs, fulfilling EIUR1's requirement for software compatibility.				
Traceability	Use cases	UC-001, UC-002			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**			
Requirement #	EISR2	Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm	User		

Modification			System		✓
Description	The system shall maintain consistent data exchange formats used by KMD				
Priority	Highest	High	**Medium ✓**	Low	Lowest
This Req. is Engineered From		EIUR1			
Justify why meeting EISR2 Can contribute to the fulfillment of EIUR1		EISR2 maintains consistent data formats, fulfilling EIUR1's goal of seamless communication with KMD systems.			
Traceability	Use cases	UC-001, UC-002			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	EISR3		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The system shall define and publish a complete JSON schema for all output data				
Priority	Highest	**High ✓**	Medium	Low	Lowest
This Req. is Engineered From		EIUR2			
Justify why meeting EISR3 Can contribute to the fulfillment of EIUR2		EISR3 defines a complete JSON schema, fulfilling EIUR2's requirement for structured output documentation.			
Traceability	Use cases	UC-001, UC-002			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	EISR4		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The JSON schema shall be version-controlled and included in system documentation				
Priority	Highest	**High ✓**	Medium	Low	Lowest

This Req. is Engineered From		EIUR2
Justify why meeting EISR4 Can contribute to the fulfillment of EIUR2		EISR4 version-controls the schema, fulfilling EIUR2's goal of maintaining reliable and traceable data exchange.
Traceability	Use cases	
	Test cases	

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	EISR5		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The system shall support data exchange with existing Manufacturing Execution Systems and ERP tools				
Priority	Highest	**High** ✓	Medium	Low	Lowest
This Req. is Engineered From		EIUR3			
Justify why meeting EISR5 Can contribute to the fulfillment of EIUR3		EISR5 supports integration with MES and ERP tools, fulfilling EIUR3's goal of broader system interoperability.			
Traceability	Use cases	UC-001, UC-002			
	Test cases				

Project Name	**AIBI Computer Vision for Manufacturing**				
Requirement #	EISR6		Type	Functional	Non-Functional
Creation	10/13/2025, 3:39pm		User		
Modification			System		✓
Description	The system shall be configurable to align with existing factory workflows and data pipelines				
Priority	Highest	**High** ✓	Medium	Low	Lowest
This Req. is Engineered From		EIUR3			
Justify why meeting EISR6 Can contribute to the fulfillment of EIUR3		EISR6 allows configuration to match factory workflows, fulfilling EIUR3's requirement for alignment with existing processes.			
Traceability	Use cases	UC-001, UC-002			

Team 9

	Test cases	

5. Trace Table

Project Name: AIBI CV for Manufacturing			
User Requirements		System Requirements	
Req ID	Description	Req ID	Description
UFR1	Fixed cameras at each workstation must detect, decode, and process multiple QR codes from objects presented within the camera's field of view	SFR1	Fixed-station cameras shall detect all QR codes visible in a single frame
		SFR4	Aggregate decoded and recognized values into a structured JSON document per scan event
		PPSR2	The system shall prioritize real-time operations over background tasks to minimize latency
		PPSR3	The system shall achieve a QR code detection accuracy of at most one error per 100 scans.
		SFR15	Fixed-station cameras shall decode all QR codes visible in a single frame
		SFR16	Fixed-station cameras shall output all QR codes visible in a single frame
UFR2	Provide configurable field mapping system to define required and optional QR code data fields per workstation	SFR2	Provide per-workstation configurable mapping to assign QR code spatial position to named data fields
UFR3	Identify and track QR codes on large parts rather than sheets of paper	PPSR5	The system shall maintain required accuracy levels using cameras with resolutions as low as 720p
		PPSR6	The system shall automatically adjust detection parameters for varying camera quality and lighting conditions
UFR4	Aggregate all decoded values into structured JSON payload for downstream system integration	SFR4	Aggregate decoded and recognized values into a structured JSON document per scan event
		SFR12	Publish stable JSON schema and human readable documentation for scan and event payloads including examples and required fields

		EISR3	The system shall define and publish a complete JSON schema for all output data
		EISR4	The JSON schema shall be version-controlled and included in system documentation
UFR5	Monitor and validate that manufacturing steps are performed in correct sequence at each workstation	SFR6	Detect and report start/completion of defined manufacturing steps and validate configured sequence
		SFR10	The system shall log every detected deviation from the step process including errors.
		PPSR4	The system shall detect and report process deviations with an error margin of less than 1%
UFR6	Provide immediate visual feedback and alerts for process status and violations on terminals at workstations	SFR7	Provide per workstation UI overlay with live video
		SFR26	Provide per workstation UI overlay with step progress indicators
		SFR27	Provide per workstation UI overlay with optional audio alerts for violations
		PPSR1	The system shall process video input and deliver analysis feedback within 200 milliseconds
UFR7	Log all errors caught by the system for the users to review	SFR10	The system shall log every detected deviation from the step process including errors.
		SFR11	Persist scan and event data locally with audit metadata and support replay/synchronization when downstream systems are unavailable
		PDSR1	The system shall store all data on local servers within the facility
		PDSR2	The system shall perform all processing operations using on-

			premises hardware without cloud reliance
		PDSR3	The system shall maintain a minimum uptime of 99% during operational hours
UFR8	Provide intuitive UI for non-technical users to train the system on correct manufacturing processes	SFR8	Provide an in-site capture tool for non-technical users to record reference video, mark step start/end, label steps, and save training sessions with metadata
		SFR9	Models must tolerate natural worker variations and provide configurable sensitivity per step to reduce nuisance alerts
		PSUR1	The UI shall be intuitive and require no more than 30 minutes of training for first time users
		PSUR2	The system shall provide clear on-screen guidance and tooltips for all major functions
		PUSR3	The UI shall minimize the amount of effort required to complete required tasks compared to the existing process
		PUSR4	The system shall support touch-free operation through automatic detection and feedback mechanisms where possible
		PUSR5	The system shall automate common workflow steps to reduce manual input from operators
		PUSR6	The configuration interface shall use plain-language prompts and visual menus, avoiding technical jargon
		PUSR7	The system shall include a guided setup and tutorial modes for initial configuration
		PUSR8	The system shall allow non-administrative users to modify basic settings without needing programming knowledge

		PUSR9	The interface shall include support for large text and high-contrast display options
		PUSR10	The UI shall use simple iconography, color coding, and minimal text to accommodate diverse language proficiencies and provide clear workflows for non-technical staff
UFR9	Seamless integration with existing KMD Technology Solutions workflow management system	SFR5	Support real-time delivery via local REST API, local DB persistence, and keyboard-emulation mode for KMD integration
		SFR12	Publish stable JSON schema and human readable documentation for scan and event payloads including examples and required fields
UFR10	Implement intermediate data storage to decouple computer vision processing from KMD data consumption	SFR11	Persist scan and event data locally with audit metadata and support replay/synchronization when downstream systems are unavailable
		PDSR1	The system shall store all data on local servers within the facility
		PDSR2	The system shall perform all processing operations using on-premises hardware without cloud reliance
UFR11	Provide simulated workstation environment for development and validation before production deployment	SFR13	Provide a simulated workstation environment to replay camera feeds, inject synthetic QR codes, and verify API/keyboard-emulation outputs before production deployment
		PDSR4	The system shall be tested for stable operation under factory environmental conditions
		SFR17	Provide a simulated workstation environment to inject synthetic QR codes
		SFR18	Provide a simulated workstation environment to verify keyboard-emulation outputs before production deployment

UFR12	Provide tools for capturing, organizing, and managing training datasets for model development	SFR14	Provide tools and storage to capture, tag, version, and export labeled datasets for model training, including metadata
UFR13	Show all errors in the process on screen	SFR19	Provide a message to the user for all errors in the step processing on screen
UFR14	Show all steps for the process on screen	SFR20	Provide a list of all steps in the process on the UI
UFR15	Have a visual marker for completed or missed steps	SFR21	Provide a visual marker in the form of a green checkmark to indicate if a step was completed
		SFR22	Provide a visual marker in the form of a red “X” to indicate if a step was missed
UFR16	Click on check box to show the step	SFR23	Click on the step in the list to view recorded footage of the step
UFR17	Admin function for the in-site capture tool	SFR24	Implement Administrative functionality for the in-site capture tool
UFR18	Pass in what workstation when running the tool	SFR32	Provide the ability for the user to enter the workstation that the program starts in
UFR19	Command line control	SFR25	Integrate command line control for the users
UFR20	Provide a reset button for transition to the next part	SFR28	Provide a reset key to allow the program to get ready for the next part
UFR21	Have configuration for the number of QR codes expected	SFR29	Provide configuration for the user to specify how many QR codes are to be expected in the scanning process
UFR22	Have configuration for the direction of which the user wants them to be scanned	SFR30	Provide configuration for the user to specify what direction they want the QR codes to be scanned
UFR23	Have configuration for if they want to add a character at the end of the QR data	SFR31	Provide configuration for the user to specify if they would like to add a character at the end of the QR data
PUUR1	System must be extremely easy to use for manufacturing workers with minimal training, needs to be	PUSR1	The UI shall be intuitive and require no more than 30 minutes of training for first time users

	easier than what is currently being done	PUSR2	The system shall provide clear on-screen guidance and tooltips for all major functions
		PUSR6	The configuration interface shall use plain-language prompts and visual menus, avoiding technical jargon
		PUSR7	The system shall include a guided setup and tutorial modes for initial configuration
		PUSR10	The UI shall use simple iconography, color coding, and minimal text to accommodate diverse language proficiencies and provide clear workflows for non-technical staff
PUUR2	Minimize physical interaction requirements during normal operation	PUSR4	The system shall support touch-free operation through automatic detection and feedback mechanisms where possible
		PUSR5	The system shall automate common workflow steps to reduce manual input from operators
PUUR3	Training and configuration interfaces must be accessible to non-technical staff	PUSR6	The configuration interface shall use plain-language prompts and visual menus, avoiding technical jargon
		PUSR7	The system shall include a guided setup and tutorial modes for initial configuration
		PUSR8	The system shall allow non-administrative users to modify basic settings without needing programming knowledge.
PUUR4	Design system to accommodate workers with varying technical skills and possible disabilities	PUSR9	The interface shall include support for large text and high-contrast display options
		PUSR10	The UI shall use simple iconography, color coding, and minimal text to accommodate diverse language proficiencies and provide clear workflows for non-technical staff
		PUSR11	The system shall comply with accessibility standards

		PUSR12	The system shall support both mouse and keyboard input/navigation for inclusivity
PPUR1	Process video feeds and provide feedback with minimal latency to prevent manufacturing errors	PPSR1	The system shall process video input and deliver analysis feedback within 200 milliseconds
		PPSR2	The system shall prioritize real-time operations over background tasks to minimize latency
PPUR2	Maintain high accuracy in both QR code detection and process monitoring for aerospace quality standards	PPSR3	The system shall achieve a QR code detection accuracy of at most one error per 100 scans
PPUR3	Function effectively with lower-quality, cost-effective camera equipment	PPSR5	The system shall maintain required accuracy levels using cameras with resolutions as low as 720p
		PPSR6	The system shall automatically adjust detection parameters for varying camera quality and lighting condition
PDUR1	All processing and data storage must remain on-premises with no cloud dependencies or external services	PDSR1	The system shall store all data on local servers within the facility
		PDSR2	The system shall perform all processing operations using on-premises hardware without cloud reliance
PDUR2	Reliable operation in industrial factory conditions with minimal downtime	PDSR3	The system shall maintain a minimum uptime of 99% during operational hours
		PDSR4	The system shall be tested for stable operation under factory environmental conditions
ODUR1	Utilize open-source libraries and frameworks to minimize licensing costs	ODSR1	The system shall use open-source libraries or frameworks unless a commercial dependency is explicitly approved
		ODSR2	All open-source components shall be documented, including their licenses and version numbers

OOUR1	Minimize hardware and deployment costs while meeting performance requirements	OOSR1	The system shall maintain performance benchmarks on standard hardware
		OOSR2	The installation process shall include customizable deployment options
OOUR2	Support flexible deployment configurations for different workstation setups	OOSR2	The installation process shall include customizable deployment options
		OOSR3	The system shall support phased deployment alongside existing systems
OOUR3	Support gradual implementation with minimal disruption to existing operations	OOSR3	The system shall support phased deployment alongside existing systems
		OOSR4	The system shall maintain backward compatibility with legacy data formats
EIUR1	Maintain compatibility with existing KMD Technology Solutions software interfaces	EISR1	The system shall integrate with all existing KMD Technology Solutions APIs and communication protocols
		EISR2	The system shall maintain consistent data exchange formats used by KMD
EIUR2	Publish comprehensive JSON schema for scan results and system outputs	EISR3	The system shall define and publish a complete JSON schema for all output data
		EISR4	The JSON schema shall be version-controlled and included in system documentation
EIUR3	Support integration with existing manufacturing systems and processes	EISR5	The system shall support data exchange with existing Manufacturing Execution Systems and ERP tools
		EISR6	The system shall be configurable to align with existing factory workflows and data pipelines

Appendix T

Project Name	**AIBI CV for Manufacturing**

Test Suit	QR Scanner Manual Functional	
Test Case ID	TC-SFR1-MAN-01	
What to Test	Verify that the live camera feed detects and decodes all visible QR codes within the camera's field of view in real time.	
Test Data Input	Printed QR codes placed in camera FOV.	
Expected Result	All visible codes appear in UI/output.	
Traceability	Relevant User Req.(s)	UFR1
	Relevant System Req.(s)	SFR1, SFR15, SFR16
	Relevant Use Case.(s)	UC-001

Project Name	**AIBI CV for Manufacturing**	
Test Suit	QR Scanner Manual Functional	
Test Case ID	TC-SFR2-MAN-01	
What to Test	Verify that an admin can configure options through the workstation config so decoded QR values map to the correct logical fields.	
Test Data Input	Workstation config with fields: "workstation_id": "workstation_11", "expected_qr_count": 6, "scan_direction": "row-major", "append_key": "None", "camera_index": 0	
Expected Result	Output JSON contains "workstation_id": "workstation_11", "expected_qr_count": 6, "scan_direction": "row-major", "append_key": "None", "camera_index": 0	
Traceability	Relevant User Req.(s)	UFR2
	Relevant System Req.(s)	SFR2
	Relevant Use Case.(s)	UC-001

Project Name	**AIBI CV for Manufacturing**	
Test Suit	QR Scanner Manual Functional	
Test Case ID	TC-SFR4-MAN-01	
What to Test	Verify that a live scan event produces a single, correct JSON file containing all scanned data and metadata.	
Test Data Input	QR Codes. Config file.	
Expected Result	One JSON record created with all data and metadata.	
Traceability	Relevant User Req.(s)	UFR1, UFR4

	Relevant System Req.(s)	SFR4, SFR11
	Relevant Use Case.(s)	UC-001

Project Name	**AIBI CV for Manufacturing**	
Test Suit	QR Scanner Manual Functional	
Test Case ID	TC-SFR5-MAN-01	
What to Test	Verify that keyboard-emulation mode outputs correct keystrokes to a target application (e.g., Excel) after a live scan.	
Test Data Input	Live scan and notepad field in focus.	
Expected Result	Excel displays correct values in order with configered delimiters.	
Traceability	Relevant User Req.(s)	UFR9
	Relevant System Req.(s)	SFR5, SFR18
	Relevant Use Case.(s)	UC-001

Project Name	**AIBI CV for Manufacturing**	
Test Suit	QR Scanner Manual Functional	
Test Case ID	TC-SFR11-MAN-01	
What to Test	Verify that every scan event generates a JSON file persisted to the local output directory with no data loss.	
Test Data Input	Five consecutive scan events with distinct QR code payloads.	
Expected Result	Exactly five JSON files present with correct, matching payloads and sequential timestamps.	
Traceability	Relevant User Req.(s)	UFR7
	Relevant System Req.(s)	SFR11, PDSR1
	Relevant Use Case.(s)	UC-002

Commented [ND3]: @Miller, Collin Added

Project Name	**AIBI CV for Manufacturing**
Test Suit	QR Scanner Manual Functional
Test Case ID	TC-SFR12-MAN-01

What to Test	Verify that JSON output files contain all required fields with correct data types matching the expected structure.	
Test Data Input	Three JSON output files from different scans.	
Expected Result	All files contain required fields with correct data types.	
Traceability	Relevant User Req.(s)	UFR4
	Relevant System Req.(s)	SFR12, EISR3
	Relevant Use Case.(s)	UC-001

Commented [ND4]: @Miller, Collin

Project Name	**AIBI CV for Manufacturing**	
Test Suit	QR Scanner Manual Functional	
Test Case ID	TC-SFR15-MAN-01	
What to Test	Verify that the system decodes all QR codes on a multi-code test sheet under typical factory lighting.	
Test Data Input	Printed QR sheet with known values	
Expected Result	All five codes decoded correctly across all conditions.	
Traceability	Relevant User Req.(s)	UFR1
	Relevant System Req.(s)	SFR15
	Relevant Use Case.(s)	UC-001

Project Name	**AIBI CV for Manufacturing**	
Test Suit	QR Scanner Manual Functional	
Test Case ID	TC-SFR16-MAN-01	
What to Test	Verify that the system output includes exactly one entry per visible QR code with no missed codes or duplicates.	
Test Data Input	Live multi-code scan.	
Expected Result	Exactly three entries, one per visible code, no duplicates.	
Traceability	Relevant User Req.(s)	UFR1
	Relevant System Req.(s)	SFR16

	Relevant Use Case.(s)	UC-001

Project Name	AIBI CV for Manufacturing	
Test Suit	System/Manual Functional	
Test Case ID	TC-SFR18-MAN-01	
What to Test	Simulation produces correct keyboard-emulation output.	
Test Data Input	Simulation scenario and sandbox text field.	
Expected Result	Typed text matches expected sequence.	
Traceability	Relevant User Req.(s)	UFR11
	Relevant System Req.(s)	SFR18
	Relevant Use Case.(s)	UC-005

Project Name	AIBI CV for Manufacturing	
Test Suit	Step Verification Manual Functional	
Test Case ID	TC-SFR6-MAN-01	
What to Test	Verify that the system correctly detects when manufacturing steps are performed in the correct sequence and flags out-of-order or skipped steps.	
Test Data Input	Trained process with steps: "Step A", "Step B", "Step C". Operator performs steps in correct order, then in wrong order.	
Expected Result	Correct-order steps transition through IDLE → CORRECT_STEP → CONFIRMED. Out-of-order steps trigger WRONG_ORDER/SKIPPED state with logging.	
Traceability	Relevant User Req.(s)	UFR5
	Relevant System Req.(s)	SFR6, SFR10, PPSR4
	Relevant Use Case.(s)	UC-002

Project Name	AIBI CV for Manufacturing
Test Suit	Step Verification Manual Functional
Test Case ID	TC-SFR7-MAN-01
What to Test	Verify that the Streamlit UI displays a live camera feed with real-time overlay showing current step status, confidence scores, and progress indicators.
Test Data Input	Normal operation with a trained process containing 3+ steps.

Expected Result	Live feed displays with accurate step name, confidence score, and progress bar.	
Traceability	Relevant User Req.(s)	UFR6
	Relevant System Req.(s)	SFR7, SFR26
	Relevant Use Case.(s)	UC-002

Project Name	**AIBI CV for Manufacturing**	
Test Suit	Step Verification Manual Functional	
Test Case ID	TC-SFR8-MAN-01	
What to Test	Verify that a user can define process steps, record reference frames, and save a trained process using the Training mode.	
Test Data Input	New process with three steps	
Expected Result	All three steps are defined, frames recorded, embeddings computed, and process saved as .pkl.	
Traceability	Relevant User Req.(s)	UFR8, UFR12
	Relevant System Req.(s)	SFR8, SFR14
	Relevant Use Case.(s)	UC-003

Project Name	**AIBI CV for Manufacturing**	
Test Suit	Step Verification Manual Functional	
Test Case ID	TC-SFR9-MAN-01	
What to Test	Verify that adjusting the similarity threshold, window size, and required votes sliders changes the system's detection behavior.	
Test Data Input	Default threshold (0.75), then adjusted to 0.9 (strict) and 0.3 (loose).	
Expected Result	Higher thresholds require more precise matches; lower thresholds accept looser matches.	
Traceability	Relevant User Req.(s)	UFR8
	Relevant System Req.(s)	SFR9
	Relevant Use Case.(s)	UC-002, UNC-003

Project Name	**AIBI CV for Manufacturing**

Test Suit	Step Verification Manual Functional	
Test Case ID	TC-SFR10-MAN-01	
What to Test	Verify that the system logs every detected deviation (wrong order, skipped step) with step name and violation type in the run history.	
Test Data Input	3-step process where operator skips step 2 and jumps to step 3.	
Expected Result	Run history shows the violation entry with correct details.	
Traceability	Relevant User Req.(s)	UFR7
	Relevant System Req.(s)	SFR10
	Relevant Use Case.(s)	UC-002

Project Name	**AIBI CV for Manufacturing**	
Test Suit	Step Verification Manual Functional	
Test Case ID	TC-SFR14-MAN-01	
What to Test	Verify that a trained process can be saved as a .pkl file and loaded back with all step data intact.	
Test Data Input	Trained process with steps: "Step A" (with embeddings), "Step B" (with embeddings).	
Expected Result	Loaded process contains all original steps with functional embeddings.	
Traceability	Relevant User Req.(s)	UFR12
	Relevant System Req.(s)	SFR14
	Relevant Use Case.(s)	UC-003

Project Name	**AIBI CV for Manufacturing**	
Test Suit	Step Verification Manual Functional	
Test Case ID	TC-SFR19-MAN-01	
What to Test	Verify that the system displays clear error messages on screen for all process errors (WRONG_ORDER, SKIPPED steps).	
Test Data Input	Operator deliberately triggers a sequence violation.	
Expected Result	A visible warning or error message appears on screen identifying the violation type and step.	
Traceability	Relevant User Req.(s)	UFR13

	Relevant System Req.(s)	SFR19
	Relevant Use Case.(s)	UC-002

Project Name	**AIBI CV for Manufacturing**	
Test Suit	Step Verification Manual Functional	
Test Case ID	TC-SFR20-MAN-01	
What to Test	Verify that the UI displays a complete checklist of all process steps during Operation mode.	
Test Data Input	Process with 4 steps.	
Expected Result	All steps are listed with correct names in the correct order.	
Traceability	Relevant User Req.(s)	UFR14
	Relevant System Req.(s)	SFR20
	Relevant Use Case.(s)	UC-002

Project Name	**AIBI CV for Manufacturing**	
Test Suit	Step Verification Manual Functional	
Test Case ID	TC-SFR21-MAN-01	
What to Test	Verify that completed steps display a green checkmark visual marker in the step list.	
Test Data Input	3-step process.	
Expected Result	Correct visual indicators are shown	
Traceability	Relevant User Req.(s)	UFR15
	Relevant System Req.(s)	SFR21
	Relevant Use Case.(s)	UC-002

Project Name	**AIBI CV for Manufacturing**
Test Suit	Step Verification Manual Functional
Test Case ID	TC-SFR22-MAN-01
What to Test	Verify that skipped or missed steps display a red indicator in the step list.
Test Data Input	3-step process where step 2 is skipped.

Expected Result	Skipped step displays a distinct visual indicator clearly identifying it as missed.	
Traceability	Relevant User Req.(s)	UFR15
	Relevant System Req.(s)	SFR22
	Relevant Use Case.(s)	UC-002

Project Name	**AIBI CV for Manufacturing**	
Test Suit	Step Verification Manual Functional	
Test Case ID	TC-SFR26-MAN-01	
What to Test	Verify that the UI displays progress indicators showing voting status (e.g., votes/required) during step verification.	
Test Data Input	Window size: 5, Required votes: 3.	
Expected Result	Progress indicator shows current votes vs. required (e.g., "2/3 votes").	
Traceability	Relevant User Req.(s)	UFR6
	Relevant System Req.(s)	SFR26
	Relevant Use Case.(s)	UC-002

Project Name	**AIBI CV for Manufacturing**	
Test Suit	Step Verification Manual Functional	
Test Case ID	TC-SFR28-MAN-01	
What to Test	Verify that the restart button resets the step verification state to prepare for the next part/run.	
Test Data Input	A process where 2 of 3 steps have been completed.	
Expected Result	All step states reset. Current step returns to step 1. Previous run preserved in history.	
Traceability	Relevant User Req.(s)	UFR20
	Relevant System Req.(s)	SFR28
	Relevant Use Case.(s)	UC-002

Project Name	**AIBI CV for Manufacturing**
Test Suit	Usability and Operator Workflow

Test Case ID	TC-PUSR1-MAN-01	
What to Test	Verify that a new operator with no prior experience can complete the basic scanning workflow within 30 minutes.	
Test Data Input	New user, pre-configured workstation, task checklist: launch app, perform scan, verify output, configure field mapping.	
Expected Result	Operator completes all tasks within 30 minutes.	
Traceability	Relevant User Req.(s)	UFR8
	Relevant System Req.(s)	PUUR1 · PUSR1
	Relevant Use Case.(s)	UC-001

Project Name	**AIBI CV for Manufacturing**	
Test Suit	Usability and Operator Workflow	
Test Case ID	TC-PUSR2-MAN-01	
What to Test	Verify that the Streamlit UI provides adequate help text (slider descriptions, st.info messages, button labels) for users to understand major functions.	
Test Data Input	System UI. Questionnaire listing 8 major controls: mode selector, record button, stop button, finalize button, save button, load button, restart button, threshold sliders.	
Expected Result	User correctly identifies ≥ 75% of controls (≥ 6 of 8).	
Traceability	Relevant User Req.(s)	PUUR1
	Relevant System Req.(s)	PUSR2
	Relevant Use Case.(s)	UC-001

Project Name	**AIBI CV for Manufacturing**	
Test Suit	Usability and Operator Workflow	
Test Case ID	TC-PUSR3-MAN-01	
What to Test	Verify that the new system requires fewer manual steps than the legacy workflow.	
Test Data Input	Legacy documentation. Same task on both systems (scan three parts, record data).	
Expected Result	New system requires fewer manual steps (≥25% reduction).	
Traceability	Relevant User Req.(s)	UFR8
	Relevant System Req.(s)	PUSR3

	Relevant Use Case.(s)	UC-001

Project Name	AIBI CV for Manufacturing	
Test Suit	Usability and Operator Workflow	
Test Case ID	TC-PUSR4-MAN-01	
What to Test	Verify that the operator rarely needs to touch the UI during a typical QR scan cycle.	
Test Data Input	10 consecutive scan cycles.	
Expected Result	Average ≤ 2 UI interactions per cycle.	
Traceability	Relevant User Req.(s)	PUUR2
	Relevant System Req.(s)	PUSR4
	Relevant Use Case.(s)	UC-001, UC-002

Project Name	AIBI CV for Manufacturing	
Test Suit	Usability and Operator Workflow	
Test Case ID	TC-PUSR10-MAN-01	
What to Test	Verify that users can correctly interpret step status from the emoji markers and color-coded overlays (green/red) used in the UI.	
Test Data Input	Screenshots showing: all-pending state, mid-progress state, violation state.	
Expected Result	Users correctly identify step states ≥ 85% of the time.	
Traceability	Relevant User Req.(s)	PUUR4, PUUR1
	Relevant System Req.(s)	PUSR10
	Relevant Use Case.(s)	UC-002

Project Name	AIBI CV for Manufacturing
Test Suit	System-Level Non-Functional
Test Case ID	TC-PPSR2-MAN-01
What to Test	Verify that feedback appears immediate, operator perceives no delay during scanning or step verification.
Test Data Input	10 consecutive scan/verification cycles. Stopwatch or system logging.

Expected Result	Response time <500 ms. Operator reports no perceived delay.	
Traceability	Relevant User Req.(s)	UFR1
	Relevant System Req.(s)	PPSR2
	Relevant Use Case.(s)	UC-001, UC-002

Project Name	**AIBI CV for Manufacturing**	
Test Suit	System-Level Non-Functional	
Test Case ID	TC-PPSR5-MAN-01	
What to Test	Verify that the system works reliably with a 720p camera with no accuracy degradation.	
Test Data Input	720p camera. Test sheet with five QR codes.	
Expected Result	Accuracy ≥95%.	
Traceability	Relevant User Req.(s)	UFR3
	Relevant System Req.(s)	PPSR5
	Relevant Use Case.(s)	UC-001

Project Name	**AIBI CV for Manufacturing**	
Test Suit	System-Level Non-Functional	
Test Case ID	TC-PDSR1-MAN-01	
What to Test	Verify no data is sent to external servers; all processing is local.	
Test Data Input	Network logs during 1-hour operational cycle.	
Expected Result	Zero outbound connections.	
Traceability	Relevant User Req.(s)	UFR10
	Relevant System Req.(s)	PDSR1
	Relevant Use Case.(s)	UC-001, UC-002

Project Name	**AIBI CV for Manufacturing**
Test Suit	System-Level Non-Functional

Test Case ID	TC-PDSR2-MAN-01	
What to Test	TC-PDSR2-MAN-01	
Test Data Input	Full operational test: launch, scan, process, JSON output, keyboard emulation, step verification.	
Expected Result	All features function identically to online operation.	
Traceability	Relevant User Req.(s)	UFR10
	Relevant System Req.(s)	PDSR2
	Relevant Use Case.(s)	UC-001

Project Name	**AIBI CV for Manufacturing**	
Test Suit	System-Level Non-Functional	
Test Case ID	TC-PDSR4-MAN-01	
What to Test	Verify stable operation for 8+ hours of continuous use with no crashes or memory leaks.	
Test Data Input	8+ hour continuous operation. Resource monitoring logs.	
Expected Result	Zero crashes. Stable memory/CPU.	
Traceability	Relevant User Req.(s)	UFR11
	Relevant System Req.(s)	PDSR4
	Relevant Use Case.(s)	UC-002

Project Name	**AIBI CV for Manufacturing**	
Test Suit	System-Level Non-Functional	
Test Case ID	TC-ODSR2-MAN-01	
What to Test	Verify all open-source components are documented with name, version, and license.	
Test Data Input	Dependency list. OSS documentation section.	
Expected Result	100% of dependencies documented.	
Traceability	Relevant User Req.(s)	ODUR1
	Relevant System Req.(s)	ODSR2
	Relevant Use Case.(s)	UC-005

Project Name	AIBI CV for Manufacturing	
Test Suit	System-Level Non-Functional	
Test Case ID	TC-EISR6-MAN-01	
What to Test	Verify that modifying workstation config JSON files changes the required fields without code changes.	
Test Data Input	Initial config: part_number, serial_number. Modified: part_number, batch_id, inspector_id.	
Expected Result	New config enforced without code changes.	
Traceability	Relevant User Req.(s)	EIUR3
	Relevant System Req.(s)	EISR6
	Relevant Use Case.(s)	UC-005

Project Name	AIBI CV for Manufacturing	
Test Suit	Automated Functional	
Test Case ID	TC-SFR1-AUTO-01	
What to Test	Verify that decode_qr() returns all QR codes from a static test image.	
Test Data Input	200x200 image with 3 QR codes: part_number:PN-001, serial_number:SN-002, batch_id:BATCH-003.	
Expected Result	decode_qr() returns exactly 3 codes with matching payloads.	
Traceability	Relevant User Req.(s)	UFR1
	Relevant System Req.(s)	SFR1, SFR15, SFR16
	Relevant Use Case.(s)	UC-001

Project Name	AIBI CV for Manufacturing	
Test Suit	Automated Functional	
Test Case ID	TC-SFR5-AUTO-01	
What to Test	Verify keystroke builder produces correct sequence with proper delimiters.	
Test Data Input	Payload: {"part_number": "PN-12345", "serial_number": "SN-67890"}. Delimiter: TAB.	
Expected Result	Builder returns correct sequence.	
Traceability	Relevant User Req.(s)	UFR9

	Relevant System Req.(s)	SFR5
	Relevant Use Case.(s)	UC-001

Project Name	AIBI CV for Manufacturing	
Test Suit	Automated Functional	
Test Case ID	TC-SFR11-AUTO-01	
What to Test	Verify that writing scan data to JSON and reading it back preserves all data.	
Test Data Input	Scan event with workstation_id: "workstation_01", barcodes: [{"name": "part_number", "value": "PN-001"}].	
Expected Result	Loaded data matches original with no loss.	
Traceability	Relevant User Req.(s)	UFR7
	Relevant System Req.(s)	SFR11
	Relevant Use Case.(s)	UC-001

Project Name	AIBI CV for Manufacturing	
Test Suit	Automated Functional	
Test Case ID	TC-SFR14-AUTO-01	
What to Test	Verify that serialize_process() and deserialize_process() preserve all training data (step names, embeddings) through a save/load cycle.	
Test Data Input	Process with 2 steps, each with computed centroids (numpy arrays).	
Expected Result	All step names, order, and centroid values are preserved exactly.	
Traceability	Relevant User Req.(s)	UFR12
	Relevant System Req.(s)	SFR14
	Relevant Use Case.(s)	UC-003

Project Name	AIBI CV for Manufacturing
Test Suit	Automated Functional
Test Case ID	TC-SFR15-AUTO-01
What to Test	Verify decoder detects all QR codes with error rate ≤1%.

Test Data Input	250x250 image with 5 QR codes.	
Expected Result	All 5 detected. Error rate = 0%.	
Traceability	Relevant User Req.(s)	UFR1
	Relevant System Req.(s)	SFR15
	Relevant Use Case.(s)	UC-001

Project Name	**AIBI CV for Manufacturing**	
Test Suit	Automated Functional	
Test Case ID	TC-SFR16-AUTO-01	
What to Test	Verify output builder produces exactly one entry per detected QR code with no duplicates.	
Test Data Input	Raw detections with 1 duplicate (3 items, 1 dup).	
Expected Result	2 unique entries, no duplicates.	
Traceability	Relevant User Req.(s)	UFR1
	Relevant System Req.(s)	SFR16
	Relevant Use Case.(s)	UC-001

Project Name	**AIBI CV for Manufacturing**	
Test Suit	Automated Functional	
Test Case ID	TC-SFR17-AUTO-01	
What to Test	Verify simulation_scanner generates synthetic QR codes and processes them through the decode/parse pipeline.	
Test Data Input	Simulation scenarios with known expected field-value pairs.	
Expected Result	All simulation scenarios produce correct detection and parsing results.	
Traceability	Relevant User Req.(s)	UFR11
	Relevant System Req.(s)	SFR17
	Relevant Use Case.(s)	UC-005

Project Name	AIBI CV for Manufacturing	
Test Suit	Automated Non-Functional	
Test Case ID	TC-PPSR2-AUTO-01	
What to Test	Verify QR detection latency stays below 200 ms.	
Test Data Input	640x480 test image.	
Expected Result	Latency <200 ms.	
Traceability	Relevant User Req.(s)	UFR1
	Relevant System Req.(s)	PPSR2
	Relevant Use Case.(s)	UC-001

Project Name	AIBI CV for Manufacturing	
Test Suit	Automated Non-Functional	
Test Case ID	TC-PPSR5-AUTO-01	
What to Test	Verify QR detection accuracy ≥95% at 720p.	
Test Data Input	100 test cases at 720x1280, mocked 95% success.	
Expected Result	≥95%.	
Traceability	Relevant User Req.(s)	UFR3
	Relevant System Req.(s)	PPSR5
	Relevant Use Case.(s)	UC-001

Project Name	AIBI CV for Manufacturing	
Test Suit	Automated Non-Functional	
Test Case ID	TC-EISR3-AUTO-01	
What to Test	Verify JSON outputs from scan events contain required fields with correct types.	
Test Data Input	JSON outputs from multiple scan events. Required fields: workstation_id (str), timestamp (str), barcodes (list of {name: str, value: str}).	
Expected Result	All outputs contain required fields with correct types.	
Traceability	Relevant User Req.(s)	EIUR2

	Relevant System Req.(s)	EISR3
	Relevant Use Case.(s)	UC-001, UC-002

Project Name	**AIBI CV for Manufacturing**	
Test Suit	Automated Non-Functional	
Test Case ID	TC-EISR6-AUTO-01	
What to Test	Automated export to MES/ERP.	
Test Data Input	Config 1: part_number, serial_number. Config 2: part_number, batch_id, inspector_id.	
Expected Result	Different configs produce different required field sets and completion states.	
Traceability	Relevant User Req.(s)	EIUR3
	Relevant System Req.(s)	EISR6
	Relevant Use Case.(s)	UC-005

Appendix TE

Project Name	AIBI CV for Manufacturing	
Test Case ID	TC-SFR1-AUTO-01	
Testing Tools Used	pytest, unittest.mock, numpy, cv2.QRCodeDetector	
Testing Type	Automated Functional	
Execution Steps:	1	Mock cv2.QRCodeDetector to return 3 known QR codes
Execution Steps:	2	Create a 200x200 numpy test image.
Execution Steps:	3	Call AdvancedScanner.decode_qr() with the test image.
Execution Steps:	4	Assert returned count equals 3.
Execution Steps:	5	Compare each decoded payload against the ground truth values.
Execution Steps:	6	Verify no extra or missing entries in the result.

Test Execution Records:

#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Nathan	2025-11-11	decode_qr() returned 2 of 3 codes; third QR code payload truncated due to incorrect buffer handling in mock setup	FAIL	Mock did not simulate multi-code return correctly	2025-11-24 by Nathan
2	Nathan	2025-11-24	All 3 codes returned with correct payloads after fixing mock to use detectAndDecodeMulti	PASS	-	-
3	Jacob	2025-11-30	All 3 codes returned, payloads match, no extras or missing entries	PASS	-	-

Execution Summary	Total executions: 3. Pass rate: 67%. Initial mock configuration did not properly simulate multi-code detection; corrected mock to use detectAndDecodeMulti interface and all subsequent runs passed.

Project Name	AIBI CV for Manufacturing	
Test Case ID	TC-SFR5-AUTO-01	
Testing Tools Used	pytest, unittest.mock	
Testing Type	Automated Functional	
Execution Steps:	1	Create a payload dictionary with part_number="PN-12345" and serial_number="SN-67890".

Execution Steps:	2	Set field_order to [part_number, serial_number] and delimiter to TAB.
Execution Steps:	3	Execute the keystroke sequence builder function.
Execution Steps:	4	Capture the output sequence list.
Execution Steps:	5	Compare against expected sequence: ["PN-12345", "TAB", "SN-67890", "TAB", "ENTER"].

Test Execution Records:

#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Jacob	2025-11-12	Sequence output was ["PN-12345", "SN-67890", "ENTER"] — TAB delimiters missing between fields	FAIL	Builder did not insert delimiter tokens between field values	2025-11-24 by Jacob
2	Jacob	2025-11-24	Sequence matched expected: ["PN-12345", "TAB", "SN-67890", "TAB", "ENTER"]	PASS	-	-
3	Collin	2025-12-01	Sequence matched expected output exactly	PASS	-	-

Execution Summary	Total executions: 3. Pass rate: 67%. Initial implementation omitted delimiter insertion between fields; after adding TAB token injection logic all executions passed.

Project Name	**AIBI CV for Manufacturing**	
Test Case ID	TC-SFR11-AUTO-01	
Testing Tools Used	pytest, unittest.mock, json	
Testing Type	Automated Functional	
Execution Steps:	1	Create scan data with workstation_id="workstation_01" and barcodes=[{"name": "part_number", "value": "PN-001"}].

Execution Steps:	2	Write the scan data to a JSON file via OutputData.to_json().
Execution Steps:	3	Read back the saved JSON file using json.load().
Execution Steps:	4	Compare the loaded workstation_id against the original value.
Execution Steps:	5	Compare the loaded barcodes list against the original barcodes.
Execution Steps:	6	Assert exact equality on all fields.

Test Execution Records:

#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Collin	2025-11-13	JSON file written but read-back failed with KeyError on "barcodes" — field was serialized as "barcode_data" instead	FAIL	OutputData.to_json() used internal field name "barcode_data" instead of "barcodes"	2025-11-25 by Collin
2	Collin	2025-11-25	Write/read cycle completed; workstation_id and barcodes matched exactly	PASS	-	-
3	Nathan	2025-12-01	All fields matched after round-trip write/read	PASS	-	-

Execution Summary	Total executions: 3. Pass rate: 67%. Field naming inconsistency between internal model and serialized output was fixed; all subsequent integrity checks passed.

Project Name	**AIBI CV for Manufacturing**	
Test Case ID	TC-SFR14-AUTO-01	
Testing Tools Used	pytest, pickle, numpy	
Testing Type	Automated Functional	
Execution Steps:	1	Create a trained process object with 2 steps ("Install Part A", "Tighten Bolts"), each with computed centroid numpy arrays.

Execution Steps:	2	Call serialize_process() to produce a pickle byte stream.
Execution Steps:	3	Call deserialize_process() on the byte stream to reconstruct the process.
Execution Steps:	4	Compare step names and ordering between original and deserialized process.
Execution Steps:	5	Compare centroid arrays using np.allclose() to verify numerical equality.

Test Execution Records:

#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Daniel	2025-11-14	Deserialization raised UnpicklingError — serialize_process() was writing raw dict instead of using pickle.dumps()	FAIL	serialize_process() wrote JSON-style dict, not pickle format	2025-11-24 by Daniel
2	Daniel	2025-11-24	Deserialization succeeded but np.allclose() failed — centroid arrays were converted to lists during serialization, losing dtype	FAIL	Numpy arrays were not preserved through pickle; intermediate list conversion introduced float precision loss	2025-11-25 by Daniel
3	Daniel	2025-11-28	All step names match, order preserved, np.allclose() returns True for all centroid arrays	PASS	-	-
4	Nathan	2025-12-02	Full round-trip integrity confirmed; step names, order, and centroid arrays all match	PASS	-	-

Execution Summary	Total executions: 4. Pass rate: 50%. Two iterations required: first to fix serialization format from dict to pickle, second to preserve numpy array dtypes through the round-trip. Final executions confirmed full data integrity.

Project Name	**AIBI CV for Manufacturing**
Test Case ID	TC-SFR15-AUTO-01
Testing Tools Used	pytest, unittest.mock, numpy
Testing Type	Automated Functional

Execution Steps:	1	Mock 5 ground truth QR codes with known payloads on a 250x250 image.
Execution Steps:	2	Execute AdvancedScanner.decode_qr() on the mocked image.
Execution Steps:	3	Count the number of detected codes.
Execution Steps:	4	Compare each detected payload against the ground truth set.
Execution Steps:	5	Calculate the error rate (missed + incorrect / total).
Execution Steps:	6	Assert error rate is less than or equal to 1%.

Test Execution Records:

#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Nathan	2025-11-12	Only 4 of 5 codes detected; fifth code at image edge was missed; error rate = 20%	FAIL	decode_qr() did not handle QR codes near image boundaries correctly	2025-11-25 by Nathan
2	Nathan	2025-11-25	All 5 codes detected, payloads match, error rate = 0%	PASS	-	-
3	Jacob	2025-12-01	5 of 5 detected, 0% error rate	PASS	-	-

Execution Summary	Total executions: 3. Pass rate: 67%. Boundary handling issue in decode_qr() caused one code to be missed initially; after adjusting detection bounds all codes detected with 0% error rate.

Project Name	**AIBI CV for Manufacturing**	
Test Case ID	TC-SFR16-AUTO-01	
Testing Tools Used	pytest, numpy, set-based dedup	
Testing Type	Automated Functional	
Execution Steps:	1	Create a raw detections list with 3 items, where 1 is a duplicate (same payload as another).
Execution Steps:	2	Process the raw detections through the deduplication logic.
Execution Steps:	3	Count the number of unique entries in the result.
Execution Steps:	4	Assert exactly 2 unique entries remain.

Execution Steps:	5	Verify the duplicate entry was removed, not the wrong original.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Collin	2025-11-13	Dedup returned 3 entries — duplicate was not removed because comparison used object identity instead of value equality	FAIL	Deduplication compared object references, not payload values	2025-11-24 by Collin
2	Collin	2025-11-24	Dedup returned 2 unique entries; duplicate correctly removed	PASS	-	-
3	Daniel	2025-11-30	2 unique entries returned, correct items preserved	PASS	-	-
Execution Summary	Total executions: 3. Pass rate: 67%. Deduplication logic was comparing object identity rather than payload values; switching to value-based comparison fixed the issue.					

Project Name	**AIBI CV for Manufacturing**	
Test Case ID	TC-SFR17-AUTO-01	
Testing Tools Used	pytest, simulation_scanner.py	
Testing Type	Automated Functional	
Execution Steps:	1	Import and invoke simulation_scanner scenarios programmatically.
Execution Steps:	2	Run each simulation scenario to generate synthetic QR codes.
Execution Steps:	3	Capture the detection results from each scenario.
Execution Steps:	4	Verify that all expected QR codes were generated and detected.
Execution Steps:	5	Parse field-value pairs from detected codes and compare against expected data.

Execution Steps:	6	Assert no exceptions were raised during any simulation run.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Jacob	2025-11-14	Simulation ran but synthetic QR generation produced blank images — qrcode library not generating valid image matrices	FAIL	simulation_scanner.py used incorrect image format for synthetic QR rendering	2025-11-25 by Jacob
2	Jacob	2025-11-25	All scenarios produced valid synthetic QR codes; field-value pairs matched expected data	PASS	-	-
3	Nathan	2025-12-02	All simulation scenarios completed successfully with correct parsed output	PASS	-	-
Execution Summary	Total executions: 3. Pass rate: 67%. Synthetic QR image generation was producing blank frames due to incorrect image format conversion; after fixing the rendering pipeline all simulation scenarios passed.					

Project Name	**AIBI CV for Manufacturing**	
Test Case ID	TC-PPSR2-AUTO-01	
Testing Tools Used	pytest, time.perf_counter, numpy	
Testing Type	Automated Non-Functional	
Execution Steps:	1	Create a 640x480 numpy test image with embedded QR code data.
Execution Steps:	2	Record start time using time.perf_counter().

Execution Steps:	3	Execute AdvancedScanner.decode_qr() on the test image.
Execution Steps:	4	Record end time using time.perf_counter().
Execution Steps:	5	Calculate elapsed time in milliseconds.
Execution Steps:	6	Assert elapsed time is less than 200 ms.

Test Execution Records:

#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Daniel	2025-11-11	Latency measured at 342 ms — exceeded 200 ms threshold due to redundant image preprocessing step	FAIL	decode_qr() performed unnecessary color space conversion on every call	2025-11-24 by Daniel
2	Daniel	2025-11-24	Latency measured at 87 ms after removing redundant preprocessing	PASS	-	-
3	Collin	2025-11-30	Latency measured at 91 ms	PASS	-	-

Execution Summary	Total executions: 3. Pass rate: 67%. Initial latency exceeded threshold due to redundant image preprocessing; removing unnecessary color conversion brought latency well under 200 ms.

Project Name	**AIBI CV for Manufacturing**	
Test Case ID	TC-PPSR5-AUTO-01	
Testing Tools Used	pytest, unittest.mock, numpy	
Testing Type	Automated Non-Functional	
Execution Steps:	1	Create 100 test cases with 720x1280 resolution images.
Execution Steps:	2	Mock detectAndDecodeMulti to simulate a 95% success rate across test cases.
Execution Steps:	3	Execute decode_qr() on all 100 test images.
Execution Steps:	4	Count successful detections.
Execution Steps:	5	Calculate overall accuracy percentage.
Execution Steps:	6	Assert accuracy is greater than or equal to 95%.

Test Execution Records:

#	Tester	Test Date	Actual Result	Status	Defect	Correction

1	Nathan	2025-11-13	Accuracy measured at 88% — mock was returning incorrect dimensions causing 12 false negatives	FAIL	Test harness generated images at wrong resolution (640x480 instead of 720x1280)	2025-11-25 by Nathan
2	Nathan	2025-11-25	Accuracy measured at 96% after correcting image dimensions in test harness	PASS	-	-
3	Jacob	2025-12-01	Accuracy measured at 95%, meets threshold	PASS	-	-
Execution Summary	Total executions: 3. Pass rate: 67%. Test harness was generating images at incorrect resolution, producing misleading accuracy numbers; correcting image dimensions yielded consistent 95%+ accuracy.					

Project Name	**AIBI CV for Manufacturing**					
Test Case ID	TC-EISR3-AUTO-01					
Testing Tools Used	pytest, json validation					
Testing Type	Automated Non-Functional					
Execution Steps:	1	Generate JSON outputs from multiple scan events.				
Execution Steps:	2	Load each JSON file and validate that the "workstation_id" field exists and is a string.				
Execution Steps:	3	Validate that the "timestamp" field exists and is a string.				
Execution Steps:	4	Validate that the "barcodes" field exists and is a list.				
Execution Steps:	5	For each barcode object, verify it contains "name" (string) and "value" (string) fields.				
Execution Steps:	6	Assert 100% compliance across all output files.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction

1	Collin	2025-11-12	2 of 5 outputs failed — "timestamp" field was an integer (Unix epoch) instead of ISO 8601 string	FAIL	OutputData serialized timestamp as int instead of formatted string	2025-11-24 by Collin
2	Collin	2025-11-24	All 5 outputs passed structure validation; all fields present with correct types	PASS	-	-
3	Daniel	2025-12-02	100% compliance across all generated outputs	PASS	-	-
Execution Summary	Total executions: 3. Pass rate: 67%. Timestamp was initially serialized as Unix epoch integer; converting to ISO 8601 string format brought all outputs into compliance.					

Project Name	**AIBI CV for Manufacturing**	
Test Case ID	TC-EISR6-AUTO-01	
Testing Tools Used	pytest, ConfigManager, WorkstationConfig	
Testing Type	Automated Non-Functional	
Execution Steps:	1	Create Config 1 with required fields: part_number, serial_number.
Execution Steps:	2	Create Config 2 with required fields: part_number, batch_id, inspector_id.
Execution Steps:	3	Process an identical scan sequence through Config 1 and record required fields and completion state.
Execution Steps:	4	Process the same scan sequence through Config 2 and record required fields and completion state.
Execution Steps:	5	Assert that the two configs produce different required field sets.
Execution Steps:	6	Verify completion states differ when scan data satisfies one config but not the other.
Test Execution Records:		

#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Jacob	2025-11-14	Both configs returned identical completion states — ConfigManager was hardcoded to always use default field list	FAIL	ConfigManager ignored the loaded config and used hardcoded defaults	2025-11-25 by Jacob
2	Jacob	2025-11-25	Config 1 and Config 2 produced different required field sets; completion states differed correctly	PASS	-	-
3	Nathan	2025-12-03	Distinct behavior confirmed for both configs	PASS	-	-
Execution Summary	Total executions: 3. Pass rate: 67%. ConfigManager was ignoring loaded configuration files and using hardcoded defaults; after fixing the loader to respect config-driven fields, behavior diverged correctly per config.					

Project Name	**AIBI CV for Manufacturing**	
Test Case ID	TC-SFR1-MAN-01	
Testing Tools Used	Live 720p USB camera, printed QR codes, AIBI CV scanner UI	
Testing Type	Manual Functional	
Execution Steps:	1	Launch the scanner application with a 720p USB camera connected.
Execution Steps:	2	Position the camera so all three printed QR codes are visible in the field of view.
Execution Steps:	3	Observe the UI for real-time detection overlays on each QR code.
Execution Steps:	4	Record how many codes are detected and verify decoded payloads.
Execution Steps:	5	Repeat with codes positioned at the edges of the field of view.
Test Execution Records:		

#	Tester	Test Date	Actual Result	Status	Defect	Correction

1	Nathan	2025-11-11	Only 2 of 3 QR codes detected — third code at bottom-right corner of FOV was consistently missed	FAIL	Camera FOV too narrow at default resolution; edge detection threshold too strict	2025-11-24 by Nathan
2	Nathan	2025-11-24	All 3 codes detected after adjusting detection sweep to cover full frame area	PASS	-	-
3	Daniel	2025-11-30	All 3 codes detected correctly including edge positions; payloads match expected values	PASS	-	-
Execution Summary	Total executions: 3. Pass rate: 67%. Edge-of-frame QR codes were initially missed due to overly strict detection boundaries; expanding the sweep area resolved the issue.					

Project Name	**AIBI CV for Manufacturing**					
Test Case ID	TC-SFR2-MAN-01					
Testing Tools Used	Configuration files, printed test QR codes, JSON editor					
Testing Type	Manual Functional					
Execution Steps:	1	Open the workstation JSON configuration file.				
Execution Steps:	2	Define barcode_fields mappings for part_number and serial_number.				
Execution Steps:	3	Save the configuration file.				
Execution Steps:	4	Scan the prepared QR codes containing part_number:PN-12345 and serial_number:SN-67890.				
Execution Steps:	5	Inspect the output JSON to verify field-value assignments match the configured mappings.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction

1	Jacob	2025-11-12	Output JSON had "part_number": "SN-67890" and "serial_number": "PN-12345" — fields swapped	FAIL	Field mapping applied in alphabetical order instead of config-defined order	2025-11-24 by Jacob
2	Jacob	2025-11-24	Field mappings correct: "part_number": "PN-12345", "serial_number": "SN-67890"	PASS	-	-
3	Collin	2025-12-01	Mappings verified correct for both fields	PASS	-	-
Execution Summary	Total executions: 3. Pass rate: 67%. Field mapping was applying values in alphabetical key order rather than config-specified order; fixing the mapping logic to respect config ordering resolved the swap.					

Project Name	**AIBI CV for Manufacturing**	
Test Case ID	TC-SFR4-MAN-01	
Testing Tools Used	Live scanner, file system monitor	
Testing Type	Manual Functional	
Execution Steps:	1	Trigger a scan event by presenting QR codes to the camera.
Execution Steps:	2	Wait for the system to process the scan.
Execution Steps:	3	Navigate to the output directory and locate the generated JSON file.
Execution Steps:	4	Open the JSON file and verify it contains workstation_id, timestamp, and barcodes array.
Execution Steps:	5	Validate the timestamp is in ISO 8601 format.
Execution Steps:	6	Confirm the barcodes array contains correct name-value pairs.

Test Execution Records:

#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Collin	2025-11-11	JSON file created but timestamp was in Unix epoch format (integer), not ISO 8601 string	FAIL	Timestamp serialization used time.time() integer instead of datetime.isoformat()	2025-11-24 by Collin

2	Collin	2025-11-24	JSON file generated with correct ISO 8601 timestamp, valid structure, and matching barcodes	PASS	-	-
3	Nathan	2025-11-29	JSON output verified: workstation_id, ISO timestamp, and barcodes all correct	PASS	-	-
Execution Summary	Total executions: 3. Pass rate: 67%. Timestamp format was Unix epoch integer instead of ISO 8601; switching serialization to datetime.isoformat() resolved the issue.					

Project Name	**AIBI CV for Manufacturing**					
Test Case ID	TC-SFR5-MAN-01					
Testing Tools Used	Live scanner, Microsoft Excel, keyboard monitoring					
Testing Type	Manual Functional					
Execution Steps:	1	Open Microsoft Excel and click into cell A1.				
Execution Steps:	2	Configure the scanner application for keyboard-emulation mode with TAB delimiter.				
Execution Steps:	3	Present QR codes (part_number:PN-12345, serial_number:SN-67890) and trigger the scan.				
Execution Steps:	4	Observe the keystrokes typed into Excel.				
Execution Steps:	5	Verify the output sequence: PN-12345 in A1, SN-67890 in B1 (TAB moved to next cell), then ENTER moved to next row.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Daniel	2025-11-12	PN-12345 typed correctly but TAB keystroke was not sent — both values appeared in cell A1 concatenated	FAIL	Keyboard emulation did not send TAB virtual key code between fields	2025-11-25 by Daniel
2	Daniel	2025-11-25	PN-12345 in A1, TAB moved	PASS	-	-

			cursor, SN-67890 in B1, ENTER advanced to next row			
3	Jacob	2025-12-01	Keystroke sequence matched expected pattern exactly in Excel	PASS	-	-
Execution Summary	Total executions: 3. Pass rate: 67%. TAB virtual key was not being sent between field values; implementing proper VK_TAB keystroke injection resolved the issue.					

Project Name	**AIBI CV for Manufacturing**					
Test Case ID	TC-SFR11-MAN-01					
Testing Tools Used	Live scanner, output directory, file explorer					
Testing Type	Manual Functional					
Execution Steps:	1	Perform five scan events with distinct QR code payloads.				
Execution Steps:	2	Navigate to the output directory after all scans are complete.				
Execution Steps:	3	Count the number of JSON files created (should be 5).				
Execution Steps:	4	Open each file and verify contents match the scanned payloads.				
Execution Steps:	5	Check that timestamps across the five files are sequential and unique.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Nathan	2025-11-13	Only 3 JSON files found after 5 scan events — files 2 and 4 were overwritten because filenames used second-level timestamp granularity and two scans happened	FAIL	File naming used timestamp with 1-second granularity, causing overwrites for rapid scans	2025-11-25 by Nathan

			within the same second			
2	Nathan	2025-11-25	All 5 JSON files present after switching to millisecond-precision filenames; contents match payloads	PASS	-	-
3	Collin	2025-12-02	5 files present, all payloads correct, timestamps sequential and unique	PASS	-	-
Execution Summary	Total executions: 3. Pass rate: 67%. Filename timestamp granularity caused overwrites during rapid scanning; switching to millisecond precision eliminated collisions.					

Project Name	**AIBI CV for Manufacturing**					
Test Case ID	TC-SFR12-MAN-01					
Testing Tools Used	JSON output files, manual inspection, json.loads()					
Testing Type	Manual Functional					
Execution Steps:	1	Open three JSON output files generated from different scan events.				
Execution Steps:	2	Verify each file contains a workstation_id field of type string.				
Execution Steps:	3	Verify each file contains a timestamp field as an ISO 8601 formatted string.				
Execution Steps:	4	Verify each file contains a barcodes field that is an array of objects.				
Execution Steps:	5	Verify each barcode object has "name" (string) and "value" (string) fields.				
Execution Steps:	6	Run json.loads() on each file to confirm valid JSON syntax.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Jacob	2025-11-14	File 2 had barcodes as a flat list of strings instead of	FAIL	Certain scan paths serialized barcodes	2025-11-25 by Jacob

			array of {name, value} objects		as string list instead of object array	
2	Jacob	2025-11-25	All 3 files have correct structure: workstation_id (str), timestamp (ISO 8601 str), barcodes (array of {name, value} objects)	PASS	-	-
3	Daniel	2025-12-01	All 3 files validated via json.loads() and manual inspection; 100% compliant	PASS	-	-
Execution Summary	Total executions: 3. Pass rate: 67%. One serialization path produced flat string arrays for barcodes instead of structured objects; unifying the serialization format fixed the inconsistency.					

Project Name	**AIBI CV for Manufacturing**					
Test Case ID	TC-SFR15-MAN-01					
Testing Tools Used	Multi-code test sheet, live 720p camera, factory lighting (~200-500 lux)					
Testing Type	Manual Functional					
Execution Steps:	1	Position the 5-QR-code test sheet at approximately 40 cm from the camera.				
Execution Steps:	2	Run detection under direct overhead factory lighting (~300 lux).				
Execution Steps:	3	Record which codes are detected and their decoded payloads.				
Execution Steps:	4	Tilt the test sheet to approximately 30 degrees and repeat detection.				
Execution Steps:	5	Move the test sheet to the edge of the field of view and repeat.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Collin	2025-11-11	4 of 5 codes detected under overhead lighting; code_3 at	FAIL	No glare compensation in image preprocessing pipeline	2025-11-24 by Nathan

			center was washed out by glare at ~450 lux			
2	Nathan	2025-11-24	All 5 codes detected after adding adaptive histogram equalization; 30-degree tilt also passed	PASS	-	-
3	Daniel	2025-11-29	All 5 codes decoded correctly across all lighting conditions and angles	PASS	-	-
Execution Summary	Total executions: 3. Pass rate: 67%. Glare from overhead factory lighting washed out one QR code; adding adaptive histogram equalization to the preprocessing pipeline resolved the issue.					

Project Name	**AIBI CV for Manufacturing**					
Test Case ID	TC-SFR16-MAN-01					
Testing Tools Used	Live scanner, 3 distinct printed QR codes					
Testing Type	Manual Functional					
Execution Steps:	1	Arrange all three distinct QR codes (part_number:PN-001, serial_number:SN-002, batch_id:BATCH-003) in the camera's field of view.				
Execution Steps:	2	Trigger a scan event.				
Execution Steps:	3	Inspect the output JSON barcodes array.				
Execution Steps:	4	Count the number of entries in the barcodes array.				
Execution Steps:	5	Confirm no duplicate entries exist and each visible code has exactly one entry.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Daniel	2025-11-13	Barcodes array contained 4 entries — PN-001 appeared twice due to multi-frame accumulation	FAIL	Scan accumulated detections across multiple frames without deduplication	2025-11-24 by Daniel

			without dedup			
2	Daniel	2025-11-24	Barcodes array contained exactly 3 entries, no duplicates, all payloads correct	PASS	-	-
3	Jacob	2025-12-01	3 entries, no duplicates, all correct	PASS	-	-
Execution Summary	Total executions: 3. Pass rate: 67%. Multi-frame accumulation was including duplicate detections; adding deduplication before output generation resolved the duplicate entries.					

Project Name	**AIBI CV for Manufacturing**					
Test Case ID	TC-SFR6-MAN-01					
Testing Tools Used	Streamlit app, trained 3-step process, USB camera					
Testing Type	Manual Functional					
Execution Steps:	1	Load the trained 3-step process ("Step A", "Step B", "Step C") in Operation mode.				
Execution Steps:	2	Perform Step A in front of the camera and verify the system transitions to CORRECT_STEP then CONFIRMED.				
Execution Steps:	3	Perform Step B and verify it is also confirmed in correct order.				
Execution Steps:	4	Skip Step C and perform an unrelated action — verify the system flags WRONG_ORDER or SKIPPED.				
Execution Steps:	5	Expand the run log and check for violation entries.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Nathan	2025-11-15	Steps A and B confirmed correctly, but skipping to an unrelated action did not trigger any violation flag — system remained in IDLE state instead of detecting a skip	FAIL	Sequence validator did not track expected-vs-actual step progression; skip detection was not implemented	2025-11-25 by Nathan

2	Nathan	2025-11-25	Steps A and B confirmed; skipping C triggered WRONG_ORDER flag but no log entry was written to run history	FAIL	Violation detection worked but logging to run history was not wired up	2025-11-28 by Nathan
3	Nathan	2025-11-28	All steps confirmed correctly in order; skip detected, flagged, and logged in run history with step name and violation type	PASS	-	-
4	Jacob	2025-12-02	Correct-order steps confirmed, out-of-order detected and logged properly	PASS	-	-
Execution Summary	Total executions: 4. Pass rate: 50%. Required two fix cycles: first to implement skip/out-of-order detection logic, then to wire violation logging into run history. Final executions confirmed full sequence validation functionality.					

Project Name	**AIBI CV for Manufacturing**					
Test Case ID	TC-SFR7-MAN-01					
Testing Tools Used	Streamlit app, trained process, USB camera					
Testing Type	Manual Functional					
Execution Steps:	1	Start Operation mode with a loaded trained process containing 3 or more steps.				
Execution Steps:	2	Observe the live video feed displayed on the Streamlit interface.				
Execution Steps:	3	Verify the current step name is displayed as a text overlay or UI element.				
Execution Steps:	4	Verify the similarity confidence percentage is shown and updates in real time.				
Execution Steps:	5	Complete a step and verify the progress bar updates to reflect completion.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Jacob	2025-11-15	Live video feed displayed but no step name or	FAIL	Streamlit st.image() call was overwriting the overlay	2025-11-25 by Jacob

			confidence score shown — overlay elements were not rendering		div; overlay placement logic missing	
2	Jacob	2025-11-25	Step name and confidence score now visible; progress bar present but did not update when step completed	FAIL	Progress bar was reading initial step count but not subscribing to state updates	2025-11-28 by Collin
3	Collin	2025-11-28	All overlay elements visible: step name, confidence percentage, and progress bar all updating in real time	PASS	-	-
4	Nathan	2025-12-02	Full overlay functionality confirmed across multiple steps	PASS	-	-
Execution Summary	Total executions: 4. Pass rate: 50%. Two iterations needed: first to add overlay rendering alongside the video feed, second to make the progress bar reactive to state changes. All elements functional in final runs.					

Project Name	**AIBI CV for Manufacturing**	
Test Case ID	TC-SFR8-MAN-01	
Testing Tools Used	Streamlit app, USB camera	
Testing Type	Manual Functional	
Execution Steps:	1	Select Training mode from the Streamlit app mode selector.
Execution Steps:	2	Add three step names using the UI: "Install Part A", "Tighten Bolts", "Inspect Assembly".

Execution Steps:	3	For each step, click "Start Recording" and capture reference frames from the camera.
Execution Steps:	4	Click "Stop" to end recording for each step.
Execution Steps:	5	Click "Finalize Training" to compute DINOv2 embeddings for all recorded frames.
Execution Steps:	6	Click "Save Process" and download the resulting .pkl file.
Execution Steps:	7	Verify the downloaded .pkl file is non-empty.

Test Execution Records:

#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Collin	2025-11-16	Step names added and frames recorded, but "Finalize Training" raised RuntimeError — DINOv2 model not loaded at training time	FAIL	DINOv2 model was only initialized in Operation mode, not in Training mode	2025-11-25 by Daniel
2	Daniel	2025-11-25	Training finalized and .pkl file downloaded, but file size was 0 bytes	FAIL	serialize_process() returned empty bytes when called before session state was fully committed	2025-11-28 by Daniel
3	Daniel	2025-11-28	Full training workflow completed: 3 steps defined, frames recorded, embeddings computed, .pkl file downloaded (42 KB)	PASS	-	-
4	Collin	2025-12-02	Training workflow verified end-to-end; .pkl file non-empty and downloadable	PASS	-	-

Execution Summary	Total executions: 4. Pass rate: 50%. Two defects addressed: DINOv2 model not being available in Training mode, and serialization producing empty files. Both fixed by ensuring model pre-loading and proper session state management.

Project Name	**AIBI CV for Manufacturing**	
Test Case ID	TC-SFR9-MAN-01	
Testing Tools Used	Streamlit app, trained process	
Testing Type	Manual Functional	
Execution Steps:	1	Note the default similarity threshold value (0.6) displayed on the slider.
Execution Steps:	2	Perform a step at default threshold and observe confirmation behavior.
Execution Steps:	3	Increase the threshold to 0.9 (stricter) and repeat the same step — observe that it becomes harder to confirm.
Execution Steps:	4	Decrease the threshold to 0.3 (looser) and repeat the same step — observe that it confirms more easily.
Execution Steps:	5	Verify that each threshold change produces a measurably different detection behavior.

Test Execution Records:

#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Daniel	2025-11-16	Slider moved but behavior did not change — threshold value was read once at startup and not re-read per frame	FAIL	Threshold was cached at initialization instead of being read dynamically from slider state	2025-11-25 by Daniel
2	Daniel	2025-11-25	Threshold changes now affect detection: 0.9 required near-exact match, 0.3 accepted loose match	PASS	-	-
3	Jacob	2025-12-01	Behavior differences confirmed	PASS	-	-

			at all three threshold levels			
Execution Summary	Total executions: 3. Pass rate: 67%. Threshold was being cached at startup rather than read dynamically from the Streamlit slider; making the read per-frame resolved the issue.					

Project Name	**AIBI CV for Manufacturing**					
Test Case ID	TC-SFR10-MAN-01					
Testing Tools Used	Streamlit app, trained 3-step process					
Testing Type	Manual Functional					
Execution Steps:	1	Load a trained 3-step process in Operation mode.				
Execution Steps:	2	Complete step 1 correctly and wait for confirmation.				
Execution Steps:	3	Skip step 2 and perform step 3 instead.				
Execution Steps:	4	Expand the run history section in the Streamlit UI.				
Execution Steps:	5	Verify the violation is logged with the step name and violation type (SKIPPED or WRONG_ORDER).				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Nathan	2025-11-16	Step 1 completed, step 2 skipped, step 3 performed — but run history section showed no violation entry; only completed steps were logged	FAIL	Logging only recorded CONFIRMED transitions, not violation events	2025-11-25 by Nathan
2	Nathan	2025-11-25	Violation logged in run history with step name "Step B" and type "SKIPPED"	PASS	-	-
3	Collin	2025-12-01	Violation correctly logged; step name	PASS	-	-

			and type both accurate			
Execution Summary	Total executions: 3. Pass rate: 67%. Logging was initially limited to successful step confirmations and did not capture violations; extending the logger to record SKIPPED and WRONG_ORDER events resolved the issue.					

Project Name	**AIBI CV for Manufacturing**					
Test Case ID	TC-SFR14-MAN-01					
Testing Tools Used	Streamlit app, trained process, file uploader					
Testing Type	Manual Functional					
Execution Steps:	1	Complete a training session with at least 2 steps and finalize the process.				
Execution Steps:	2	Click "Save Process" to download the .pkl file.				
Execution Steps:	3	Close and restart the Streamlit application.				
Execution Steps:	4	Use the file uploader widget to load the saved .pkl file.				
Execution Steps:	5	Verify all step names from the original training are present in the loaded process.				
Execution Steps:	6	Switch to Operation mode and verify step detection works with the loaded embeddings.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Jacob	2025-11-17	.pkl file loaded but step names displayed as empty strings — step name field was not included in serialization	FAIL	serialize_process() omitted step_name attribute from the serialized dictionary	2025-11-25 by Jacob
2	Jacob	2025-11-25	Step names present after load; Operation mode started but detection threw ValueError on centroid	FAIL	Centroid arrays were flattened during serialization, losing their 2D shape	2025-11-28 by Daniel

			shape mismatch			
3	Daniel	2025-11-28	Full save/load cycle successful: step names present, detection functional in Operation mode	PASS	-	-
4	Collin	2025-12-03	Save, restart, load, and detection all verified working end-to-end	PASS	-	-
Execution Summary	Total executions: 4. Pass rate: 50%. Two defects required fixing: step names omitted from serialization, and centroid array shape lost during pickling. Both resolved for full round-trip integrity.					

Project Name	**AIBI CV for Manufacturing**					
Test Case ID	TC-SFR19-MAN-01					
Testing Tools Used	Streamlit app, trained process					
Testing Type	Manual Functional					
Execution Steps:	1	Load a trained process in Operation mode.				
Execution Steps:	2	Perform step 1 correctly and wait for confirmation.				
Execution Steps:	3	Deliberately skip step 2 and perform step 3.				
Execution Steps:	4	Observe the Streamlit UI for any error or warning messages.				
Execution Steps:	5	Verify the displayed message identifies the violation type and the affected step name.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Collin	2025-11-16	Violation detected internally (confirmed via logs) but no visual error message appeared	FAIL	Violation state was logged to console but st.warning()/st.error() calls were not implemented	2025-11-25 by Collin

			in the Streamlit UI			
2	Collin	2025-11-25	st.warning() message displayed identifying violation type as "SKIPPED" and step name "Step B"	PASS	-	-
3	Nathan	2025-12-01	Error message displayed clearly with correct violation type and step name	PASS	-	-
Execution Summary	Total executions: 3. Pass rate: 67%. Violations were detected and logged to console but not surfaced to the UI; adding st.warning() calls to render violations on screen resolved the issue.					

Project Name	**AIBI CV for Manufacturing**					
Test Case ID	TC-SFR20-MAN-01					
Testing Tools Used	Streamlit app, trained 4-step process					
Testing Type	Manual Functional					
Execution Steps:	1	Load a trained process with 4 steps in Operation mode.				
Execution Steps:	2	Start Operation mode.				
Execution Steps:	3	Verify the step checklist is visible on the Streamlit UI.				
Execution Steps:	4	Count the number of steps shown in the checklist.				
Execution Steps:	5	Verify each step name matches the names defined during training.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Daniel	2025-11-17	Step checklist visible but only showed 3 of 4 steps — last step was truncated due to	FAIL	Loop iterated steps[0:n-1] instead of steps[0:n], cutting off the last step	2025-11-25 by Daniel

			rendering logic using range(len-1)			
2	Daniel	2025-11-25	All 4 steps displayed in checklist with correct names matching the trained process	PASS	-	-
3	Jacob	2025-12-02	4 steps visible, names match, correct order	PASS	-	-
Execution Summary	Total executions: 3. Pass rate: 67%. Off-by-one error in the step list rendering loop truncated the last step; correcting the loop range to include all steps resolved the issue.					

Project Name	**AIBI CV for Manufacturing**	
Test Case ID	TC-SFR21-MAN-01	
Testing Tools Used	Streamlit app, trained 3-step process	
Testing Type	Manual Functional	
Execution Steps:	1	Load the trained process and start Operation mode.
Execution Steps:	2	Complete step 1 and wait for the system to confirm it.
Execution Steps:	3	Observe the step list — verify step 1 now shows a green checkmark (checkmark marker).
Execution Steps:	4	Complete step 2 and verify it also shows the completed marker.
Execution Steps:	5	Verify step 3 still shows the pending marker (empty box).

Test Execution Records:

#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Nathan	2025-11-17	Step 1 completed but marker did not change from pending to completed — UI did not refresh step list after state transition	FAIL	Step list was rendered once on load and not re-rendered on state changes	2025-11-28 by Nathan
2	Nathan	2025-11-28	Step 1 shows completed marker after completion; step 2 shows completed after completion; step 3 shows pending	PASS	-	-
3	Collin	2025-12-02	All markers update correctly as steps are completed	PASS	-	-

Execution Summary	Total executions: 3. Pass rate: 67%. Step list was not re-rendering on state changes; adding Streamlit rerun triggers after state transitions ensured markers update in real time.

Project Name	**AIBI CV for Manufacturing**	
Test Case ID	TC-SFR22-MAN-01	
Testing Tools Used	Streamlit app, trained 3-step process	
Testing Type	Manual Functional	
Execution Steps:	1	Load a trained 3-step process in Operation mode.
Execution Steps:	2	Complete step 1 and wait for confirmation.
Execution Steps:	3	Skip step 2 by performing step 3 instead.
Execution Steps:	4	Observe the step list and verify step 2 shows a distinct missed/skipped indicator (not a completed marker).
Execution Steps:	5	Verify the missed indicator is visually different from both completed and pending markers.

Test Execution Records:

#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Jacob	2025-11-17	Step 2 still showed pending marker after being skipped — no distinct skipped marker was implemented	FAIL	Only CONFIRMED and PENDING states had visual markers; SKIPPED state fell through to default pending marker	2025-11-28 by Jacob
2	Jacob	2025-11-28	Step 2 now shows a distinct red marker when skipped, clearly different from completed and pending	PASS	-	-
3	Daniel	2025-12-02	Skipped step marker is visually distinct and clearly	PASS	-	-

			identifies the missed step			
Execution Summary	Total executions: 3. Pass rate: 67%. No visual marker existed for the SKIPPED state; implementing a distinct red indicator for skipped steps resolved the issue.					

Project Name	**AIBI CV for Manufacturing**					
Test Case ID	TC-SFR26-MAN-01					
Testing Tools Used	Streamlit app, trained process, sliding window configuration					
Testing Type	Manual Functional					
Execution Steps:	1	Configure the sliding window size to 5 and required votes to 3 using the UI sliders.				
Execution Steps:	2	Start Operation mode with a loaded trained process.				
Execution Steps:	3	Begin performing step 1 in front of the camera.				
Execution Steps:	4	Observe the progress display as frames are matched against the step reference.				
Execution Steps:	5	Verify the vote count increments toward the required threshold (e.g., "2/3 votes").				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Collin	2025-11-17	Vote count displayed as "0/3" and never incremented even though frames were matching — vote accumulation logic was not connected to the display	FAIL	Voting counter was computed internally but the UI label was hardcoded to show "0/{required}"	2025-11-28 by Collin
2	Collin	2025-11-28	Vote count increments correctly: observed "1/3", "2/3", "3/3" before step confirmation	PASS	-	-
3	Nathan	2025-12-02	Progress indicator updates in real time showing	PASS	-	-

			accurate vote counts			
Execution Summary	Total executions: 3. Pass rate: 67%. UI label was hardcoded instead of reading from the live vote counter; binding the display to the actual voting state resolved the issue.					

Project Name	**AIBI CV for Manufacturing**					
Test Case ID	TC-SFR28-MAN-01					
Testing Tools Used	Streamlit app, trained 3-step process					
Testing Type	Manual Functional					
Execution Steps:	1	Complete 2 of 3 steps in a trained process during Operation mode.				
Execution Steps:	2	Click the "Restart Run" button.				
Execution Steps:	3	Verify all steps reset to pending state (empty box markers).				
Execution Steps:	4	Verify the current step indicator resets to step 1.				
Execution Steps:	5	Verify the previous run is logged in the run history section.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Daniel	2025-11-17	Clicking restart reset step markers to pending but current step remained at step 3 — step pointer was not reset	FAIL	Restart logic cleared step statuses but did not reset the current_step_index to 0	2025-11-28 by Daniel
2	Daniel	2025-11-28	All steps reset to pending, current step resets to step 1, but previous run was not saved to history	FAIL	Restart did not call the run history append function before clearing state	2025-11-30 by Daniel
3	Daniel	2025-11-30	Full reset: all steps pending, current step at 1,	PASS	-	-

			previous run logged in history with timestamps and results			
4	Nathan	2025-12-03	Restart verified: state fully reset, history preserved	PASS	-	-
Execution Summary	Total executions: 4. Pass rate: 50%. Two issues fixed: current step index not resetting on restart, and previous run not being archived before clearing state. Both resolved for full restart functionality.					

Project Name	**AIBI CV for Manufacturing**					
Test Case ID	TC-PUSR1-MAN-01					
Testing Tools Used	System UI, stopwatch, task checklist					
Testing Type	Manual Usability					
Execution Steps:	1	Recruit a new operator with no prior experience with the system.				
Execution Steps:	2	Give a 5-minute verbal overview of the system.				
Execution Steps:	3	Start a stopwatch timer.				
Execution Steps:	4	Operator completes the task checklist without assistance: launch app, perform scan, verify output, configure field mapping.				
Execution Steps:	5	Record time per task and total time.				
Execution Steps:	6	Stop timer at completion or at 30-minute mark.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Collin	2025-11-14	Operator completed in 38 minutes — spent 12 minutes on field mapping configuration due to unclear UI labels	FAIL	Configuration UI lacked descriptive labels and help text for field mapping	2025-11-25 by Jacob
2	Jacob	2025-11-28	Operator completed in 22 minutes after adding help text to	PASS	-	-

			configuration fields			
3	Nathan	2025-12-01	New operator completed all tasks in 19 minutes	PASS	-	-
Execution Summary	Total executions: 3. Pass rate: 67%. Initial onboarding exceeded 30 minutes due to unclear field mapping UI; adding descriptive help text reduced onboarding time to under 25 minutes consistently.					

Project Name	**AIBI CV for Manufacturing**					
Test Case ID	TC-PUSR2-MAN-01					
Testing Tools Used	Streamlit app, test users, questionnaire					
Testing Type	Manual Usability					
Execution Steps:	1	Present the Streamlit UI to a test user unfamiliar with the application.				
Execution Steps:	2	Ask the user to identify the purpose of 8 major controls using only on-screen text: mode selector, record button, stop button, finalize button, save button, load button, restart button, threshold sliders.				
Execution Steps:	3	Record correct and incorrect identifications for each control.				
Execution Steps:	4	Calculate the accuracy rate (correct / total).				
Execution Steps:	5	Repeat with additional test users.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Nathan	2025-11-15	User identified 4 of 8 controls correctly (50%) — finalize, save, load, and threshold sliders were unclear	FAIL	Buttons lacked descriptive labels; sliders had no help_text parameter set	2025-11-25 by Jacob
2	Jacob	2025-11-25	User identified 6 of 8 controls correctly (75%) after adding	PASS	-	-

			st.help_text to sliders and renaming buttons			
3	Daniel	2025-12-01	User identified 7 of 8 controls correctly (87.5%)	PASS	-	-
Execution Summary	Total executions: 3. Pass rate: 67%. Initial UI lacked adequate help text on sliders and descriptive button labels; adding st.help_text parameters and more descriptive labels improved identification accuracy from 50% to 87.5%.					

Project Name	**AIBI CV for Manufacturing**					
Test Case ID	TC-PUSR3-MAN-01					
Testing Tools Used	Legacy system documentation, new system, comparison sheet					
Testing Type	Manual Usability					
Execution Steps:	1	Document the legacy workflow steps for scanning three parts and recording data.				
Execution Steps:	2	Perform the same task on the new system.				
Execution Steps:	3	Count the number of manual inputs required on each system.				
Execution Steps:	4	Compare the step counts and calculate the percentage reduction.				
Execution Steps:	5	Verify the new system achieves at least 25% reduction in manual steps.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Jacob	2025-11-14	Legacy: 12 steps. New system: 5 steps. Reduction: 58%. Exceeds 25% threshold	PASS	-	-
2	Daniel	2025-11-29	Legacy: 12 steps. New system: 4 steps.	PASS	-	-

			Reduction: 67%			
3	Collin	2025-12-01	Legacy: 12 steps. New system: 5 steps. Reduction: 58%	PASS	-	-
Execution Summary	Total executions: 3. Pass rate: 100%. New system consistently achieved 58-67% reduction in manual steps compared to the legacy workflow, well exceeding the 25% threshold.					

Project Name	**AIBI CV for Manufacturing**					
Test Case ID	TC-PUSR4-MAN-01					
Testing Tools Used	Live system, operator observation					
Testing Type	Manual Usability					
Execution Steps:	1	Configure the system for a standard scanning workflow.				
Execution Steps:	2	Operator performs 10 consecutive scan cycles.				
Execution Steps:	3	An observer counts the number of UI touches (mouse clicks, keyboard presses) per scan cycle.				
Execution Steps:	4	Calculate the average number of UI interactions per cycle.				
Execution Steps:	5	Verify the average is 2 or fewer.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Nathan	2025-11-15	Average 3.2 UI touches per cycle — operator had to click "confirm" button after each scan event	FAIL	Scan confirmation required an explicit button click instead of auto-confirming on successful detection	2025-11-25 by Nathan
2	Nathan	2025-11-25	Average 1.4 UI touches per cycle after removing manual confirmation step	PASS	-	-

3	Collin	2025-12-01	Average 1.2 UI touches per cycle	PASS	-	-
Execution Summary	Total executions: 3. Pass rate: 67%. Manual confirmation button was adding unnecessary UI interactions; removing the explicit confirm step and auto-confirming on successful detection brought the average well under 2.					

Project Name	**AIBI CV for Manufacturing**					
Test Case ID	TC-PUSR10-MAN-01					
Testing Tools Used	UI screenshots, 3 test users					
Testing Type	Manual Usability					
Execution Steps:	1	Capture screenshots of the app in three states: all-pending (all empty box markers), mid-progress (mix of completed, current, and pending markers), and violation detected (red overlay/marker).				
Execution Steps:	2	Present each screenshot to 3 test users.				
Execution Steps:	3	Ask users to identify: which steps are done, which is current, which are pending, and whether an error/violation is shown.				
Execution Steps:	4	Record accuracy for each user across all states.				
Execution Steps:	5	Calculate overall accuracy (need at least 85%).				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Jacob	2025-11-16	Average accuracy 70% across 3 users — users confused current step marker with pending marker because both looked similar in small text	FAIL	Current step marker was too small and not color-differentiated from pending	2025-11-28 by Jacob
2	Jacob	2025-11-28	Average accuracy 88% after enlarging current step	PASS	-	-

			marker and adding blue color highlighting for current step			
3	Collin	2025-12-02	Average accuracy 92% across 3 users	PASS	-	-
Execution Summary	Total executions: 3. Pass rate: 67%. Users initially confused the current step marker with the pending marker; enlarging the current step indicator and adding color differentiation improved recognition accuracy from 70% to 92%.					

Project Name	**AIBI CV for Manufacturing**					
Test Case ID	TC-PPSR2-MAN-01					
Testing Tools Used	Live system, operator, response time monitoring					
Testing Type	Manual Non-Functional					
Execution Steps:	1	Set up the system under normal operating load.				
Execution Steps:	2	Operator performs 10 consecutive scan/verification cycles.				
Execution Steps:	3	Measure the time from operator action to visible UI feedback for each cycle.				
Execution Steps:	4	Ask the operator whether they perceived any delay.				
Execution Steps:	5	Record all response times and operator feedback.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Daniel	2025-11-14	Average response time 620 ms; operator reported noticeable lag between scan and result display	FAIL	UI was waiting for full JSON write to complete before showing detection result	2025-11-25 by Daniel
2	Daniel	2025-11-25	Average response time 180 ms after making JSON write asynchronous; operator reported "immediate" feel	PASS	-	-

3	Nathan	2025-12-01	Average response time 195 ms; operator reported no perceived delay	PASS	-	-
Execution Summary	Total executions: 3. Pass rate: 67%. UI was blocking on file I/O before displaying results; making JSON write asynchronous reduced perceived response time from 620 ms to under 200 ms.					

Project Name	**AIBI CV for Manufacturing**					
Test Case ID	TC-PPSR5-MAN-01					
Testing Tools Used	720p USB camera, test QR sheet, live testing					
Testing Type	Manual Non-Functional					
Execution Steps:	1	Connect a 720p USB camera to the system.				
Execution Steps:	2	Position the 5-QR-code test sheet at 40 cm from the camera.				
Execution Steps:	3	Run 20 detection cycles.				
Execution Steps:	4	Record the success rate (cycles where all codes were correctly detected).				
Execution Steps:	5	Verify the success rate is at least 95%.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Collin	2025-11-13	17 of 20 cycles successful (85%) — 3 cycles had partial detections due to camera auto-focus hunting at 720p	FAIL	Auto-focus instability at 720p caused intermittent blur during detection window	2025-11-24 by Collin
2	Collin	2025-11-24	19 of 20 cycles successful (95%) after adding frame stability check before detection	PASS	-	-

3	Jacob	2025-11-30	20 of 20 cycles successful (100%)	PASS	-	-
Execution Summary	Total executions: 3. Pass rate: 67%. Auto-focus hunting at 720p caused intermittent detection failures; adding a frame stability check before running detection eliminated the issue.					

Project Name	**AIBI CV for Manufacturing**					
Test Case ID	TC-PDSR1-MAN-01					
Testing Tools Used	Network monitoring tools (Wireshark), live system					
Testing Type	Manual Non-Functional					
Execution Steps:	1	Start Wireshark or equivalent network monitoring tool on the host machine.				
Execution Steps:	2	Launch the AIBI CV application and run it through a full operational cycle.				
Execution Steps:	3	Perform 20 or more scan events over a 1-hour period.				
Execution Steps:	4	Stop the network capture.				
Execution Steps:	5	Review all outbound connection logs and verify zero connections to external IP addresses or domains.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Nathan	2025-11-12	Wireshark captured 2 outbound HTTPS requests to pypi.org during application startup — pip check was running at launch	FAIL	requirements check script triggered outbound pip queries on startup	2025-11-24 by Nathan
2	Nathan	2025-11-24	Zero outbound connections during full 1-hour test after removing startup pip check	PASS	-	-

3	Daniel	2025-12-01	Zero external connections across 25 scan events over 1 hour	PASS	-	-
Execution Summary	Total executions: 3. Pass rate: 67%. A startup pip check was triggering outbound network requests; removing the startup dependency check eliminated all external traffic.					

Project Name	**AIBI CV for Manufacturing**	
Test Case ID	TC-PDSR2-MAN-01	
Testing Tools Used	Network disconnect, live system	
Testing Type	Manual Non-Functional	
Execution Steps:	1	Disconnect all internet connections (Ethernet and Wi-Fi) from the host machine.
Execution Steps:	2	Launch the AIBI CV application.
Execution Steps:	3	Perform 10 QR scan events and verify JSON output files are created.
Execution Steps:	4	Test keyboard emulation mode and verify keystrokes are sent correctly.
Execution Steps:	5	Load a trained process via file uploader and run step verification.
Execution Steps:	6	Continue operating for at least 1 hour.
Execution Steps:	7	Verify all features function identically to online operation.

Test Execution Records:

#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Jacob	2025-11-13	All features functional offline: 10 scans produced valid JSON, keyboard emulation worked, step verification operated normally for 1+ hour	PASS	-	-
2	Collin	2025-11-29	Full offline operation confirmed across all	PASS	-	-

			features for 1.5 hours			
3	Nathan	2025-12-01	All features verified offline; no degradation observed	PASS	-	-
Execution Summary	Total executions: 3. Pass rate: 100%. System operated fully offline with no feature degradation across all test runs. All processing is local as designed.					

Project Name	**AIBI CV for Manufacturing**					
Test Case ID	TC-PDSR4-MAN-01					
Testing Tools Used	Factory deployment environment, CPU/memory monitoring tools					
Testing Type	Manual Non-Functional					
Execution Steps:	1	Deploy the system in the factory or simulated factory environment.				
Execution Steps:	2	Start continuous operation with periodic scan events (approximately every 5 minutes).				
Execution Steps:	3	Monitor CPU and memory usage throughout the test period.				
Execution Steps:	4	Log any errors, crashes, or anomalies.				
Execution Steps:	5	Review monitoring data after 8 or more hours of continuous operation.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Daniel	2025-11-15	System crashed at 5.5 hours — memory usage climbed from 250 MB to 1.8 GB due to frame buffer not being cleared between scans	FAIL	Frame buffer accumulated raw frames without cleanup, causing memory growth	2025-11-25 by Daniel
2	Daniel	2025-11-25	System ran for 8.5 hours with stable memory (250-320	PASS	-	-

			MB range); zero crashes after adding frame buffer cleanup			
3	Collin	2025-12-03	9 hours continuous operation; CPU stable at 15-25%, memory stable at 280-310 MB, zero crashes	PASS	-	-
Execution Summary	Total executions: 3. Pass rate: 67%. Memory leak from uncleaned frame buffer caused a crash at 5.5 hours; implementing frame buffer cleanup between scans stabilized memory usage for 8+ hour operation.					

Project Name	**AIBI CV for Manufacturing**					
Test Case ID	TC-ODSR2-MAN-01					
Testing Tools Used	Documentation review, pip freeze output					
Testing Type	Manual Non-Functional					
Execution Steps:	1	Run pip freeze to generate the full list of installed dependencies.				
Execution Steps:	2	Compare the pip freeze output against the documented OSS component list.				
Execution Steps:	3	Verify each documented component has name, version, and license information.				
Execution Steps:	4	Identify any undocumented dependencies present in pip freeze but missing from documentation.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Jacob	2025-11-14	pip freeze showed 28 packages; documentation listed 22 — 6 transitive dependencies (e.g., Pillow,	FAIL	Transitive dependencies were not included in the OSS documentation	2025-11-25 by Collin

			urllib3) were undocumented			
2	Collin	2025-11-25	All 28 packages documented with name, version, and license after adding missing transitive deps	PASS	-	-
3	Nathan	2025-12-01	100% of pip freeze packages documented with complete information	PASS	-	-
Execution Summary	Total executions: 3. Pass rate: 67%. Six transitive dependencies were missing from OSS documentation; auditing and adding all transitive packages with their license information brought documentation to 100% coverage.					

Project Name	**AIBI CV for Manufacturing**					
Test Case ID	TC-EISR6-MAN-01					
Testing Tools Used	Configuration JSON files, field engineer					
Testing Type	Manual Non-Functional					
Execution Steps:	1	Run a scan with the initial configuration (part_number, serial_number fields).				
Execution Steps:	2	Verify the system enforces those fields in the output.				
Execution Steps:	3	Edit the workstation config JSON to change required fields to part_number, batch_id, inspector_id (no code changes).				
Execution Steps:	4	Restart the system.				
Execution Steps:	5	Run the same scan and verify the new fields are enforced and old fields no longer apply.				
Test Execution Records:						
#	Tester	Test Date	Actual Result	Status	Defect	Correction
1	Collin	2025-11-15	Initial config worked; edited config to add batch_id and inspector_id, restarted, but system still enforced old fields —	FAIL	ConfigManager cached config at first load and did not re-read from disk on restart	2025-11-25 by Collin

			config was cached in memory and not reloaded			
2	Collin	2025-11-25	Config changes reflected after restart; new fields enforced, old serial_number field no longer required	PASS	-	-
3	Daniel	2025-12-03	Config-driven customization verified: editing JSON and restarting changes workflow without code modifications	PASS	-	-
Execution Summary	Total executions: 3. Pass rate: 67%. ConfigManager was caching the config in memory and not re-reading on application restart; fixing the loader to read from disk on each startup resolved the issue.					